



T.C.

BURSA ULUDAĞ ÜNİVERSİTESİ
KARACABEY MESLEK YÜKSEKOKULU

THERAPP

Duygusal Destek ve Paylaşım Topluluğu

Hazırlayanlar

Gaye GÖRKEM — 262384049

Rabia Nur GÜN — 262484033

Dersin Yürütücüsü / Danışman

Öğr. Gör. Koray COŞKUN

BURSA — 2026

İÇİNDEKİLER

İÇİNDEKİLER	1
ÖZET	2
1. GİRİŞ	3
1.1. Konu	4
1.2. Amaç	4
1.3. Kapsam	4
1.4. Problem Tanımı	5
2. GELİŞME	5
2.1. Planlama	6
2.1.1. Proje Kapsamı ve Yaklaşımı	6
2.1.2. Kaynaklar ve Ekip Yapısı	6
2.1.3. Maliyet Kestirimi	6
2.1.4. Zaman Planı ve Gantt Şeması	7
2.2. Çözümleme (Analiz)	8
2.2.1. Gereksinim Analizi	9
2.2.2. Veri Akış Şemaları (VAD)	9
2.2.3. Karar Tabloları	11
2.3. Tasarım	12
2.3.1. Sistem Mimarisi	13
2.3.2. Arayüz Tasarımı ve Kağıt Prototip	13
2.3.3. Varlık-İlişki (ER) Modeli	14
2.3.4. Veritabanı Tabloları ve Veri Sözlüğü	14
2.4. Gerçekleştirim (Kodlama)	16
2.4.1. Kullanılan Diller ve Teknolojiler	17

2.4.2. Uygulama Mimarisi ve Kodlama Yaklaşımı	18
2.4.3. Kimlik Doğrulama Akışı	18
2.4.4. API Uç Noktaları	19
2.4.5. Kodlama Adımları	20
2.5. Bakım ve Destek	20
2.5.1. Güncellemeler ve Sürüm Yönetimi	21
2.5.2. Hata Düzeltme ve İzleme.....	21
2.5.3. Sürdürülebilirlik ve Gelecek Geliştirmeler.....	21
3. SONUÇ	21
EKRAN GÖRÜNTÜLERİ	23
KAYNAK KODLAR.....	26
Backend Kaynak Kodları (.NET 8 / C#)	27
Mobil Kaynak Kodları (React Native / TypeScript)	41
KAYNAKÇA.....	41

ÖZET

Bu proje, ruhsal iyi oluş ve duygusal destek alanında faaliyet gösteren, kullanıcıların yaşadıkları duygusal deneyimleri kategori bazlı ve dilerlerse anonim biçimde paylaşabildiği bir topluluk uygulaması olan Therapp'in analiz, tasarım ve gerçekleştirim sürecini kapsamaktadır. Uygulamanın amacı; yargılanma kaygısı olmadan paylaşım yapılabilecek güvenli bir alan sunmak, benzer deneyimleri okuyarak kullanıcıların yalnız olmadıklarını hissetmelerini sağlamak ve bunu yaparken içeriklerin tıbbi tavsiye olmadığını sürekli hatırlatan bir sorumluluk çerçevesi kurmaktır.

Yöntem olarak yazılım geliştirme yaşam döngüsü (planlama, çözümleme, tasarım, gerçekleştirim, bakım) izlenmiştir. Sistem iki bağımsız katmandan oluşur: (1) .NET 8 ve ASP.NET Core Web API ile Clean Architecture prensiplerine göre geliştirilmiş, PostgreSQL veritabanı kullanan bir arka uç servisi; (2) React Native (Expo SDK 54) ve TypeScript ile geliştirilmiş, REST API üzerinden bu servisle haberleşen bir mobil istemci. Kimlik doğrulama JWT (erişim + yenileme belirteci) ve BCrypt parola özütleme ile; girdi doğrulama FluentValidation ve Zod ile; sunucu-istemci veri senkronizasyonu TanStack React Query ile sağlanmıştır.

Sonuç olarak; kullanıcı kaydı/girişi, kategori bazlı gönderi akışı (en yeni/popüler), anonim paylaşım, yorum ve yanıtlar, beğeni/kaydetme, bildirimler, içerik/kullanıcı raporlama ve moderasyon altyapısı ile acil destek yönlendirmelerini içeren, uçtan uca çalışan bir prototip ortaya konmuştur. Geliştirme sırasında karşılaşılan gerçek dünya entegrasyon sorunları (yerel ağ erişimi, modül döngüsü, oturum yenileme) sistematik biçimde çözülmüştür. Rapor; planlama, veri akış şemaları, karar tabloları, varlık-ilişki modeli, veri sözlüğü, sistem mimarisi, kimlik doğrulama akışı ve seçilmiş kaynak kodları ile bu süreci belgelemektedir.

Anahtar Kelimeler: ruhsal sağlık, duygusal destek, mobil uygulama, React Native, ASP.NET Core, Clean Architecture, PostgreSQL, JWT, REST API, anonim paylaşım.

1. GİRİŞ

1.1. Konu

Bu raporun konusu; kullanıcıların anksiyete, depresyon, panik atak, yalnızlık, stres, ilişkiler, öz bakım ve uyku gibi başlıklar altında duygusal deneyimlerini paylaştığı, birbirlerine destek verdiği bir mobil topluluk uygulaması olan Therapp'in sistem analizi ve tasarımıdır. Uygulama, bir mobil istemci ve bunu besleyen bir arka uç (backend) web servisinden oluşan iki katmanlı bir yazılım sistemidir.

1.2. Amaç

Projenin temel amacı, ruhsal iyi oluş konusunda insanların kendilerini güvende ve yargılanmadan hissedecekleri dijital bir paylaşım alanı oluşturmaktır. Bu genel amaç aşağıdaki alt hedeflere ayrılmıştır:

- Kategori bazlı, sakin ve düzenli bir paylaşım ortamı sunmak.
- Anonim paylaşım seçeneğiyle yargılanma kaygısını azaltmak.
- Benzer deneyimlerin okunmasını sağlayarak kullanıcıların yalnız olmadıklarını hissettirmek.
- İçeriklerin tıbbi tavsiye olmadığını açıkça belirten uyarılar ve acil destek kaynakları ile güvenli bir kullanım çerçevesi kurmak.
- Raporlama, gizleme ve moderasyon altyapısıyla zararlı içerik riskini yönetmek.

1.3. Kapsam

Proje kapsamına; kullanıcı kimlik doğrulama (kayıt, giriş, oturum yenileme, çıkış), profil yönetimi, gönderi oluşturma/güncelleme/silme, kategori bazlı ve genel akış (en yeni/popüler sıralama, sayfalama), yorum ve yanıt sistemi, beğeni ve kaydetme etkileşimleri, bildirimler, içerik/kullanıcı raporlama ve temel moderasyon işlevleri dâhildir. Kapsam dışı bırakılan (gelecek sürümlere ertelenen) unsurlar; anlık push bildirimleri, görsel/medya yükleme, arama,

kapsamlı yönetici paneli, çoklu dil ve bulut dağıtımıdır. Bu unsurlar 2.5 ve 3. bölümlerde gelecek geliştirmeler olarak ele alınmıştır.

1.4. Problem Tanımı

Ruhsal zorluklar yaşayan bireyler çoğu zaman duygularını güvenle paylaşabilecekleri bir alan bulamamakta; sosyal medyada yargılanma, etiketlenme veya kimliklerinin açığa çıkması kaygısı yaşamaktadır. Ayrıca bu tür platformlarda zararlı veya yanlış yönlendirici içeriklerle karşılaşma riski bulunmaktadır. Therapp bu probleme; kategori bazlı düzen, isteğe bağlı anonimlik, raporlama/moderasyon altyapısı ve içeriklerin tıbbi tavsiye olmadığını vurgulayan kalıcı uyarılar ile 112 gibi acil destek yönlendirmeleri sunarak yanıt vermektedir. Aşağıdaki tablo, tanımlanan problemleri ve uygulamanın bunlara karşılık gelen çözüm yaklaşımını özetlemektedir.

Tablo 1: Problem tanımı ve çözüm yaklaşımı

Tanımlanan Problem	Therapp'in Çözüm Yaklaşımı
Duyguları paylaşacak güvenli bir alanın olmayışı	Kategori bazlı, sade ve sakin paylaşım ortamı
Yargılanma / kimliğin açığa çıkma korkusu	İsteğe bağlı anonim paylaşım seçeneği
Yalnız hissetme	Benzer deneyimleri okuma ve topluluk ile bağ kurma
Zararlı veya uygunsuz içerik riski	Raporlama, içerik gizleme ve moderasyon altyapısı
Yanlış tıbbi yönlendirme riski	Kalıcı uyarı (disclaimer) + acil destek kaynakları (112)

2. GELİŞME

2.1. Planlama

2.1.1. Proje Kapsamı ve Yaklaşımı

Proje, yinelemeli (iteratif) bir yaklaşımla ve yazılım geliştirme yaşam döngüsünün planlama, çözümleme, tasarım, gerçekleştirim ve bakım aşamaları izlenerek yürütülmüştür. Öncelikle ihtiyaçlar belirlenmiş, ardından veri modeli ve mimari tasarlanmış, geliştirme arka uçtan başlayıp mobil istemci ile devam etmiş ve süreç test/dokümantasyon ile tamamlanmıştır.

2.1.2. Kaynaklar ve Ekip Yapısı

Proje iki kişilik bir öğrenci ekibi tarafından geliştirilmiştir. Sorumluluklar, uçtan uca katkı sağlanacak biçimde paylaşılmış; her iki üye de hem arka uç hem mobil tarafta çalışmıştır.

Tablo 2: Ekip yapısı ve başlıca sorumluluklar

Ekip Üyesi	Başlıca Sorumluluk Alanları
Gaye GÖRKEM	Backend API geliştirme, veritabanı tasarımı, kimlik doğrulama, dokümantasyon
Rabia Nur GÜN	Mobil arayüz ve ekranlar, durum yönetimi, API entegrasyonu, test

Kullanılan başlıca kaynaklar; geliştirme bilgisayarları, Visual Studio / VS Code, .NET 8 SDK, Node.js ve Expo araç zinciri, PostgreSQL veritabanı, Git sürüm kontrolü ve fiziksel test için Expo Go yüklü bir akıllı telefondur. Kullanılan yazılımların tamamı ücretsiz ve/veya açık kaynaklıdır.

2.1.3. Maliyet Kestirimi

Bir öğrenci projesi olması nedeniyle doğrudan yazılım/lisans maliyeti oluşmamıştır; kullanılan tüm araçlar ücretsiz veya açık kaynaklıdır. Aşağıdaki tablo, projenin adam-saat cinsinden eforuna dayalı temsili (kavramsal) bir maliyet

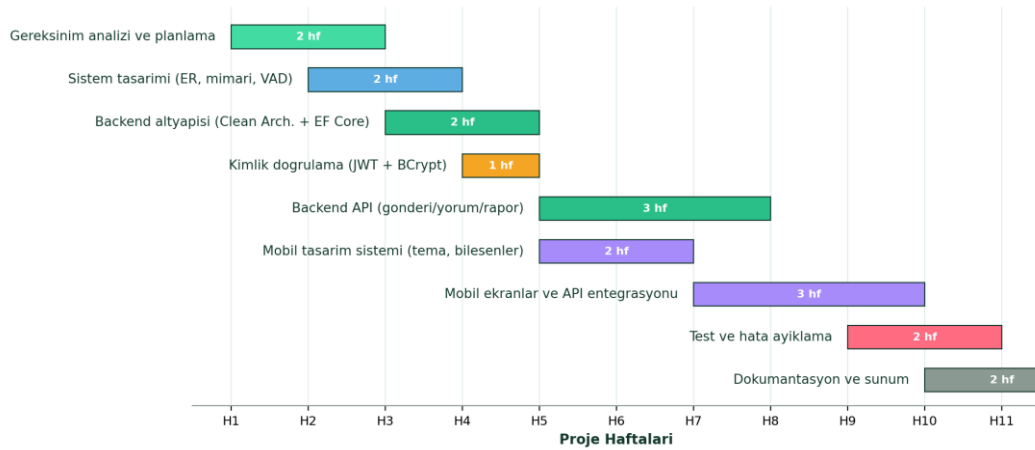
kestirimi sunar. Saatlik ücret, ülkemizdeki junior yazılım geliştirici için varsayımsal 250 TL/saat olarak alınmıştır.

Tablo 3: Temsilî maliyet kestirimi (adam-saat esaslı)

Kalem	Tahmini Efor (saat)	Birim (TL/saat)	Tutar (TL)
Analiz ve tasarım	40	250	10.000
Backend geliştirme	90	250	22.500
Mobil geliştirme	100	250	25.000
Test ve hata ayıklama	30	250	7.500
Dokümantasyon ve sunum	20	250	5.000
Yazılım / lisans (açık kaynak)	—	—	0
TOPLAM	280	—	70.000

2.1.4. Zaman Planı ve Gantt Şeması

Proje yaklaşık 11 haftalık bir dönem içinde planlanmıştır. Aşağıdaki Gantt şeması, ana iş paketlerinin zamana yayılımını ve örtüşmelerini göstermektedir. Bazı iş paketleri (örneğin backend API ile mobil tasarım sistemi) paralel yürütülmüştür.



Şekil 1: Proje zaman planı — Gantt şeması

Haftalık ilerleyiş planı özet olarak aşağıdaki gibidir:

Tablo 4: Haftalık ilerleyiş planı

Hafta	Yapılan İş
1	Gereksinim analizi, konu ve kapsam belirleme
2	Sistem tasarımı: ER modeli, mimari ve veri akış şemaları
3	Backend altyapısı: Clean Architecture katmanları, EF Core, veritabanı
4	Kimlik doğrulama: JWT üretimi/doğrulaması, BCrypt, oturum yenileme
5-7	Backend API (gönderi, yorum, beğeni, bildirim, rapor) ve mobil tasarım sistemi
7-9	Mobil ekranlar ve API entegrasyonu, durum yönetimi
9-10	Uçtan uca test, hata ayıklama, entegrasyon sorunlarının çözümü
10-11	Dokümantasyon, rapor ve sunum hazırlığı

2.2. Çözümleme (Analiz)

2.2.1. Gereksinim Analizi

Sistem gereksinimleri, fonksiyonel (sistemin ne yapması gerektiği) ve fonksiyonel olmayan (sistemin hangi niteliklere sahip olması gerektiği) olmak üzere iki grupta çözümlenmiştir.

Tablo 5: Fonksiyonel gereksinimler

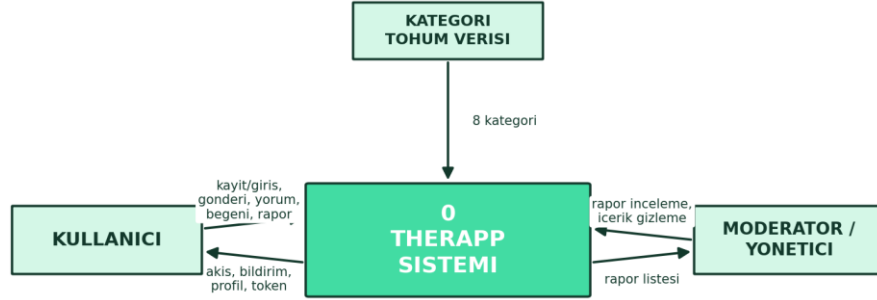
No	Gereksinim
FG-01	Kullanıcı e-posta/parola ile kayıt olabilmeli ve giriş yapabilmeli.
FG-02	Oturum, erişim belirtecinin süresi dolduğunda otomatik yenilenebilmeli.
FG-03	Kullanıcı gönderi oluşturabilmeli, güncelleyebilmeli ve silebilmeli.
FG-04	Gönderi isteğe bağlı olarak anonim paylaşılabilir.
FG-05	Gönderiler kategoriye ve sıralamaya (en yeni/popüler) göre listelenebilmeli.
FG-06	Kullanıcı gönderilere yorum ve yorumlara yanıt yazabilmeli.
FG-07	Kullanıcı gönderi/yorum beğenebilmeli ve gönderi kaydedebilmeli.
FG-08	Beğeni/yorum durumunda ilgili kullanıcıya bildirim üretilmeli.
FG-09	Kullanıcı uygunsuz içerik veya kullanıcıyı raporlayabilmeli.
FG-10	Kullanıcı profilini düzenleyebilmeli ve hesabını silebilmeli.

Tablo 6: Fonksiyonel olmayan gereksinimler

No	Gereksinim
FOG-01	Güvenlik: Parolalar BCrypt ile özetlenmeli, düz metin saklanmamalı.
FOG-02	Güvenlik: Kaynaklar JWT ile korunmalı; belirteçler cihazda güvenli saklanmalı.
FOG-03	Performans: Akış sayfalama ve indekslerle verimli sunulmalı.
FOG-04	Kullanılabilirlik: Arayüz sade, Türkçe ve tutarlı bir tasarım sistemine sahip olmalı.
FOG-05	Güvenilirlik: Girdi doğrulama sunucu ve istemci tarafında yapılmalı.
FOG-06	Sürdürülebilirlik: Kod katmanlı (Clean Architecture) ve bakımı kolay olmalı.
FOG-07	Etik/Yasal: İçeriklerin tıbbi tavsiye olmadığı sürekli belirtilmeli.

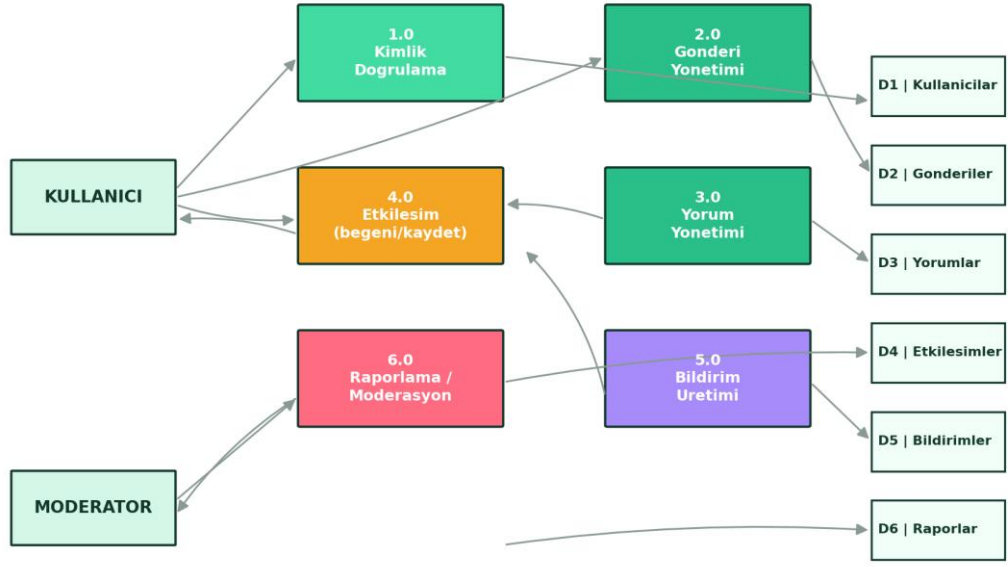
2.2.2. Veri Akış Şemaları (VAD)

Sistemin veri akışı, önce en üst düzeyde bağlam (Seviye 0) diyagramı ile, ardından ana süreçlere ayrıştırılarak (Seviye 1) modellenmiştir. Bağlam diyagramında sistem tek bir süreç olarak ele alınır ve dış varlıklar (Kullanıcı, Moderatör, kategori tohum verisi) ile arasındaki veri akışları gösterilir.



Şekil 2: Veri akış diyagramı — Seviye 0 (bağlam diyagramı)

Seviye 1 diyagramı, sistemi altı ana sürece ayırır: kimlik doğrulama, gönderi yönetimi, yorum yönetimi, etkileşim (beğeni/kaydetme), bildirim üretimi ve raporlama/moderasyon. Diyagram ayrıca verilerin saklandığı depoları (kullanıcılar, gönderiler, yorumlar, etkileşimler, bildirimler, raporlar) gösterir.



Şekil 3: Veri akış diyagramı — Seviye 1

2.2.3. Karar Tabloları

Sistemdeki koşullu iş kuralları, karar tabloları ile netleştirilmiştir. İlk tablo, bir gönderinin oluşturulabilmesi için gereken koşulları ve sonuçlarını gösterir (E: Evet, H: Hayır).

Tablo 7: Karar tablosu — Gönderi oluşturma doğrulaması

Koşul / Kural	K1	K2	K3	K4	K5
Oturum açık (kimlik doğrulanmış)	H	E	E	E	E
Geçerli kategori seçilmiş	-	H	E	E	E
Başlık 3-150 karakter	-	-	H	E	E
İçerik 10-5000 karakter	-	-	-	H	E
SONUÇ	401	Hata	Hata	Hata	Kaydet

İkinci tablo, bir gönderinin silinmesinde yetkilendirme kuralını modellemektedir.

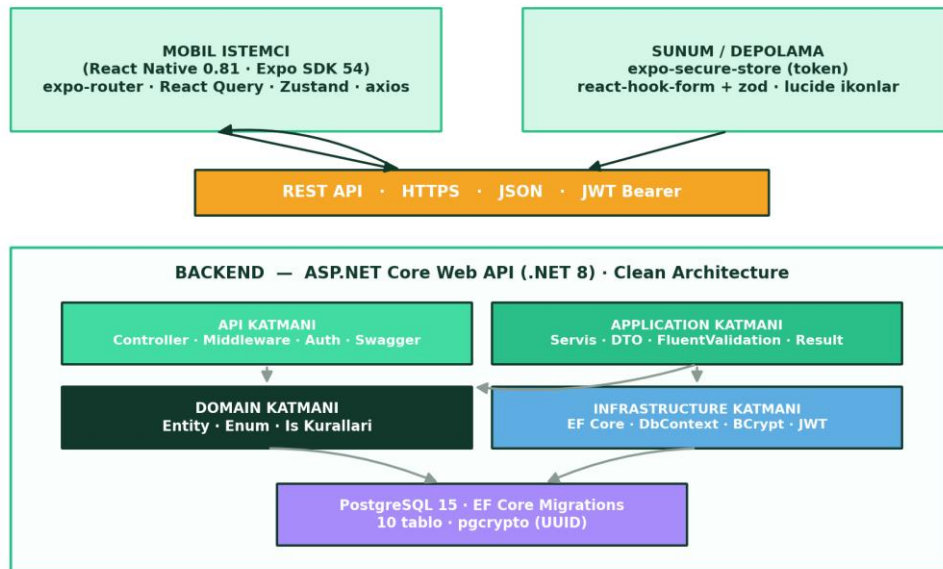
Tablo 8: Karar tablosu — Gönderi silme yetkilendirmesi

Koşul / Kural	K1	K2	K3
İstek sahibi gönderinin sahibi mi?	E	H	H
İstek sahibi Moderatör/Yönetici mi?	-	E	H
SONUÇ	Sil (soft delete)	Sil (soft delete)	403 Yetkisiz

2.3. Tasarım

2.3.1. Sistem Mimarisi

Sistem, birbirinden bağımsız iki ana bileşenden oluşur: REST API üzerinden haberleşen bir mobil istemci ve bir arka uç servisi. Arka uç, bağımlılıkların içeri (Domain'e) doğru aktığı Clean Architecture yaklaşımıyla dört katmana ayrılmıştır: API (sunum), Application (iş mantığı/servisler), Domain (varlıklar ve iş kuralları) ve Infrastructure (veritabanı, güvenlik). Bu ayırım, iş kurallarını çerçeve ve altyapı ayrıntılarından yalıtarak test edilebilirliği ve bakımı kolaylaştırır [8].



Şekil 4: Sistem mimarisi — katmanlı yapı ve teknoloji yığını

2.3.2. Arayüz Tasarımı ve Kağıt Prototip

Arayüz tasarımı, merkezi bir tasarım sistemi (renk paleti, boşluk, köşe yarıçapı, tipografi ölçeği) üzerine kurulmuştur. Ana renk sakinlik çağrıştıran nane yeşilidir (#42DCA3). Ekranlar başlangıçta düşük maliyetli kağıt/eskiz prototipleri olarak tasarlanmış; onaylanan akış, aşağıdaki uygulama ekranlarına dönüştürülmüştür. Uygulamanın temel ekran akışı ve gezinme yapısı şu şekildedir:

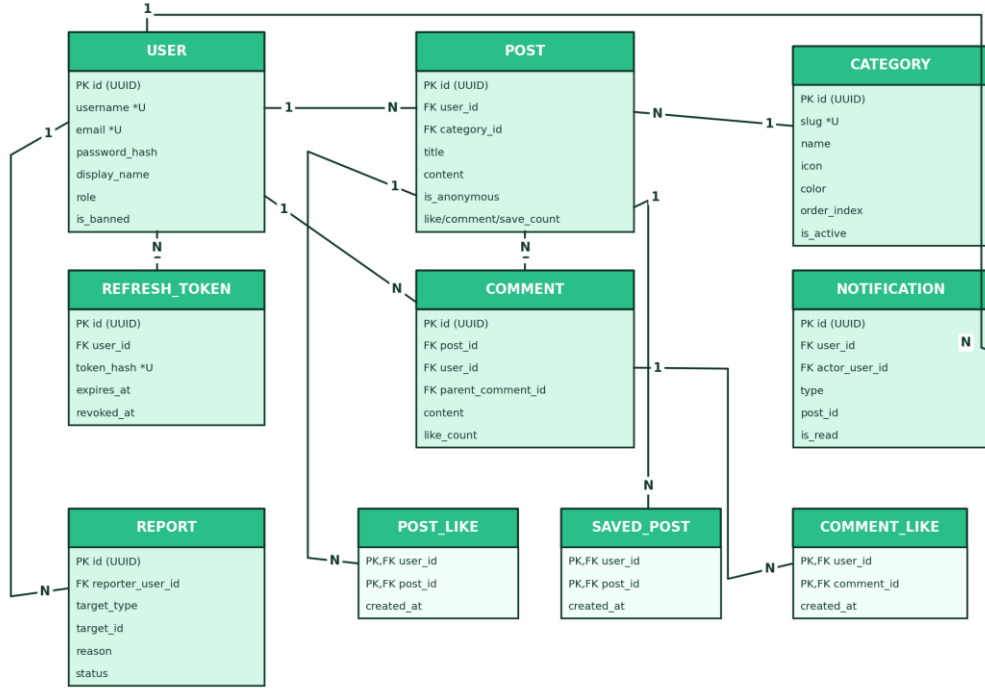
- Onboarding → Giriş/Kayıt → Ana sekmeler (Akış, Kategoriler, Paylaş, Bildirim, Profil).

- Akış ekranından gönderi detayına, oradan yorum ve yanıtlara geçiş.
- Paylaş akışında kategori seçimi, başlık/içerik ve anonimlik tercihi.
- Profil ve Ayarlar ekranlarından profil düzenleme, acil destek ve hesap işlemleri.

Gerçek uygulama ekran görüntüleri, raporun sonundaki "Ekran Görüntüleri" bölümünde açıklamalı olarak sunulmuştur.

2.3.3. Varlık-İlişki (ER) Modeli

Veritabanı, on varlık (tablo) etrafında modellenmiştir. Kullanıcı (User), Gönderi (Post), Kategori (Category), Yorum (Comment), gönderi/yorum beğenileri ve kaydetmeler için üç ara tablo (PostLike, CommentLike, SavedPost), Rapor (Report), Bildirim (Notification) ve Yenileme Belirteci (RefreshToken). Çoktan-çoğa ilişkiler (bir kullanıcının çok gönderiyi beğenmesi vb.) bileşik birincil anahtara sahip ara tablolarla çözülmüştür. Tüm birincil anahtarlar UUID'dir.



* U = benzersiz (unique) | PK = birincil anahtar | FK = yabancı anahtar | ara tablolar (POST_LIKE, SAVED_POST, COMMENT_LIKE) bileşik PK ile M:N cozer
Not: Tüm ara tablolar ayrıca USER'a (user_id) bağlıdır; NOTIFICATION hem alıcı (user_id) hem aktör (actor_user_id) ile USER'a bağlıdır.

Şekil 5: Varlık-İlişki (ER) modeli

2.3.4. Veritabanı Tabloları ve Veri Sözlüğü

Aşağıdaki tablolar, temel varlıkların alanlarını, veri türlerini ve açıklamalarını içeren veri sözlüğünü sunar. Veritabanı PostgreSQL'dir; birincil anahtarlar `gen_random_uuid()` ile üretilir, tablo/kolon adları `snake_case` biçimindedir ve numaralandırma (enum) alanları short olarak saklanır.

Tablo 9: Veri sözlüğü — users tablosu

Alan	Tür	Kısıt	Açıklama
id	UUID	PK	Birincil anahtar
username	varchar(32)	Benzersiz, zorunlu	Kullanıcı adı
email	varchar(255)	Benzersiz, zorunlu	E-posta
password_hash	varchar(255)	Zorunlu	BCrypt parola özü
display_name	varchar(64)	Zorunlu	Görünen ad
avatar_url	varchar(500)	Boş olabilir	Profil görseli adresi
bio	varchar(280)	Boş olabilir	Hakkında metni
role	short	Zorunlu	Rol (User/Moderator/Admin)
is_banned	boolean	Zorunlu	Yasaklı mı
created_at / updated_at	timestampz	Zorunlu	Oluşturma/güncelleme zamanı
deleted_at	timestampz	Boş olabilir	Soft delete zaman damgası

Tablo 10: Veri sözlüğü — posts tablosu

Alan	Tür	Kısıt	Açıklama
id	UUID	PK	Birincil anahtar
user_id	UUID	FK → users	Gönderiyi oluşturan kullanıcı
category_id	UUID	FK → categories	Gönderi kategorisi
title	varchar(150)	Zorunlu	Başlık (3-150 karakter)
content	text	Zorunlu	İçerik (10-5000 karakter)
is_anonymous	boolean	Zorunlu	Anonim paylaşım mı
like_count	int	Zorunlu	Beğeni sayacı (denormalize)
comment_count	int	Zorunlu	Yorum sayacı (denormalize)
save_count	int	Zorunlu	Kaydetme sayacı

			(denormalize)
is_hidden	boolean	Zorunlu	Moderasyonla gizlenmiş mi
created_at / updated_at / deleted_at	timestampz	—	Zaman damgaları / soft delete

Tablo 11: Veri sözlüğü — categories, comments ve reports (seçilmiş alanlar)

Tablo	Öne Çıkan Alanlar	Açıklama
categories	slug (benzersiz), name, icon, color, order_index, is_active	8 hazır kategori (tohumlanır)
comments	post_id (FK), user_id (FK), parent_comment_id (FK), content, like_count	Tek düzey yanıt destekli yorum
reports	reporter_user_id (FK), target_type, target_id, reason, status	İçerik/kullanıcı raporları
notifications	user_id (FK), actor_user_id (FK), type, post_id, is_read	Beğeni/yorum/yanıt bildirimleri
refresh_tokens	user_id (FK), token_hash (benzersiz), expires_at, revoked_at	SHA-256 özlütlü yenileme belirteçleri

2.4. Gerçekleştirim (Kodlama)

2.4.1. Kullanılan Diller ve Teknolojiler

Projede, arka uç ve mobil taraf için ayrı ancak REST üzerinden entegre iki teknoloji yığını kullanılmıştır. Aşağıdaki tablolar başlıca teknolojileri ve sürümlerini listeler.

Tablo 12: Backend teknoloji yığını

Teknoloji	Sürüm	Amaç
C# / .NET	8.0	Programlama dili ve çalışma zamanı
ASP.NET Core Web API	8.0	REST API çatısı
Entity Framework Core	8.0.10	ORM / veritabanı erişimi
Npgsql (PostgreSQL sağlayıcısı)	8.0.10	PostgreSQL bağlantısı
PostgreSQL	15	İlişkisel veritabanı
JWT Bearer / IdentityModel	8.0.10 / 7.6.0	Belirteç tabanlı kimlik doğrulama
BCrypt.Net-Next	4.0.3	Parola özütleme (work factor 11)
FluentValidation	11.9.2	Sunucu tarafı girdi doğrulama
Serilog	8.0.2	Yapılandırılmış günlükleme
Swashbuckle (Swagger)	6.7.3	API dokümantasyonu (OpenAPI)

Tablo 13: Mobil teknoloji yığını

Teknoloji	Sürüm	Amaç
React Native	0.81	Çapraz platform mobil çatı
Expo (SDK)	54	Geliştirme/derleme araç zinciri
TypeScript	5.3	Tip güvenli JavaScript
expo-router	6.0	Dosya tabanlı yönlendirme
TanStack React Query	5.59	Sunucu durumu / önbellek yönetimi

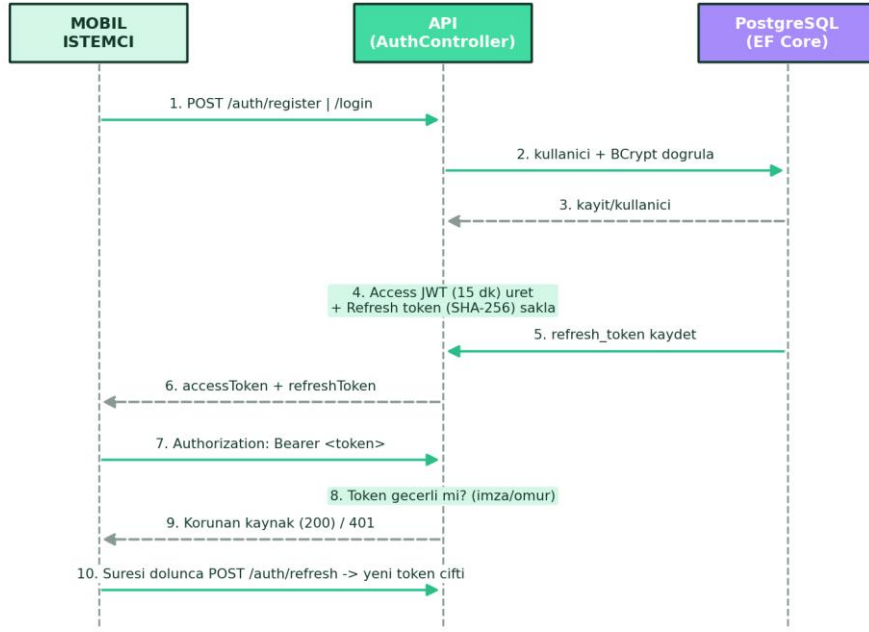
Zustand	5.0	İstemci durumu yönetimi
axios	1.7	HTTP istemcisi
react-hook-form + Zod	7.53 / 3.23	Form yönetimi ve doğrulama
expo-secure-store	15.0	Belirteçlerin güvenli saklanması

2.4.2. Uygulama Mimarisi ve Kodlama Yaklaşımı

Backend, Clean Architecture katmanlarına ayrılmıştır. İstekler API katmanındaki denetleyicilere (Controller) gelir; iş mantığı Application katmanındaki servislerde yürütülür; veriler Infrastructure katmanındaki EF Core DbContext üzerinden PostgreSQL'e yazılır. Hata yönetimi için Result deseni ve merkezi bir istisna yakalama ara katmanı (middleware) kullanılmıştır. Girdi doğrulama FluentValidation ile otomatik yapılır. Mobil tarafta ise sunucu durumu React Query, oturum/kullanıcı durumu Zustand ile yönetilir; API çağrıları merkezi bir axios istemcisi üzerinden yapılır ve 401 durumunda oturum otomatik yenilenir.

2.4.3. Kimlik Doğrulama Akışı

Kimlik doğrulama, JWT tabanlıdır. Başarılı kayıt/giriş sonrasında sunucu, 15 dakika ömürlü bir erişim belirteci (access token) ile 30 gün ömürlü bir yenileme belirteci (refresh token) üretir. Yenileme belirtecinin yalnızca SHA-256 özü veritabanında saklanır. Korunan kaynaklara erişimde belirteç Authorization başlığında gönderilir; süresi dolduğunda istemci yenileme belirteciyle yeni bir belirteç çifti alır (belirteç rotasyonu). Bu akış aşağıda gösterilmiştir.



Şekil 6: JWT kimlik doğrulama ve belirteç yenileme akışı

2.4.4. API Uç Noktaları

REST API, kaynak bazlı denetleyicilere ayrılmıştır. Kimlik doğrulama uç noktaları hariç tüm uç noktalar JWT ile korunur. Başlıca uç noktalar aşağıdaki tabloda özetlenmiştir.

Tablo 14: Başlıca API uç noktaları

Yöntem ve Yol	Açıklama
POST /api/auth/register	Kayıt ol (belirteç çifti döner)
POST /api/auth/login	Giriş yap
POST /api/auth/refresh	Belirteç yenile (rotasyon)
GET /api/posts	Akış (sayfalama + en yeni/popüler sıralama)
GET /api/posts/category/{slug}	Kategoriye göre gönderiler
POST /api/posts	Gönderi oluştur
POST /api/posts/{id}/like	Gönderi beğen / (DELETE) beğeni geri al
POST /api/posts/{id}/save	Gönderi kaydet / (DELETE) kaydı kaldır
GET /api/posts/{postId}/comments	Gönderi yorumlarını getir

POST /api/comments/{id}/replies	Yoruma yanıt yaz
GET /api/notifications	Bildirimleri listele
POST /api/reports	İçerik/kullanıcı raporla
GET /api/users/me	Profil bilgisi (gönderi/kaydetme sayıları ile)

2.4.5. Kodlama Adımları

Gerçekleştirim, aşağıdaki sıra izlenerek yapılmıştır: (1) Domain varlıklarının ve iş kurallarının tanımlanması; (2) EF Core ile veritabanı bağlamı, ilişki yapılandırmaları ve göçlerin (migration) oluşturulması; (3) kimlik doğrulama servisleri (JWT + BCrypt); (4) kaynak servisleri ve denetleyicileri (gönderi, yorum, beğeni, bildirim, rapor); (5) doğrulayıcılar ve merkezi hata yönetimi; (6) mobil tasarım sistemi ve ortak bileşenler; (7) API istemcisi ve durum yönetimi; (8) ekranların yazılması ve entegrasyon; (9) test ve hata ayıklama. Seçilmiş kaynak kodları raporun "Kaynak Kodlar" bölümünde sunulmuştur.

2.5. Bakım ve Destek

2.5.1. Güncellemeler ve Sürüm Yönetimi

Kaynak kodu Git ile sürümlenmektedir. Uygulamanın mevcut sürümü 1.0.0'dır. Veritabanı şemasındaki değişiklikler EF Core göçleri (migration) ile yönetilir; uygulama açılışta bekleyen göçleri otomatik uygular ve kategori tohum verisini (idempotent biçimde) yükler. Böylece yeni bir ortama kurulum tek adımda ve tutarlı biçimde yapılabilir.

2.5.2. Hata Düzeltme ve İzleme

Sunucu tarafında Serilog ile yapılandırılmış günlükleme yapılır; istekler ve istisnalar merkezi bir ara katmanda yakalanarak tutarlı hata yanıtları üretilir. Mobil tarafta ağ ve kimlik doğrulama hataları merkezi istemcide ele alınır ve kullanıcıya anlaşılır Türkçe mesajlar gösterilir. Bu yapı, hataların hızlı tespit ve düzeltilmesini kolaylaştırır.

2.5.3. Sürdürülebilirlik ve Gelecek Geliştirmeler

Katmanlı mimari, tip güvenliği (TypeScript, C# nullable) ve merkezî doğrulama, sistemin sürdürülebilirliğini artırır. Planlanan gelecek geliştirmeler şunlardır:

- Anlık (push) bildirimler.
- Görsel/medya yükleme desteği ve içerik arama.
- Kapsamlı yönetici/moderatör paneli.
- Koyu tema ve çoklu dil desteği.
- Bulut dağıtımı ve otomatik testlerin (CI/CD) eklenmesi.

3. SONUÇ

Bu proje kapsamında, ruhsal iyi oluş alanında kullanıcıların kategori bazlı ve isteğe bağlı anonim biçimde paylaşım yapabildiği, birbirlerine yorum ve beğeni ile destek olabildiği, uçtan uca çalışan bir mobil topluluk uygulaması (Therapp) analiz edilmiş, tasarlanmış ve gerçekleştirilmiştir. Başta belirlenen hedeflerin tamamına (güvenli paylaşım alanı, anonimlik, moderasyon altyapısı, tıbbi uyarı ve acil destek) ulaşılmıştır.

Elde edilen başlıca bulgular; Clean Architecture ve katmanlı yapının geliştirme ve bakımı belirgin biçimde kolaylaştırdığı, JWT + yenileme belirteci deseninin mobil oturum yönetiminde hem güvenli hem kullanıcı dostu bir deneyim sağladığı ve React Query ile sunucu durumunun verimli yönetildiğidir. Geliştirme sürecinde karşılaşılan sorunlar ve çözümleri aşağıdaki tabloda özetlenmiştir.

Tablo 15: Karşılaşılan sorunlar ve çözümleri

Sorun	Çözüm
Telefonun yerel ağda backend'e erişememesi	Sunucunun 0.0.0.0 üzerinden dinlemesi ve doğru yerel IP'nin kullanılması
Expo Go sürüm uyumsuzluğu (SDK 52 → 54)	React 19 / React Native 0.81'e yükseltme
axios ile durum deposu arasında modül döngüsü	Deponun başlangıçta işleyicileri kaydettiği (configureAuth) desen
Yeniden açılışta uygulamanın yükleme ekranında takılması	Oturum durumuna göre yönlendiren AuthGate bileşeni
Gizli anahtarların korunması	.NET user secrets ve .env dosyaları ile yapılandırma dışlaştırma

Sonuç olarak Therapp, tanımlanan probleme somut ve etik bir çözüm sunan, ölçeklenebilir bir temele sahip bir prototip olarak ortaya konmuştur. Belirtilen gelecek geliştirmelerle birlikte uygulamanın gerçek kullanıma hazır bir ürüne dönüştürülmesi mümkündür. Proje, sistem analizi ve tasarımı sürecinin (planlama,

özümleme, tasarım, gerekleřtirim, bakım) uçtan uca uygulanması açısından da öğretici bir deneyim olmuřtur.

EKRAN GÖRÜNTÜLERİ

Bu bölümde uygulamanın başlıca ekranlarının gerçek görüntüleri açıklamalarıyla sunulmuştur.



Şekil 7: Ana akış (Akış) ekranı — en yeni/popüler sıralama ve tıbbi uyarı bandı



Şekil 8: Kategoriler ekranı — sekiz hazır kategori ve gönderi sayıları

×

Paylaş

Kategori

Anksiyete Depresyon Panik Atak Yalnızlık

Stres İlişkiler Öz Bakım Uyku

Başlık

Kısa bir başlık yaz

0/150

İçerik

Bugün nasıl hissediyorsun? Yaşadıklarını paylaşmak iyi gelir...

0/5000 (en az 10)

Anonim paylaş

İsmin görünmez, sadece "Anonim" yazar.

Paylaş

Şekil 9: Paylaş ekranı — kategori seçimi, başlık/içerik ve anonim paylaşım seçeneği

20:00

Profil

T

Test Kullanici

@testuser

1 Gönderi 0 Kaydedilen

Paylaşımlarım Kaydettiklerim

Anksiyete 19 saat önce

T Test Kullanici

Dünya güvenli değil

dünya güvenli değil

1 0 0

Akış Kategoriler Paylaş Bildirim Profil

Şekil 10: Profil ekranı — kullanıcı bilgileri, paylaşımlar ve kaydedilenler



Şekil 11: Ayarlar ekranı — profil düzenleme, acil destek kaynakları ve hesap işlemleri

KAYNAK KODLAR

Bu bölümde projeye ait tüm kaynak kodları dosya bazında, tablo hücreleri içinde sunulmuştur. Her tablonun koyu renkli başlık satırı ilgili dosyanın proje içindeki yolunu (backend/ veya mobile/ altında) gösterir. Kod dosyalarının başına, öğrencilerin ad-soyad ve numara bilgilerini içeren bir yorum satırı eklenmiştir. Yalnızca otomatik üretilen ve derleme sırasında yeniden oluşturulan dosyalar (package-lock.json bağımlılık kilidi ile EF Core migration anlık-görüntüleri *.Designer.cs ve AppDbContextModelSnapshot.cs) rapor hacmini korumak amacıyla kapsam dışıdır. Kaynak kodların tamamı ayrıca proje deposunda da yer almaktadır.

Backend Kaynak Kodları (.NET 8 / C#)

backend/Directory.Build.props

```
<Project>
  <PropertyGroup>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>false</TreatWarningsAsErrors>
    <GenerateDocumentationFile>false</GenerateDocumentationFile>
  </PropertyGroup>
</Project>
```

backend/Therapp.sln

```
Microsoft Visual Studio Solution File, Format Version 12.00
# Visual Studio Version 17
VisualStudioVersion = 17.0.31903.59
MinimumVisualStudioVersion = 10.0.40219.1
Project("{2150E333-8FDC-42A3-9474-1A3956D46DE8}") = "src", "src", "{A1B2C3D4-1111-2222-3333-444455556666}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "Therapp.Domain",
"src\Therapp.Domain\Therapp.Domain.csproj", "{11111111-1111-1111-1111-111111111111}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "Therapp.Application",
"src\Therapp.Application\Therapp.Application.csproj", "{22222222-2222-2222-2222-222222222222}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "Therapp.Infrastructure",
"src\Therapp.Infrastructure\Therapp.Infrastructure.csproj", "{33333333-3333-3333-3333-333333333333}"
EndProject
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "Therapp.Api", "src\Therapp.Api\Therapp.Api.csproj",
"{44444444-4444-4444-4444-444444444444}"
EndProject
Global
    GlobalSection(SolutionConfigurationPlatforms) = preSolution
        Debug|Any CPU = Debug|Any CPU
        Release|Any CPU = Release|Any CPU
    EndGlobalSection
    GlobalSection(ProjectConfigurationPlatforms) = postSolution
        {11111111-1111-1111-1111-111111111111}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
        {11111111-1111-1111-1111-111111111111}.Debug|Any CPU.Build.0 = Debug|Any CPU
        {11111111-1111-1111-1111-111111111111}.Release|Any CPU.ActiveCfg = Release|Any CPU
        {11111111-1111-1111-1111-111111111111}.Release|Any CPU.Build.0 = Release|Any CPU
        {22222222-2222-2222-2222-222222222222}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
        {22222222-2222-2222-2222-222222222222}.Debug|Any CPU.Build.0 = Debug|Any CPU
        {22222222-2222-2222-2222-222222222222}.Release|Any CPU.ActiveCfg = Release|Any CPU
        {22222222-2222-2222-2222-222222222222}.Release|Any CPU.Build.0 = Release|Any CPU
        {33333333-3333-3333-3333-333333333333}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
        {33333333-3333-3333-3333-333333333333}.Debug|Any CPU.Build.0 = Debug|Any CPU
        {33333333-3333-3333-3333-333333333333}.Release|Any CPU.ActiveCfg = Release|Any CPU
        {33333333-3333-3333-3333-333333333333}.Release|Any CPU.Build.0 = Release|Any CPU
        {44444444-4444-4444-4444-444444444444}.Debug|Any CPU.ActiveCfg = Debug|Any CPU
        {44444444-4444-4444-4444-444444444444}.Debug|Any CPU.Build.0 = Debug|Any CPU
        {44444444-4444-4444-4444-444444444444}.Release|Any CPU.ActiveCfg = Release|Any CPU
        {44444444-4444-4444-4444-444444444444}.Release|Any CPU.Build.0 = Release|Any CPU
```

```

EndGlobalSection
GlobalSection(NestedProjects) = preSolution
{11111111-1111-1111-1111-111111111111} = {A1B2C3D4-1111-2222-3333-444455556666}
{22222222-2222-2222-2222-222222222222} = {A1B2C3D4-1111-2222-3333-444455556666}
{33333333-3333-3333-3333-333333333333} = {A1B2C3D4-1111-2222-3333-444455556666}
{44444444-4444-4444-4444-444444444444} = {A1B2C3D4-1111-2222-3333-444455556666}
EndGlobalSection
EndGlobal

```

backend/global.json

```

{
  "sdk": {
    "version": "8.0.0",
    "rollForward": "latestFeature",
    "allowPrerelease": false
  }
}

```

backend/src/Therapp.Api/Auth/CurrentUser.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using Therapp.Application.Abstractions;

namespace Therapp.Api.Auth;

public class CurrentUser : ICurrentUser
{
    private readonly IHttpContextAccessor _accessor;

    public CurrentUser(IHttpContextAccessor accessor) => _accessor = accessor;

    private ClaimsPrincipal? User => _accessor.HttpContext?.User;

    public bool IsAuthenticated => User?.Identity?.IsAuthenticated ?? false;

    public Guid? UserId
    {
        get
        {
            var sub = User?.FindFirst(JwtRegisteredClaimNames.Sub)?.Value
                ?? User?.FindFirst(ClaimTypes.NameIdentifier)?.Value;
            return Guid.TryParse(sub, out var id) ? id : null;
        }
    }

    public string? Username => User?.FindFirst("username")?.Value;

    public bool IsInRole(string role) => User?.IsInRole(role) ?? false;
}

```

backend/src/Therapp.Api/Controllers/AuthController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Auth;
using Therapp.Application.Auth.Dtos;

namespace Therapp.Api.Controllers;

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _auth;

    public AuthController(IAuthService auth) => _auth = auth;

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegisterDto dto, CancellationToken ct)
        => (await _auth.RegisterAsync(dto, ct)).ToActionResult();

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginDto dto, CancellationToken ct)
        => (await _auth.LoginAsync(dto, ct)).ToActionResult();

    [HttpPost("refresh")]
    public async Task<IActionResult> Refresh([FromBody] RefreshTokenDto dto, CancellationToken ct)
        => (await _auth.RefreshAsync(dto.RefreshToken, ct)).ToActionResult();
}

```

```

[HttpPost("logout")]
[Authorize]
public async Task<IActionResult> Logout([FromBody] RefreshTokenDto dto, CancellationToken ct)
{
    await _auth.LogoutAsync(dto.RefreshToken, ct);
    return NoContent();
}
}

```

backend/src/Therapp.Api/Controllers/CategoriesController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Application.Categories;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/categories")]
public class CategoriesController : ControllerBase
{
    private readonly ICategoryService _svc;
    public CategoriesController(ICategoryService svc) => _svc = svc;

    [HttpGet]
    public async Task<IActionResult> List(CancellationToken ct)
    => Ok(await _svc.ListAsync(ct));

    [HttpGet("{slug}")]
    public async Task<IActionResult> Get(string slug, CancellationToken ct)
    {
        var c = await _svc.GetBySlugAsync(slug, ct);
        return c is null ? NotFound() : Ok(c);
    }
}

```

backend/src/Therapp.Api/Controllers/CommentsController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Comments;
using Therapp.Application.Comments.Dtos;
using Therapp.Application.Common;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api")]
public class CommentsController : ControllerBase
{
    private readonly ICommentService _svc;
    public CommentsController(ICommentService svc) => _svc = svc;

    [HttpGet("posts/{postId:guid}/comments")]
    public async Task<IActionResult> ByPost(Guid postId, [FromQuery] PagedQuery q, CancellationToken ct)
    => Ok(await _svc.GetByPostAsync(postId, q, ct));

    [HttpGet("comments/{id:guid}/replies")]
    public async Task<IActionResult> Replies(Guid id, [FromQuery] PagedQuery q, CancellationToken ct)
    => Ok(await _svc.GetRepliesAsync(id, q, ct));

    [HttpPost("posts/{postId:guid}/comments")]
    public async Task<IActionResult> Create(Guid postId, [FromBody] CreateCommentDto dto, CancellationToken ct)
    => (await _svc.CreateAsync(postId, dto, ct)).ToActionResult();

    [HttpPost("comments/{id:guid}/replies")]
    public async Task<IActionResult> Reply(Guid id, [FromBody] CreateCommentDto dto, CancellationToken ct)
    => (await _svc.ReplyAsync(id, dto, ct)).ToActionResult();

    [HttpDelete("comments/{id:guid}")]
    public async Task<IActionResult> Delete(Guid id, CancellationToken ct)
    => (await _svc.DeleteAsync(id, ct)).ToActionResult();

    [HttpPost("comments/{id:guid}/like")]
    public async Task<IActionResult> Like(Guid id, CancellationToken ct)
    => (await _svc.LikeAsync(id, ct)).ToActionResult();

    [HttpDelete("comments/{id:guid}/like")]
    public async Task<IActionResult> Unlike(Guid id, CancellationToken ct)

```

```

=> (await _svc.UnlikeAsync(id, ct)).ToActionResult();
}

```

backend/src/Therapp.Api/Controllers/NotificationsController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Notifications;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/notifications")]
public class NotificationsController : ControllerBase
{
    private readonly INotificationService _svc;
    public NotificationsController(INotificationService svc) => _svc = svc;

    [HttpGet]
    public async Task<IActionResult> List([FromQuery] int page = 1, [FromQuery] int pageSize = 20,
        CancellationToken ct = default)
        => Ok(await _svc.ListAsync(page, pageSize, ct));

    [HttpGet("unread-count")]
    public async Task<IActionResult> UnreadCount(CancellationToken ct)
        => Ok(new { count = await _svc.GetUnreadCountAsync(ct) });

    [HttpPost("{id:guid}/read")]
    public async Task<IActionResult> Read(Guid id, CancellationToken ct)
        => (await _svc.MarkReadAsync(id, ct)).ToActionResult();

    [HttpPost("read-all")]
    public async Task<IActionResult> ReadAll(CancellationToken ct)
        => (await _svc.MarkAllReadAsync(ct)).ToActionResult();
}

```

backend/src/Therapp.Api/Controllers/PostsController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Posts;
using Therapp.Application.Posts.Dtos;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/posts")]
public class PostsController : ControllerBase
{
    private readonly IPostService _svc;
    public PostsController(IPostService svc) => _svc = svc;

    [HttpGet]
    public async Task<IActionResult> Feed([FromQuery] FeedQuery q, CancellationToken ct)
        => Ok(await _svc.GetFeedAsync(q, ct));

    [HttpGet("category/{slug}")]
    public async Task<IActionResult> ByCategory(string slug, [FromQuery] FeedQuery q, CancellationToken ct)
        => Ok(await _svc.GetByCategoryAsync(slug, q, ct));

    [HttpGet("me")]
    public async Task<IActionResult> Mine([FromQuery] FeedQuery q, CancellationToken ct)
        => Ok(await _svc.GetMineAsync(q, ct));

    [HttpGet("me/saved")]
    public async Task<IActionResult> MySaved([FromQuery] FeedQuery q, CancellationToken ct)
        => Ok(await _svc.GetMySavedAsync(q, ct));

    [HttpGet("{id:guid}")]
    public async Task<IActionResult> Get(Guid id, CancellationToken ct)
        => (await _svc.GetByIdAsync(id, ct)).ToActionResult();

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreatePostDto dto, CancellationToken ct)
        => (await _svc.CreateAsync(dto, ct)).ToActionResult();

    [HttpPatch("{id:guid}")]
    public async Task<IActionResult> Update(Guid id, [FromBody] UpdatePostDto dto, CancellationToken ct)
        => (await _svc.UpdateAsync(id, dto, ct)).ToActionResult();
}

```

```

[HttpDelete("{id:guid}")]
public async Task<IActionResult> Delete(Guid id, CancellationToken ct)
    => (await _svc.DeleteAsync(id, ct)).ToActionResult();

[HttpPost("{id:guid}/like")]
public async Task<IActionResult> Like(Guid id, CancellationToken ct)
    => (await _svc.LikeAsync(id, ct)).ToActionResult();

[HttpDelete("{id:guid}/like")]
public async Task<IActionResult> Unlike(Guid id, CancellationToken ct)
    => (await _svc.UnlikeAsync(id, ct)).ToActionResult();

[HttpPost("{id:guid}/save")]
public async Task<IActionResult> Save(Guid id, CancellationToken ct)
    => (await _svc.SaveAsync(id, ct)).ToActionResult();

[HttpDelete("{id:guid}/save")]
public async Task<IActionResult> Unsave(Guid id, CancellationToken ct)
    => (await _svc.UnsaveAsync(id, ct)).ToActionResult();
}

```

backend/src/Therapp.Api/Controllers/ReportsController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Reports;
using Therapp.Application.Reports.Dtos;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/reports")]
public class ReportsController : ControllerBase
{
    private readonly IReportService _svc;
    public ReportsController(IReportService svc) => _svc = svc;

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateReportDto dto, CancellationToken ct)
        => (await _svc.CreateAsync(dto, ct)).ToActionResult();
}

```

backend/src/Therapp.Api/Controllers/UsersController.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Therapp.Api.Extensions;
using Therapp.Application.Users;
using Therapp.Application.Users.Dtos;

namespace Therapp.Api.Controllers;

[ApiController]
[Authorize]
[Route("api/users")]
public class UsersController : ControllerBase
{
    private readonly IUserService _svc;
    public UsersController(IUserService svc) => _svc = svc;

    [HttpGet("me")]
    public async Task<IActionResult> Me(CancellationToken ct)
        => (await _svc.GetMeAsync(ct)).ToActionResult();

    [HttpPatch("me")]
    public async Task<IActionResult> UpdateMe([FromBody] UpdateProfileDto dto, CancellationToken ct)
        => (await _svc.UpdateMeAsync(dto, ct)).ToActionResult();

    [HttpDelete("me")]
    public async Task<IActionResult> DeleteMe(CancellationToken ct)
        => (await _svc.DeleteMeAsync(ct)).ToActionResult();

    [HttpGet("{username}")]
    public async Task<IActionResult> ByUsername(string username, CancellationToken ct)
        => (await _svc.GetByUsernameAsync(username, ct)).ToActionResult();
}

```

backend/src/Therapp.Api/Extensions/ResultExtensions.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.AspNetCore.Mvc;
using Therapp.Application.Common;

namespace Therapp.Api.Extensions;

public static class ResultExtensions
{
    public static IActionResult ToActionResult<T>(this Result<T> result)
    {
        if (result.Success) return new OkObjectResult(result.Value);
        var err = result.Error!;
        return new ObjectResult(new { code = err.Code, message = err.Message })
        {
            StatusCode = err.StatusCode
        };
    }

    public static IActionResult ToActionResult(this Result result)
    {
        if (result.Success) return new NoContentResult();
        var err = result.Error!;
        return new ObjectResult(new { code = err.Code, message = err.Message })
        {
            StatusCode = err.StatusCode
        };
    }
}
```

backend/src/Therapp.Api/Middleware/ExceptionHandlerMiddleware.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using System.Text.Json;
using FluentValidation;

namespace Therapp.Api.Middleware;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _log;

    public ExceptionHandlingMiddleware(RequestDelegate next, ILogger<ExceptionHandlingMiddleware> log)
    {
        _next = next;
        _log = log;
    }

    public async Task InvokeAsync(HttpContext ctx)
    {
        try
        {
            await _next(ctx);
        }
        catch (ValidationException vex)
        {
            ctx.Response.StatusCode = StatusCodes.Status400BadRequest;
            ctx.Response.ContentType = "application/json";
            var errors = vex.Errors.Select(e => new { field = e.PropertyName, message = e.ErrorMessage });
            await ctx.Response.WriteAsync(JsonSerializer.Serialize(new
            {
                code = "VALIDATION_ERROR",
                message = "Geçersiz veri gönderdiniz.",
                errors
            }));
        }
        catch (Exception ex)
        {
            _log.LogError(ex, "Unhandled exception");
            ctx.Response.StatusCode = StatusCodes.Status500InternalServerError;
            ctx.Response.ContentType = "application/json";
            await ctx.Response.WriteAsync(JsonSerializer.Serialize(new
            {
                code = "INTERNAL_ERROR",
                message = "Bir hata oluştu, lütfen tekrar deneyin."
            }));
        }
    }
}
```

backend/src/Therapp.Api/Program.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using System.Text;
using FluentValidation;
using FluentValidation.AspNetCore;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using Serilog;
using Therapp.Api.Auth;
using Therapp.Api.Middleware;
using Therapp.Application;
using Therapp.Application.Abstractions;
using Therapp.Application.Auth;
using Therapp.Application.Validators;
using Therapp.Infrastructure;
using Therapp.Infrastructure.Persistence;
using Therapp.Infrastructure.Persistence.Seed;

var builder = WebApplication.CreateBuilder(args);

// ----- Logging -----
builder.Host.UseSerilog((ctx, lc) => lc
    .ReadFrom.Configuration(ctx.Configuration)
    .Enrich.FromLogContext()
    .WriteTo.Console());

// ----- Options -----
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));
var jwt = builder.Configuration.GetSection("Jwt").Get<JwtOptions>()
    ?? throw new InvalidOperationException("Jwt configuration missing.");

// ----- Layers -----
builder.Services
    .AddInfrastructure(builder.Configuration)
    .AddApplication();

builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUser>();

// ----- Web -----
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "Therapp API", Version = "v1" });
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header
    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer" }
            },
            Array.Empty<string>()
        }
    });
});

// ----- Validation -----
builder.Services.AddFluentValidationAutoValidation();
builder.Services.AddValidatorsFromAssemblyContaining<RegisterDtoValidator>();

// ----- Auth -----
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.RequireHttpsMetadata = !builder.Environment.IsDevelopment();
        o.SaveToken = true;
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt.Issuer,
            ValidAudience = jwt.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Secret)),
        }
    });
```

```

        ClockSkew = TimeSpan.FromSeconds(30)
    });
});
builder.Services.AddAuthorization();

// ----- CORS -----
builder.Services.AddCors(o => o.AddDefaultPolicy(p =>
    p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()));

var app = builder.Build();

// ----- Migrate + Seed -----
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await db.Database.MigrateAsync();
    await CategorySeeder.SeedAsync(db);
}

// ----- Pipeline -----
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseSerilogRequestLogging();
app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

backend/src/Therapp.Api/Properties/launchSettings.json

```

{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "profiles": {
    "Therapp.Api": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "launchUrl": "swagger",
      "applicationUrl": "http://0.0.0.0:5080",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}

```

backend/src/Therapp.Api/Therapp.Api.csproj

```

<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>Therapp.Api</RootNamespace>
    <AssemblyName>Therapp.Api</AssemblyName>
    <UserSecretsId>therapp-api-secrets</UserSecretsId>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\Therapp.Application\Therapp.Application.csproj" />
    <ProjectReference Include="..\Therapp.Infrastructure\Therapp.Infrastructure.csproj" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.10">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="FluentValidation.AspNetCore" Version="11.3.0" />
    <PackageReference Include="FluentValidation.DependencyInjectionExtensions" Version="11.9.2" />
    <PackageReference Include="Serilog.AspNetCore" Version="8.0.2" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.7.3" />
    <PackageReference Include="AspNetCoreRateLimit" Version="5.0.0" />
  </ItemGroup>
</Project>

```

backend/src/Therapp.Api/appsettings.Development.json

```

{

```

```

"Serilog": {
  "MinimumLevel": {
    "Default": "Debug"
  }
}
}

```

backend/src/Therapp.Api/appsettings.json

```

{
  "ConnectionStrings": {
    "Postgres": "Host=localhost;Port=5432;Database=therapp;Username=postgres;Password=postgres"
  },
  "Jwt": {
    "Issuer": "therapp-api",
    "Audience": "therapp-mobile",
    "Secret": "CHANGE_ME_LONG_RANDOM_SECRET_AT_LEAST_32_CHARS_LONG_PLEASE",
    "AccessTokenMinutes": 15,
    "RefreshTokenDays": 30
  },
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft.AspNetCore": "Warning",
        "Microsoft.EntityFrameworkCore": "Warning"
      }
    }
  },
  "AllowedHosts": "*"
}

```

backend/src/Therapp.Application/Abstractions/IAppDbContext.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Domain.Entities;

namespace Therapp.Application.Abstractions;

public interface IAppDbContext
{
    DbSet<User> Users { get; }
    DbSet<Category> Categories { get; }
    DbSet<Post> Posts { get; }
    DbSet<Comment> Comments { get; }
    DbSet<PostLike> PostLikes { get; }
    DbSet<CommentLike> CommentLikes { get; }
    DbSet<SavedPost> SavedPosts { get; }
    DbSet<Report> Reports { get; }
    DbSet<Notification> Notifications { get; }
    DbSet<RefreshToken> RefreshTokens { get; }

    Task<int> SaveChangesAsync(CancellationToken ct = default);
}

```

backend/src/Therapp.Application/Abstractions/ICurrentUser.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Abstractions;

public interface ICurrentUser
{
    Guid? UserId { get; }
    string? Username { get; }
    bool IsAuthenticated { get; }
    bool IsInRole(string role);
}

```

backend/src/Therapp.Application/Abstractions/IPasswordHasher.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Abstractions;

public interface IPasswordHasher
{
    string Hash(string password);
    bool Verify(string password, string hash);
}

```

backend/src/Therapp.Application/Auth/AuthService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using Therapp.Application.Abstractions;
using Therapp.Application.Auth.Dtos;
using Therapp.Application.Common;
using Therapp.Domain.Entities;

namespace Therapp.Application.Auth;

public class AuthService : IAuthService
{
    private readonly IAppDbContext _db;
    private readonly IPasswordHasher _hasher;
    private readonly IJwtTokenService _jwt;
    private readonly JwtOptions _opt;

    public AuthService(
        IAppDbContext db,
        IPasswordHasher hasher,
        IJwtTokenService jwt,
        IOptions<JwtOptions> opt)
    {
        _db = db;
        _hasher = hasher;
        _jwt = jwt;
        _opt = opt.Value;
    }

    public async Task<Result<AuthResponseDto>> RegisterAsync(RegisterDto dto, CancellationToken ct)
    {
        var username = dto.Username.Trim().ToLowerInvariant();
        var email = dto.Email.Trim().ToLowerInvariant();

        if (await _db.Users.AnyAsync(u => u.Username == username, ct))
            return Result<AuthResponseDto>.Fail(Errors.UsernameTaken);
        if (await _db.Users.AnyAsync(u => u.Email == email, ct))
            return Result<AuthResponseDto>.Fail(Errors.EmailTaken);

        var user = new User
        {
            Username = username,
            Email = email,
            PasswordHash = _hasher.Hash(dto.Password),
            DisplayName = dto.DisplayName.Trim(),
            AcceptedDisclaimerAt = DateTime.UtcNow
        };
        _db.Users.Add(user);

        var response = IssueTokens(user);
        await _db.SaveChangesAsync(ct);
        return Result<AuthResponseDto>.Ok(response);
    }

    public async Task<Result<AuthResponseDto>> LoginAsync(LoginDto dto, CancellationToken ct)
    {
        var email = dto.Email.Trim().ToLowerInvariant();
        var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == email, ct);
        if (user is null || !_hasher.Verify(dto.Password, user.PasswordHash))
            return Result<AuthResponseDto>.Fail(Errors.InvalidCredentials);
        if (user.IsBanned)
            return Result<AuthResponseDto>.Fail(Errors.UserBanned);

        var response = IssueTokens(user);
        await _db.SaveChangesAsync(ct);
        return Result<AuthResponseDto>.Ok(response);
    }

    public async Task<Result<AuthResponseDto>> RefreshAsync(string refreshToken, CancellationToken ct)
    {
        if (string.IsNullOrEmpty(refreshToken))
            return Result<AuthResponseDto>.Fail(Errors.InvalidRefreshToken);

        var hash = _jwt.HashRefreshToken(refreshToken);
        var existing = await _db.RefreshTokens
            .Include(rt => rt.User)
            .FirstOrDefaultAsync(rt => rt.TokenHash == hash, ct);

        if (existing is null || existing.RevokedAt != null || existing.ExpiresAt <= DateTime.UtcNow)
            return Result<AuthResponseDto>.Fail(Errors.InvalidRefreshToken);
        if (existing.User.IsBanned)
            return Result<AuthResponseDto>.Fail(Errors.UserBanned);

        existing.RevokedAt = DateTime.UtcNow;
        var response = IssueTokens(existing.User);
    }
}
```

```

        await _db.SaveChangesAsync(ct);
        return Result<AuthResponseDto>.Ok(response);
    }

    public async Task LogoutAsync(string refreshToken, CancellationToken ct)
    {
        if (string.IsNullOrWhiteSpace(refreshToken)) return;
        var hash = _jwt.HashRefreshToken(refreshToken);
        var token = await _db.RefreshTokens.FirstOrDefaultAsync(rt => rt.TokenHash == hash, ct);
        if (token is not null && token.RevokedAt is null)
        {
            token.RevokedAt = DateTime.UtcNow;
            await _db.SaveChangesAsync(ct);
        }
    }

    private AuthResponseDto IssueTokens(User user)
    {
        var access = _jwt.CreateAccessToken(user);
        var (raw, hash) = _jwt.CreateRefreshToken();
        var expiresAt = DateTime.UtcNow.AddDays(_opt.RefreshTokenDays);

        _db.RefreshTokens.Add(new RefreshToken
        {
            UserId = user.Id,
            TokenHash = hash,
            ExpiresAt = expiresAt
        });

        return new AuthResponseDto
        {
            AccessToken = access,
            RefreshToken = raw,
            AccessTokenExpiresAt = DateTime.UtcNow.AddMinutes(_opt.AccessTokenMinutes),
            User = new AuthUserDto
            {
                Id = user.Id,
                Username = user.Username,
                Email = user.Email,
                DisplayName = user.DisplayName,
                AvatarUrl = user.AvatarUrl,
                Role = (int)user.Role
            }
        };
    }
}

```

backend/src/Therapp.Application/Auth/Dtos/AuthResponseDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Auth.Dtos;

public class AuthResponseDto
{
    public string AccessToken { get; set; } = null!;
    public string RefreshToken { get; set; } = null!;
    public DateTime AccessTokenExpiresAt { get; set; }
    public AuthUserDto User { get; set; } = null!;
}

public class AuthUserDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string Email { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
    public int Role { get; set; }
}

```

backend/src/Therapp.Application/Auth/Dtos/LoginDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Auth.Dtos;

public class LoginDto
{
    public string Email { get; set; } = null!;
    public string Password { get; set; } = null!;
}

```

backend/src/Therapp.Application/Auth/Dtos/RefreshTokenDto.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Auth.Dtos;

public class RefreshTokenDto
{
    public string RefreshToken { get; set; } = null!;
}
```

backend/src/Therapp.Application/Auth/Dtos/RegisterDto.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Auth.Dtos;

public class RegisterDto
{
    public string Username { get; set; } = null!;
    public string Email { get; set; } = null!;
    public string Password { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
}
```

backend/src/Therapp.Application/Auth/IAuthService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Auth.Dtos;
using Therapp.Application.Common;

namespace Therapp.Application.Auth;

public interface IAuthService
{
    Task<Result<AuthResponseDto>> RegisterAsync(RegisterDto dto, CancellationToken ct);
    Task<Result<AuthResponseDto>> LoginAsync(LoginDto dto, CancellationToken ct);
    Task<Result<AuthResponseDto>> RefreshAsync(string refreshToken, CancellationToken ct);
    Task LogoutAsync(string refreshToken, CancellationToken ct);
}
```

backend/src/Therapp.Application/Auth/IJwtTokenService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Entities;

namespace Therapp.Application.Auth;

public interface IJwtTokenService
{
    string CreateAccessToken(User user);
    (string Raw, string Hash) CreateRefreshToken();
    string HashRefreshToken(string raw);
}
```

backend/src/Therapp.Application/Auth/JwtOptions.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Auth;

public class JwtOptions
{
    public string Issuer { get; set; } = null!;
    public string Audience { get; set; } = null!;
    public string Secret { get; set; } = null!;
    public int AccessTokenMinutes { get; set; } = 15;
    public int RefreshTokenDays { get; set; } = 30;
}
```

backend/src/Therapp.Application/Auth/JwtTokenService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using Therapp.Domain.Entities;

namespace Therapp.Application.Auth;
```

```

public class JwtTokenService : IJwtTokenService
{
    private readonly JwtOptions _opt;

    public JwtTokenService(IOptions<JwtOptions> opt)
    {
        _opt = opt.Value;
        if (string.IsNullOrEmpty(_opt.Secret) || _opt.Secret.Length < 32)
            throw new InvalidOperationException("Jwt:Secret must be at least 32 characters.");
    }

    public string CreateAccessToken(User user)
    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_opt.Secret));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim("username", user.Username),
            new Claim(ClaimTypes.Role, user.Role.ToString()),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        var token = new JwtSecurityToken(
            issuer: _opt.Issuer,
            audience: _opt.Audience,
            claims: claims,
            notBefore: DateTime.UtcNow,
            expires: DateTime.UtcNow.AddMinutes(_opt.AccessTokenMinutes),
            signingCredentials: creds);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }

    public (string Raw, string Hash) CreateRefreshToken()
    {
        var bytes = RandomNumberGenerator.GetBytes(64);
        var raw = Base64UrlEncoder.Encode(bytes);
        return (raw, HashRefreshToken(raw));
    }

    public string HashRefreshToken(string raw)
    {
        using var sha = SHA256.Create();
        var hash = sha.ComputeHash(Encoding.UTF8.GetBytes(raw));
        return Convert.ToBase64String(hash);
    }
}

```

backend/src/Therapp.Application/Categories/CategoryService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Categories.Dtos;

namespace Therapp.Application.Categories;

public class CategoryService : ICategoryService
{
    private readonly IAppDbContext _db;
    public CategoryService(IAppDbContext db) => _db = db;

    public async Task<IReadOnlyList<CategoryDto>> ListAsync(CancellationTokens ct) =>
        await _db.Categories
            .Where(c => c.IsActive)
            .OrderBy(c => c.OrderIndex)
            .Select(c => new CategoryDto
            {
                Id = c.Id,
                Slug = c.Slug,
                Name = c.Name,
                Description = c.Description,
                Icon = c.Icon,
                Color = c.Color,
                OrderIndex = c.OrderIndex,
                PostCount = c.Posts.Count(p => !p.IsHidden)
            })
            .ToListAsync(ct);

    public async Task<CategoryDto> GetBySlugAsync(string slug, CancellationTokens ct) =>
        await _db.Categories
            .Where(c => c.Slug == slug && c.IsActive)
            .Select(c => new CategoryDto
            {

```

```

        Id = c.Id,
        Slug = c.Slug,
        Name = c.Name,
        Description = c.Description,
        Icon = c.Icon,
        Color = c.Color,
        OrderIndex = c.OrderIndex,
        PostCount = c.Posts.Count(p => !p.IsHidden)
    })
    .FirstOrDefaultAsync(ct);
}

```

backend/src/Therapp.Application/Categories/Dtos/CategoryDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Categories.Dtos;

public class CategoryDto
{
    public Guid Id { get; set; }
    public string Slug { get; set; } = null!;
    public string Name { get; set; } = null!;
    public string Description { get; set; } = null!;
    public string Icon { get; set; } = null!;
    public string Color { get; set; } = null!;
    public int OrderIndex { get; set; }
    public int PostCount { get; set; }
}

```

backend/src/Therapp.Application/Categories/ICategoryService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Categories.Dtos;

namespace Therapp.Application.Categories;

public interface ICategoryService
{
    Task<IReadOnlyList<CategoryDto>> ListAsync(Cancellation token ct);
    Task<CategoryDto?> GetBySlugAsync(string slug, Cancellation token ct);
}

```

backend/src/Therapp.Application/Comments/CommentService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Comments.Dtos;
using Therapp.Application.Common;
using Therapp.Application.Notifications;
using Therapp.Domain.Entities;
using Therapp.Domain.Enums;

namespace Therapp.Application.Comments;

public class CommentService : ICommentService
{
    private readonly IAppDbContext _db;
    private readonly ICurrentUser _current;
    private readonly INotificationService _notifier;

    public CommentService(IAppDbContext db, ICurrentUser current, INotificationService notifier)
    {
        _db = db;
        _current = current;
        _notifier = notifier;
    }

    // ----- Reads -----

    public async Task<PagedResult<CommentDto>> GetByPostAsync(Guid postId, PagedQuery q, Cancellation token ct)
    {
        var postExists = await _db.Posts.AnyAsync(p => p.Id == postId && !p.IsHidden, ct);
        if (!postExists) return Empty(q);

        var baseQuery = _db.Comments
            .Where(c => c.PostId == postId && c.ParentCommentId == null && !c.IsHidden)
            .OrderBy(c => c.CreatedAt);

        return await ProjectPagedAsync(baseQuery, q, ct);
    }
}

```

```

    public async Task<PagedResult<CommentDto>> GetRepliesAsync(Guid parentId, PagedQuery q,
CancellationToken ct)
    {
        var baseQuery = _db.Comments
            .Where(c => c.ParentCommentId == parentId && !c.IsHidden)
            .OrderBy(c => c.CreatedAt);

        return await ProjectPagedAsync(baseQuery, q, ct);
    }

    // ----- Writes -----

    public async Task<Result<CommentDto>> CreateAsync(Guid postId, CreateCommentDto dto, Cancellation token
ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result<CommentDto>.Fail(Errors.Unauthorized);

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == postId && !p.IsHidden, ct);
        if (post is null) return Result<CommentDto>.Fail(Errors.NotFound("Gönderi"));

        var comment = new Comment
        {
            PostId = postId,
            UserId = userId.Value,
            Content = dto.Content.Trim()
        };
        _db.Comments.Add(comment);
        post.CommentCount += 1;
        await _db.SaveChangesAsync(ct);

        if (post.UserId != userId.Value)
        {
            await _notifier.CreateAsync(post.UserId, userId.Value, NotificationType.Comment, post.Id,
comment.Id, null, ct);
        }

        return await LoadDtoAsync(comment.Id, ct);
    }

    public async Task<Result<CommentDto>> ReplyAsync(Guid parentId, CreateCommentDto dto, Cancellation token
ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result<CommentDto>.Fail(Errors.Unauthorized);

        var parent = await _db.Comments.FirstOrDefaultAsync(c => c.Id == parentId && !c.IsHidden, ct);
        if (parent is null) return Result<CommentDto>.Fail(Errors.NotFound("Yorum"));

        // Only one level of nesting in MVP
        if (parent.ParentCommentId != null)
            return Result<CommentDto>.Fail(Errors.Validation("Yanıt yanıt verilemez."));

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == parent.PostId && !p.IsHidden, ct);
        if (post is null) return Result<CommentDto>.Fail(Errors.NotFound("Gönderi"));

        var reply = new Comment
        {
            PostId = parent.PostId,
            UserId = userId.Value,
            ParentCommentId = parentId,
            Content = dto.Content.Trim()
        };
        _db.Comments.Add(reply);
        post.CommentCount += 1;
        await _db.SaveChangesAsync(ct);

        if (parent.UserId != userId.Value)
        {
            await _notifier.CreateAsync(parent.UserId, userId.Value, NotificationType.Reply, post.Id,
reply.Id, null, ct);
        }

        return await LoadDtoAsync(reply.Id, ct);
    }

    public async Task<Result> DeleteAsync(Guid id, Cancellation token ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var comment = await _db.Comments.FirstOrDefaultAsync(c => c.Id == id, ct);
        if (comment is null) return Result.Fail(Errors.NotFound("Yorum"));

        var isOwner = comment.UserId == userId;
        var isMod = _current.IsInRole(nameof(UserRole.Moderator)) ||
_current.IsInRole(nameof(UserRole.Admin));
        if (!isOwner && !isMod) return Result.Fail(Errors.Forbidden);
    }

```

```

        comment.DeletedAt = DateTime.UtcNow;

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == comment.PostId, ct);
        if (post is not null && post.CommentCount > 0) post.CommentCount -= 1;

        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    public async Task<Result> LikeAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var comment = await _db.Comments.FirstOrDefaultAsync(c => c.Id == id && !c.IsHidden, ct);
        if (comment is null) return Result.Fail(Errors.NotFound("Yorum"));

        var already = await _db.CommentLikes.AnyAsync(l => l.UserId == userId && l.CommentId == id, ct);
        if (already) return Result.Ok();

        _db.CommentLikes.Add(new CommentLike { UserId = userId.Value, CommentId = id });
        comment.LikeCount += 1;
        await _db.SaveChangesAsync(ct);

        if (comment.UserId != userId.Value)
        {
            await _notifier.CreateAsync(comment.UserId, userId.Value, NotificationType.CommentLike,
comment.PostId, comment.Id, null, ct);
        }

        return Result.Ok();
    }

    public async Task<Result> UnlikeAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var comment = await _db.Comments.FirstOrDefaultAsync(c => c.Id == id, ct);
        if (comment is null) return Result.Fail(Errors.NotFound("Yorum"));

        var like = await _db.CommentLikes.FirstOrDefaultAsync(l => l.UserId == userId && l.CommentId == id,
ct);
        if (like is null) return Result.Ok();

        _db.CommentLikes.Remove(like);
        if (comment.LikeCount > 0) comment.LikeCount -= 1;
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    // ----- Helpers -----

    private async Task<Result<CommentDto>> LoadDtoAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        var dto = await _db.Comments
            .Where(c => c.Id == id)
            .Select(c => new CommentDto
            {
                Id = c.Id,
                PostId = c.PostId,
                ParentCommentId = c.ParentCommentId,
                Content = c.Content,
                Author = new CommentAuthorDto
                {
                    Id = c.User.Id,
                    Username = c.User.Username,
                    DisplayName = c.User.DisplayName,
                    AvatarUrl = c.User.AvatarUrl
                },
                LikeCount = c.LikeCount,
                ReplyCount = c.Replies.Count(r => !r.IsHidden),
                IsLikedByMe = userId != null && c.Likes.Any(l => l.UserId == userId),
                IsMine = userId != null && c.UserId == userId,
                CreatedAt = c.CreatedAt
            })
            .FirstOrDefaultAsync(ct);

        return dto is null
            ? Result<CommentDto>.Fail(Errors.NotFound("Yorum"))
            : Result<CommentDto>.Ok(dto);
    }

    private async Task<PagedResult<CommentDto>> ProjectPagedAsync(
        IQueryable<Comment> baseQuery, PagedQuery q, CancellationToken ct)
    {

```

```

var userId = _current.UserId;
var total = await baseQuery.CountAsync(ct);

var items = await baseQuery
    .Skip((q.Page - 1) * q.PageSize)
    .Take(q.PageSize)
    .Select(c => new CommentDto
    {
        Id = c.Id,
        PostId = c.PostId,
        ParentCommentId = c.ParentCommentId,
        Content = c.Content,
        Author = new CommentAuthorDto
        {
            Id = c.User.Id,
            Username = c.User.Username,
            DisplayName = c.User.DisplayName,
            AvatarUrl = c.User.AvatarUrl
        },
        LikeCount = c.LikeCount,
        ReplyCount = c.Replies.Count(r => !r.IsHidden),
        IsLikedByMe = userId != null && c.Likes.Any(l => l.UserId == userId),
        IsMine = userId != null && c.UserId == userId,
        CreatedAt = c.CreatedAt
    })
    .ToListAsync(ct);

return new PagedResult<CommentDto>
{
    Items = items,
    Page = q.Page,
    PageSize = q.PageSize,
    TotalCount = total
};
}

private static PagedResult<CommentDto> Empty(PagedQuery q) => new()
{
    Items = Array.Empty<CommentDto>(),
    Page = q.Page,
    PageSize = q.PageSize,
    TotalCount = 0
};
}

```

backend/src/Therapp.Application/Comments/Dtos/CommentDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Comments.Dtos;

public class CommentAuthorDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
}

public class CommentDto
{
    public Guid Id { get; set; }
    public Guid PostId { get; set; }
    public Guid? ParentCommentId { get; set; }
    public string Content { get; set; } = null!;
    public CommentAuthorDto Author { get; set; } = null!;
    public int LikeCount { get; set; }
    public int ReplyCount { get; set; }
    public bool IsLikedByMe { get; set; }
    public bool IsMine { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

backend/src/Therapp.Application/Comments/Dtos/CreateCommentDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Comments.Dtos;

public class CreateCommentDto
{
    public string Content { get; set; } = null!;
}

```

backend/src/Therapp.Application/Comments/ICommentService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Comments.Dtos;
using Therapp.Application.Common;

namespace Therapp.Application.Comments;

public interface ICommentService
{
    Task<PagedResult<CommentDto>> GetByPostAsync(Guid postId, PagedQuery q, CancellationToken ct);
    Task<PagedResult<CommentDto>> GetRepliesAsync(Guid parentId, PagedQuery q, CancellationToken ct);
    Task<Result<CommentDto>> CreateAsync(Guid postId, CreateCommentDto dto, CancellationToken ct);
    Task<Result<CommentDto>> ReplyAsync(Guid parentId, CreateCommentDto dto, CancellationToken ct);
    Task<Result> DeleteAsync(Guid id, CancellationToken ct);
    Task<Result> LikeAsync(Guid id, CancellationToken ct);
    Task<Result> UnlikeAsync(Guid id, CancellationToken ct);
}
```

backend/src/Therapp.Application/Common/Error.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Common;

public record Error(string Code, string Message, int StatusCode = 400);
```

backend/src/Therapp.Application/Common/Errors.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Common;

public static class Errors
{
    public static Error UsernameTaken => new("USERNAME_TAKEN", "Bu kullanıcı adı zaten alınmış.", 409);
    public static Error EmailTaken => new("EMAIL_TAKEN", "Bu e-posta zaten kayıtlı.", 409);
    public static Error InvalidCredentials => new("INVALID_CREDENTIALS", "E-posta veya şifre hatalı.", 401);
    public static Error UserBanned => new("USER_BANNED", "Hesabınız askıya alınmış.", 403);
    public static Error InvalidRefreshToken => new("INVALID_REFRESH_TOKEN", "Oturum süresi doldu, lütfen tekrar giriş yapın.", 401);
    public static Error Unauthorized => new("UNAUTHORIZED", "Bu işlem için giriş yapmalısınız.", 401);
    public static Error Forbidden => new("FORBIDDEN", "Bu işlem için yetkiniz yok.", 403);
    public static Error NotFound(string what) => new("NOT_FOUND", $"{what} bulunamadı.", 404);
    public static Error Validation(string msg) => new("VALIDATION_ERROR", msg, 400);
    public static Error Conflict(string msg) => new("CONFLICT", msg, 409);
}
```

backend/src/Therapp.Application/Common/PagedQuery.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Common;

public class PagedQuery
{
    private int _page = 1;
    private int _pageSize = 20;

    public int Page
    {
        get => _page;
        set => _page = value < 1 ? 1 : value;
    }

    public int PageSize
    {
        get => _pageSize;
        set => _pageSize = value switch
        {
            < 1 => 20,
            > 50 => 50,
            _ => value
        };
    }
}
```

backend/src/Therapp.Application/Common/PagedResult.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Common;

public class PagedResult<T>
{
}
```

```

    public IReadOnlyList<T> Items { get; init; } = Array.Empty<T>();
    public int Page { get; init; }
    public int PageSize { get; init; }
    public int TotalCount { get; init; }
    public bool HasNext => Page * PageSize < TotalCount;
}

```

backend/src/Therapp.Application/Common/Result.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Common;

public class Result<T>
{
    public bool Success { get; init; }
    public T? Value { get; init; }
    public Error? Error { get; init; }

    public static Result<T> Ok(T value) => new() { Success = true, Value = value };
    public static Result<T> Fail(Error error) => new() { Success = false, Error = error };
}

public class Result
{
    public bool Success { get; init; }
    public Error? Error { get; init; }

    public static Result Ok() => new() { Success = true };
    public static Result Fail(Error error) => new() { Success = false, Error = error };
}

```

backend/src/Therapp.Application/DependencyInjection.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.Extensions.DependencyInjection;
using Therapp.Application.Auth;
using Therapp.Application.Categories;
using Therapp.Application.Comments;
using Therapp.Application.Notifications;
using Therapp.Application.Posts;
using Therapp.Application.Reports;
using Therapp.Application.Users;

namespace Therapp.Application;

public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddSingleton<JwtTokenService, JwtTokenService>();

        services.AddScoped<IAuthService, AuthService>();
        services.AddScoped<ICategoryService, CategoryService>();
        services.AddScoped<IPostService, PostService>();
        services.AddScoped<ICommentService, CommentService>();
        services.AddScoped<INotificationService, NotificationService>();
        services.AddScoped<IReportService, ReportService>();
        services.AddScoped<IUserService, UserService>();

        return services;
    }
}

```

backend/src/Therapp.Application/Notifications/Dtos/NotificationDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Notifications.Dtos;

public class NotificationActorDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
}

public class NotificationDto
{
    public Guid Id { get; set; }
    public int Type { get; set; }
    public NotificationActorDto? Actor { get; set; }
    public Guid? PostId { get; set; }
    public Guid? CommentId { get; set; }
}

```

```

    public string? Message { get; set; }
    public bool IsRead { get; set; }
    public DateTime CreatedAt { get; set; }
}

```

backend/src/Therapp.Application/Notifications/INotificationService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Common;
using Therapp.Application.Notifications.Dtos;
using Therapp.Domain.Enums;

namespace Therapp.Application.Notifications;

public interface INotificationService
{
    Task<PagedResult<NotificationDto>> ListAsync(int page, int pageSize, CancellationToken ct);
    Task<int> GetUnreadCountAsync(CancellationToken ct);
    Task<Result> MarkReadAsync(Guid id, CancellationToken ct);
    Task<Result> MarkAllReadAsync(CancellationToken ct);

    // Called by other services
    Task CreateAsync(
        Guid recipientUserId,
        Guid? actorUserId,
        NotificationType type,
        Guid? postId,
        Guid? commentId,
        string? message,
        CancellationToken ct);
}

```

backend/src/Therapp.Application/Notifications/NotificationService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Common;
using Therapp.Application.Notifications.Dtos;
using Therapp.Domain.Entities;
using Therapp.Domain.Enums;

namespace Therapp.Application.Notifications;

public class NotificationService : INotificationService
{
    private readonly IAppDbContext _db;
    private readonly ICurrentUser _current;

    public NotificationService(IAppDbContext db, ICurrentUser current)
    {
        _db = db;
        _current = current;
    }

    public async Task<PagedResult<NotificationDto>> ListAsync(int page, int pageSize, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Empty(page, pageSize);

        page = page < 1 ? 1 : page;
        pageSize = pageSize is < 1 or > 50 ? 20 : pageSize;

        var baseQuery = _db.Notifications
            .Where(n => n.UserId == userId)
            .OrderByDescending(n => n.CreatedAt);

        var total = await baseQuery.CountAsync(ct);

        var items = await baseQuery
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Select(n => new NotificationDto
            {
                Id = n.Id,
                Type = (int)n.Type,
                Actor = n.Actor == null ? null : new NotificationActorDto
                {
                    Id = n.Actor.Id,
                    Username = n.Actor.Username,
                    DisplayName = n.Actor.DisplayName,
                    AvatarUrl = n.Actor.AvatarUrl
                },
                PostId = n.PostId,
                CommentId = n.CommentId,
            })
            .ToListAsync(ct);

        return new PagedResult<NotificationDto>
        {
            Page = page,
            PageSize = pageSize,
            Total = total,
            Items = items
        };
    }
}

```

```

        Message = n.Message,
        IsRead = n.IsRead,
        CreatedAt = n.CreatedAt
    })
    .ToListAsync(ct);

    return new PagedResult<NotificationDto>
    {
        Items = items,
        Page = page,
        PageSize = pageSize,
        TotalCount = total
    };
}

public async Task<int> GetUnreadCountAsync(Cancellation token ct)
{
    var userId = _current.UserId;
    if (userId is null) return 0;
    return await _db.Notifications.CountAsync(n => n.UserId == userId && !n.IsRead, ct);
}

public async Task<Result> MarkReadAsync(Guid id, Cancellation token ct)
{
    var userId = _current.UserId;
    if (userId is null) return Result.Fail(Errors.Unauthorized);

    var notif = await _db.Notifications.FirstOrDefaultAsync(n => n.Id == id && n.UserId == userId, ct);
    if (notif is null) return Result.Fail(Errors.NotFound("Bildirim"));
    if (!notif.IsRead)
    {
        notif.IsRead = true;
        await _db.SaveChangesAsync(ct);
    }
    return Result.Ok();
}

public async Task<Result> MarkAllReadAsync(Cancellation token ct)
{
    var userId = _current.UserId;
    if (userId is null) return Result.Fail(Errors.Unauthorized);

    var unread = await _db.Notifications
        .Where(n => n.UserId == userId && !n.IsRead)
        .ToListAsync(ct);

    foreach (var n in unread) n.IsRead = true;
    if (unread.Count > 0) await _db.SaveChangesAsync(ct);
    return Result.Ok();
}

public async Task CreateAsync(
    Guid recipientUserId,
    Guid? actorUserId,
    NotificationType type,
    Guid? postId,
    Guid? commentId,
    string? message,
    Cancellation token ct)
{
    _db.Notifications.Add(new Notification
    {
        UserId = recipientUserId,
        ActorUserId = actorUserId,
        Type = type,
        PostId = postId,
        CommentId = commentId,
        Message = message,
        IsRead = false
    });
    await _db.SaveChangesAsync(ct);
}

private static PagedResult<NotificationDto> Empty(int page, int pageSize) => new()
{
    Items = Array.Empty<NotificationDto>(),
    Page = page < 1 ? 1 : page,
    PageSize = pageSize < 1 ? 20 : pageSize,
    TotalCount = 0
};
}

```

backend/src/Therapp.Application/Posts/Dtos/CreatePostDto.cs

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Posts.Dtos;

```

public class CreatePostDto
{
    public Guid CategoryId { get; set; }
    public string Title { get; set; } = null!;
    public string Content { get; set; } = null!;
    public bool IsAnonymous { get; set; }
}

```

backend/src/Therapp.Application/Posts/Dtos/FeedQuery.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Common;

namespace Therapp.Application.Posts.Dtos;

public enum FeedSort
{
    New = 0,
    Popular = 1
}

public class FeedQuery : PagedQuery
{
    public FeedSort Sort { get; set; } = FeedSort.New;
}

```

backend/src/Therapp.Application/Posts/Dtos/PostDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Posts.Dtos;

public class PostAuthorDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
}

public class PostCategoryDto
{
    public Guid Id { get; set; }
    public string Slug { get; set; } = null!;
    public string Name { get; set; } = null!;
    public string Color { get; set; } = null!;
    public string Icon { get; set; } = null!;
}

public class PostListItemDto
{
    public Guid Id { get; set; }
    public string Title { get; set; } = null!;
    public string Excerpt { get; set; } = null!;
    public bool IsAnonymous { get; set; }
    public PostCategoryDto Category { get; set; } = null!;
    public PostAuthorDto? Author { get; set; }
    public int LikeCount { get; set; }
    public int CommentCount { get; set; }
    public int SaveCount { get; set; }
    public DateTime CreatedAt { get; set; }
    public bool IsLikedByMe { get; set; }
    public bool IsSavedByMe { get; set; }
}

public class PostDto
{
    public Guid Id { get; set; }
    public string Title { get; set; } = null!;
    public string Content { get; set; } = null!;
    public bool IsAnonymous { get; set; }
    public PostCategoryDto Category { get; set; } = null!;
    public PostAuthorDto? Author { get; set; }
    public int LikeCount { get; set; }
    public int CommentCount { get; set; }
    public int SaveCount { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
    public bool IsLikedByMe { get; set; }
    public bool IsSavedByMe { get; set; }
    public bool IsMine { get; set; }
}

```

backend/src/Therapp.Application/Posts/Dtos/UpdatePostDto.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Posts.Dtos;

public class UpdatePostDto
{
    public string Title { get; set; } = null!;
    public string Content { get; set; } = null!;
}
```

backend/src/Therapp.Application/Posts/IPostService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Common;
using Therapp.Application.Posts.Dtos;

namespace Therapp.Application.Posts;

public interface IPostService
{
    Task<PagedResult<PostListItemDto>> GetFeedAsync(FeedQuery q, CancellationToken ct);
    Task<PagedResult<PostListItemDto>> GetByCategoryAsync(string slug, FeedQuery q, CancellationToken ct);
    Task<Result<PostDto>> GetByIdAsync(Guid id, CancellationToken ct);
    Task<Result<PostDto>> CreateAsync(CreatePostDto dto, CancellationToken ct);
    Task<Result<PostDto>> UpdateAsync(Guid id, UpdatePostDto dto, CancellationToken ct);
    Task<Result> DeleteAsync(Guid id, CancellationToken ct);
    Task<Result> LikeAsync(Guid id, CancellationToken ct);
    Task<Result> UnlikeAsync(Guid id, CancellationToken ct);
    Task<Result> SaveAsync(Guid id, CancellationToken ct);
    Task<Result> UnsaveAsync(Guid id, CancellationToken ct);
    Task<PagedResult<PostListItemDto>> GetMineAsync(FeedQuery q, CancellationToken ct);
    Task<PagedResult<PostListItemDto>> GetMySavedAsync(FeedQuery q, CancellationToken ct);
}
```

backend/src/Therapp.Application/Posts/PostService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Common;
using Therapp.Application.Notifications;
using Therapp.Application.Posts.Dtos;
using Therapp.Domain.Entities;
using Therapp.Domain.Enums;

namespace Therapp.Application.Posts;

public class PostService : IPostService
{
    private readonly IAppDbContext _db;
    private readonly ICurrentUser _current;
    private readonly INotificationService _notifier;

    public PostService(IAppDbContext db, ICurrentUser current, INotificationService notifier)
    {
        _db = db;
        _current = current;
        _notifier = notifier;
    }

    // ----- Reads -----

    public Task<PagedResult<PostListItemDto>> GetFeedAsync(FeedQuery q, CancellationToken ct) =>
        PageAsync(_db.Posts.Where(p => !p.IsHidden), q, ct);

    public async Task<PagedResult<PostListItemDto>> GetByCategoryAsync(string slug, FeedQuery q,
        CancellationToken ct)
    {
        var category = await _db.Categories.FirstOrDefaultAsync(c => c.Slug == slug && c.IsActive, ct);
        if (category is null) return Empty(q);

        return await PageAsync(
            _db.Posts.Where(p => !p.IsHidden && p.CategoryId == category.Id),
            q, ct);
    }

    public async Task<Result<PostDto>> GetByIdAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        var post = await _db.Posts
            .Where(p => p.Id == id && !p.IsHidden)
            .Select(p => new PostDto
            {
                Id = p.Id,
                Title = p.Title,
                Content = p.Content,
                CategoryId = p.CategoryId,
                IsHidden = p.IsHidden,
                CreatedAt = p.CreatedAt,
                UpdatedAt = p.UpdatedAt,
                User = new UserDto { Id = p.UserId, Name = p.UserName, Email = p.Email, IsActive = p.IsActive }
            })
            .FirstOrDefaultAsync(ct);

        if (post is null) return Result.NotFound();

        return Result.Ok(post);
    }
}
```

```

        Id = p.Id,
        Title = p.Title,
        Content = p.Content,
        IsAnonymous = p.IsAnonymous,
        Category = new PostCategoryDto
        {
            Id = p.Category.Id,
            Slug = p.Category.Slug,
            Name = p.Category.Name,
            Color = p.Category.Color,
            Icon = p.Category.Icon
        },
        Author = p.IsAnonymous ? null : new PostAuthorDto
        {
            Id = p.User.Id,
            Username = p.User.Username,
            DisplayName = p.User.DisplayName,
            AvatarUrl = p.User.AvatarUrl
        },
        LikeCount = p.LikeCount,
        CommentCount = p.CommentCount,
        SaveCount = p.SaveCount,
        CreatedAt = p.CreatedAt,
        UpdatedAt = p.UpdatedAt,
        IsLikedByMe = userId != null && p.Likes.Any(l => l.UserId == userId),
        IsSavedByMe = userId != null && p.Saves.Any(s => s.UserId == userId),
        IsMine = userId != null && p.UserId == userId
    })
    .FirstOrDefaultAsync(ct);

return post is null
    ? Result<PostDto>.Fail(Errors.NotFound("Gönderi"))
    : Result<PostDto>.Ok(post);
}

public async Task<PagedResult<PostListItemDto>> GetMineAsync(FeedQuery q, CancellationToken ct)
{
    var userId = _current.UserId;
    if (userId is null) return Empty(q);
    return await PageAsync(_db.Posts.Where(p => p.UserId == userId), q, ct);
}

public async Task<PagedResult<PostListItemDto>> GetMySavedAsync(FeedQuery q, CancellationToken ct)
{
    var userId = _current.UserId;
    if (userId is null) return Empty(q);

    var query = _db.SavedPosts
        .Where(s => s.UserId == userId)
        .OrderByDescending(s => s.CreatedAt)
        .Select(s => s.Post)
        .Where(p => !p.IsHidden && p.DeletedAt == null);

    return await PageAsync(query, q, ct, skipSort: true);
}

// ----- Writes -----

public async Task<Result<PostDto>> CreateAsync(CreatePostDto dto, CancellationToken ct)
{
    var userId = _current.UserId;
    if (userId is null) return Result<PostDto>.Fail(Errors.Unauthorized);

    var categoryExists = await _db.Categories.AnyAsync(c => c.Id == dto.CategoryId && c.IsActive, ct);
    if (!categoryExists) return Result<PostDto>.Fail(Errors.NotFound("Kategori"));

    var post = new Post
    {
        UserId = userId.Value,
        CategoryId = dto.CategoryId,
        Title = dto.Title.Trim(),
        Content = dto.Content.Trim(),
        IsAnonymous = dto.IsAnonymous
    };
    _db.Posts.Add(post);
    await _db.SaveChangesAsync(ct);

    return await GetByIdAsync(post.Id, ct);
}

public async Task<Result<PostDto>> UpdateAsync(Guid id, UpdatePostDto dto, CancellationToken ct)
{
    var userId = _current.UserId;
    if (userId is null) return Result<PostDto>.Fail(Errors.Unauthorized);

    var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id, ct);
    if (post is null) return Result<PostDto>.Fail(Errors.NotFound("Gönderi"));
    if (post.UserId != userId) return Result<PostDto>.Fail(Errors.Forbidden);

```

```

        post.Title = dto.Title.Trim();
        post.Content = dto.Content.Trim();
        await _db.SaveChangesAsync(ct);

        return await GetByIdAsync(id, ct);
    }

    public async Task<Result> DeleteAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id, ct);
        if (post is null) return Result.Fail(Errors.NotFound("Gönderi"));

        var isOwner = post.UserId == userId;
        var isMod = _current.IsInRole(nameof(UserRole.Moderator)) ||
        _current.IsInRole(nameof(UserRole.Admin));
        if (!isOwner && !isMod) return Result.Fail(Errors.Forbidden);

        post.DeletedAt = DateTime.UtcNow;
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    public async Task<Result> LikeAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id && !p.IsHidden, ct);
        if (post is null) return Result.Fail(Errors.NotFound("Gönderi"));

        var already = await _db.PostLikes.AnyAsync(l => l.UserId == userId && l.PostId == id, ct);
        if (already) return Result.Ok();

        _db.PostLikes.Add(new PostLike { UserId = userId.Value, PostId = id });
        post.LikeCount += 1;
        await _db.SaveChangesAsync(ct);

        if (post.UserId != userId.Value)
        {
            await _notifier.CreateAsync(post.UserId, userId.Value, NotificationType.PostLike, post.Id, null,
            null, ct);
        }

        return Result.Ok();
    }

    public async Task<Result> UnlikeAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id, ct);
        if (post is null) return Result.Fail(Errors.NotFound("Gönderi"));

        var like = await _db.PostLikes.FirstOrDefaultAsync(l => l.UserId == userId && l.PostId == id, ct);
        if (like is null) return Result.Ok();

        _db.PostLikes.Remove(like);
        if (post.LikeCount > 0) post.LikeCount -= 1;
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    public async Task<Result> SaveAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id && !p.IsHidden, ct);
        if (post is null) return Result.Fail(Errors.NotFound("Gönderi"));

        var already = await _db.SavedPosts.AnyAsync(s => s.UserId == userId && s.PostId == id, ct);
        if (already) return Result.Ok();

        _db.SavedPosts.Add(new SavedPost { UserId = userId.Value, PostId = id });
        post.SaveCount += 1;
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    public async Task<Result> UnsaveAsync(Guid id, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);
    }

```

```

        var post = await _db.Posts.FirstOrDefaultAsync(p => p.Id == id, ct);
        if (post is null) return Result.Fail(Errors.NotFound("Gönderi"));

        var saved = await _db.SavedPosts.FirstOrDefaultAsync(s => s.UserId == userId && s.PostId == id, ct);
        if (saved is null) return Result.Ok();

        _db.SavedPosts.Remove(saved);
        if (post.SaveCount > 0) post.SaveCount -= 1;
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    // ----- Helpers -----

    private async Task<PagedResult<PostListItemDto>> PageAsync(
        IQueryable<Post> baseQuery, FeedQuery q, CancellationToken ct, bool skipSort = false)
    {
        var userId = _current.UserId;

        var query = baseQuery;
        if (!skipSort)
        {
            query = q.Sort == FeedSort.Popular
                ? query.OrderByDescending(p => p.LikeCount).ThenByDescending(p => p.CreatedAt)
                : query.OrderByDescending(p => p.CreatedAt);
        }

        var total = await query.CountAsync(ct);

        var items = await query
            .Skip((q.Page - 1) * q.PageSize)
            .Take(q.PageSize)
            .Select(p => new PostListItemDto
            {
                Id = p.Id,
                Title = p.Title,
                Excerpt = p.Content.Length > 200 ? p.Content.Substring(0, 200) : p.Content,
                IsAnonymous = p.IsAnonymous,
                Category = new PostCategoryDto
                {
                    Id = p.Category.Id,
                    Slug = p.Category.Slug,
                    Name = p.Category.Name,
                    Color = p.Category.Color,
                    Icon = p.Category.Icon
                },
                Author = p.IsAnonymous ? null : new PostAuthorDto
                {
                    Id = p.User.Id,
                    Username = p.User.Username,
                    DisplayName = p.User.DisplayName,
                    AvatarUrl = p.User.AvatarUrl
                },
                LikeCount = p.LikeCount,
                CommentCount = p.CommentCount,
                SaveCount = p.SaveCount,
                CreatedAt = p.CreatedAt,
                IsLikedByMe = userId != null && p.Likes.Any(l => l.UserId == userId),
                IsSavedByMe = userId != null && p.Saves.Any(s => s.UserId == userId),
            })
            .ToListAsync(ct);

        return new PagedResult<PostListItemDto>
        {
            Items = items,
            Page = q.Page,
            PageSize = q.PageSize,
            TotalCount = total
        };
    }

    private static PagedResult<PostListItemDto> Empty(FeedQuery q) => new()
    {
        Items = Array.Empty<PostListItemDto>(),
        Page = q.Page,
        PageSize = q.PageSize,
        TotalCount = 0
    };
}

```

backend/src/Therapp.Application/Reports/Dtos/CreateReportDto.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Enums;

```

```

namespace Therapp.Application.Reports.Dtos;

public class CreateReportDto
{
    public ReportTargetType TargetType { get; set; }
    public Guid TargetId { get; set; }
    public ReportReason Reason { get; set; }
    public string? Description { get; set; }
}

```

backend/src/Therapp.Application/Reports/IReportService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Common;
using Therapp.Application.Reports.Dtos;

namespace Therapp.Application.Reports;

public interface IReportService
{
    Task<Result> CreateAsync(CreateReportDto dto, CancellationToken ct);
}

```

backend/src/Therapp.Application/Reports/ReportService.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Common;
using Therapp.Application.Reports.Dtos;
using Therapp.Domain.Entities;
using Therapp.Domain.Enums;

namespace Therapp.Application.Reports;

public class ReportService : IReportService
{
    private readonly IAppDbContext _db;
    private readonly ICurentUser _current;

    public ReportService(IAppDbContext db, ICurentUser current)
    {
        _db = db;
        _current = current;
    }

    public async Task<Result> CreateAsync(CreateReportDto dto, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        // Validate target exists
        var exists = dto.TargetType switch
        {
            ReportTargetType.Post => await _db.Posts.AnyAsync(p => p.Id == dto.TargetId, ct),
            ReportTargetType.Comment => await _db.Comments.AnyAsync(c => c.Id == dto.TargetId, ct),
            ReportTargetType.User => await _db.Users.AnyAsync(u => u.Id == dto.TargetId, ct),
            _ => false
        };
        if (!exists) return Result.Fail(Errors.NotFound("Hedef"));

        // Prevent duplicate pending reports by same user for same target
        var duplicate = await _db.Reports.AnyAsync(r =>
            r.ReporterUserId == userId &&
            r.TargetType == dto.TargetType &&
            r.TargetId == dto.TargetId &&
            r.Status == ReportStatus.Pending, ct);
        if (duplicate) return Result.Fail(Errors.Conflict("Bu içeriği zaten raporladınız."));

        _db.Reports.Add(new Report
        {
            ReporterUserId = userId.Value,
            TargetType = dto.TargetType,
            TargetId = dto.TargetId,
            Reason = dto.Reason,
            Description = string.IsNullOrEmpty(dto.Description) ? null : dto.Description.Trim(),
            Status = ReportStatus.Pending
        });
        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }
}

```

backend/src/Therapp.Application/Therapp.Application.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>Therapp.Application</RootNamespace>
    <AssemblyName>Therapp.Application</AssemblyName>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\Therapp.Domain\Therapp.Domain.csproj" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="FluentValidation" Version="11.9.2" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.10" />
    <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="8.0.2" />
    <PackageReference Include="Microsoft.Extensions.Options" Version="8.0.2" />
    <PackageReference Include="Microsoft.Extensions.Options.ConfigurationExtensions" Version="8.0.0" />
    <PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="7.6.0" />
    <PackageReference Include="Microsoft.IdentityModel.Tokens" Version="7.6.0" />
  </ItemGroup>
</Project>
```

backend/src/Therapp.Application/Users/Dtos/UserDtos.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Application.Users.Dtos;

public class UserMeDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string Email { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
    public string? Bio { get; set; }
    public int Role { get; set; }
    public DateTime CreatedAt { get; set; }
    public int PostCount { get; set; }
    public int SavedCount { get; set; }
}

public class UserProfileDto
{
    public Guid Id { get; set; }
    public string Username { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
    public string? Bio { get; set; }
    public DateTime CreatedAt { get; set; }
    public int PostCount { get; set; }
}

public class UpdateProfileDto
{
    public string DisplayName { get; set; } = null!;
    public string? Bio { get; set; }
    public string? AvatarUrl { get; set; }
}
```

backend/src/Therapp.Application/Users/IUserService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Common;
using Therapp.Application.Users.Dtos;

namespace Therapp.Application.Users;

public interface IUserService
{
    Task<Result<UserMeDto>> GetMeAsync(CancellationToken ct);
    Task<Result<UserMeDto>> UpdateMeAsync(UpdateProfileDto dto, CancellationToken ct);
    Task<Result> DeleteMeAsync(CancellationToken ct);
    Task<Result<UserProfileDto>> GetByUsernameAsync(string username, CancellationToken ct);
}
```

backend/src/Therapp.Application/Users/UserService.cs

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Application.Common;
using Therapp.Application.Users.Dtos;
```

```

namespace Therapp.Application.Users;

public class UserService : IUserService
{
    private readonly IAppDbContext _db;
    private readonly ICurrentUser _current;

    public UserService(IAppDbContext db, ICurrentUser current)
    {
        _db = db;
        _current = current;
    }

    public async Task<Result<UserMeDto>> GetMeAsync(CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result<UserMeDto>.Fail(Errors.Unauthorized);

        var dto = await _db.Users
            .Where(u => u.Id == userId)
            .Select(u => new UserMeDto
            {
                Id = u.Id,
                Username = u.Username,
                Email = u.Email,
                DisplayName = u.DisplayName,
                AvatarUrl = u.AvatarUrl,
                Bio = u.Bio,
                Role = (int)u.Role,
                CreatedAt = u.CreatedAt,
                PostCount = u.Posts.Count(p => !p.IsHidden && p.DeletedAt == null),
                SavedCount = _db.SavedPosts.Count(s => s.UserId == u.Id)
            })
            .FirstOrDefaultAsync(ct);

        return dto is null
            ? Result<UserMeDto>.Fail(Errors.NotFound("Kullanıcı"))
            : Result<UserMeDto>.Ok(dto);
    }

    public async Task<Result<UserMeDto>> UpdateMeAsync(UpdateProfileDto dto, CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result<UserMeDto>.Fail(Errors.Unauthorized);

        var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == userId, ct);
        if (user is null) return Result<UserMeDto>.Fail(Errors.NotFound("Kullanıcı"));

        user.DisplayName = dto.DisplayName.Trim();
        user.Bio = string.IsNullOrEmpty(dto.Bio) ? null : dto.Bio.Trim();
        user.AvatarUrl = string.IsNullOrEmpty(dto.AvatarUrl) ? null : dto.AvatarUrl.Trim();

        await _db.SaveChangesAsync(ct);
        return await GetMeAsync(ct);
    }

    public async Task<Result> DeleteMeAsync(CancellationToken ct)
    {
        var userId = _current.UserId;
        if (userId is null) return Result.Fail(Errors.Unauthorized);

        var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == userId, ct);
        if (user is null) return Result.Fail(Errors.NotFound("Kullanıcı"));

        // Soft delete + anonymize
        user.DeletedAt = DateTime.UtcNow;
        user.DisplayName = "Silinmiş Kullanıcı";
        user.Bio = null;
        user.AvatarUrl = null;

        // Revoke all refresh tokens
        var tokens = await _db.RefreshTokens
            .Where(t => t.UserId == userId && t.RevokedAt == null)
            .ToListAsync(ct);
        foreach (var t in tokens) t.RevokedAt = DateTime.UtcNow;

        await _db.SaveChangesAsync(ct);
        return Result.Ok();
    }

    public async Task<Result<UserProfileDto>> GetByUsernameAsync(string username, CancellationToken ct)
    {
        var normalized = username.Trim().ToLowerInvariant();
        var dto = await _db.Users
            .Where(u => u.Username == normalized)
            .Select(u => new UserProfileDto
            {
                Id = u.Id,
            })
            .FirstOrDefaultAsync(ct);
    }
}

```

```

        Username = u.Username,
        DisplayName = u.DisplayName,
        AvatarUrl = u.AvatarUrl,
        Bio = u.Bio,
        CreatedAt = u.CreatedAt,
        PostCount = u.Posts.Count(p => !p.IsHidden && !p.IsAnonymous && p.DeletedAt == null)
    })
    .FirstOrDefaultAsync(ct);

    return dto is null
        ? Result<UserProfileDto>.Fail(Errors.NotFound("Kullanıcı"))
        : Result<UserProfileDto>.Ok(dto);
}
}

```

backend/src/Therapp.Application/Validators/CreateCommentDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Comments.Dtos;

namespace Therapp.Application.Validators;

public class CreateCommentDtoValidator : AbstractValidator<CreateCommentDto>
{
    public CreateCommentDtoValidator()
    {
        RuleFor(x => x.Content)
            .NotEmpty().WithMessage("Yorum boş olamaz.")
            .MinimumLength(2).WithMessage("Yorum çok kısa.")
            .MaximumLength(1000).WithMessage("Yorum en fazla 1000 karakter olabilir.");
    }
}

```

backend/src/Therapp.Application/Validators/CreatePostDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Posts.Dtos;

namespace Therapp.Application.Validators;

public class CreatePostDtoValidator : AbstractValidator<CreatePostDto>
{
    public CreatePostDtoValidator()
    {
        RuleFor(x => x.CategoryId).NotEmpty().WithMessage("Kategori seçmelisin.");
        RuleFor(x => x.Title)
            .NotEmpty().WithMessage("Başlık zorunlu.")
            .Length(3, 150).WithMessage("Başlık 3-150 karakter olmalı.");
        RuleFor(x => x.Content)
            .NotEmpty().WithMessage("İçerik zorunlu.")
            .MinimumLength(10).WithMessage("İçerik en az 10 karakter olmalı.")
            .MaximumLength(5000).WithMessage("İçerik en fazla 5000 karakter olabilir.");
    }
}

public class UpdatePostDtoValidator : AbstractValidator<UpdatePostDto>
{
    public UpdatePostDtoValidator()
    {
        RuleFor(x => x.Title)
            .NotEmpty().Length(3, 150);
        RuleFor(x => x.Content)
            .NotEmpty().MinimumLength(10).MaximumLength(5000);
    }
}

```

backend/src/Therapp.Application/Validators/CreateReportDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Reports.Dtos;

namespace Therapp.Application.Validators;

public class CreateReportDtoValidator : AbstractValidator<CreateReportDto>
{
    public CreateReportDtoValidator()
    {
        RuleFor(x => x.TargetType).IsInEnum();
        RuleFor(x => x.Reason).IsInEnum();
        RuleFor(x => x.TargetId).NotEmpty();
    }
}

```

```

    }
    RuleFor(x => x.Description).HasMaxLength(500);
}

```

backend/src/Therapp.Application/Validators/LoginDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Auth.Dtos;

namespace Therapp.Application.Validators;

public class LoginDtoValidator : AbstractValidator<LoginDto>
{
    public LoginDtoValidator()
    {
        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Password).NotEmpty().HasMaxLength(128);
    }
}

```

backend/src/Therapp.Application/Validators/RegisterDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Auth.Dtos;

namespace Therapp.Application.Validators;

public class RegisterDtoValidator : AbstractValidator<RegisterDto>
{
    public RegisterDtoValidator()
    {
        RuleFor(x => x.Username)
            .NotEmpty().WithMessage("Kullanıcı adı zorunlu.")
            .Length(3, 32).WithMessage("Kullanıcı adı 3-32 karakter olmalı.")
            .Matches(@"^[a-zA-Z0-9_]+$").WithMessage("Kullanıcı adı yalnızca harf, rakam ve _ içerebilir.");

        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("E-posta zorunlu.")
            .EmailAddress().WithMessage("Geçerli bir e-posta girin.")
            .HasMaxLength(255);

        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Şifre zorunlu.")
            .MinimumLength(8).WithMessage("Şifre en az 8 karakter olmalı.")
            .HasMaxLength(128);

        RuleFor(x => x.DisplayName)
            .NotEmpty().WithMessage("Görünen ad zorunlu.")
            .Length(2, 64);
    }
}

```

backend/src/Therapp.Application/Validators/UpdateProfileDtoValidator.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using FluentValidation;
using Therapp.Application.Users.Dtos;

namespace Therapp.Application.Validators;

public class UpdateProfileDtoValidator : AbstractValidator<UpdateProfileDto>
{
    public UpdateProfileDtoValidator()
    {
        RuleFor(x => x.DisplayName)
            .NotEmpty().Length(2, 64);
        RuleFor(x => x.Bio)
            .HasMaxLength(280);
        RuleFor(x => x.AvatarUrl)
            .HasMaxLength(500);
    }
}

```

backend/src/Therapp.Domain/Common/BaseEntity.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Common;

public interface IHasCreatedAt
{
}

```

```

        DateTime CreatedAt { get; set; }
    }

    public abstract class BaseEntity : IHasCreatedAt
    {
        public Guid Id { get; set; } = Guid.NewGuid();
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
    }

    public abstract class SoftDeletableEntity : BaseEntity
    {
        public DateTime? DeletedAt { get; set; }
    }
}

```

backend/src/Therapp.Domain/Entities/Category.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class Category : BaseEntity
{
    public string Slug { get; set; } = null!;
    public string Name { get; set; } = null!;
    public string Description { get; set; } = null!;
    public string Icon { get; set; } = null!;
    public string Color { get; set; } = null!;
    public int OrderIndex { get; set; }
    public bool IsActive { get; set; } = true;

    public ICollection<Post> Posts { get; set; } = new List<Post>();
}

```

backend/src/Therapp.Domain/Entities/Comment.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class Comment : SoftDeletableEntity
{
    public Guid PostId { get; set; }
    public Post Post { get; set; } = null!;

    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid? ParentCommentId { get; set; }
    public Comment? ParentComment { get; set; }

    public string Content { get; set; } = null!;
    public int LikeCount { get; set; }
    public bool IsHidden { get; set; }

    public ICollection<Comment> Replies { get; set; } = new List<Comment>();
    public ICollection<CommentLike> Likes { get; set; } = new List<CommentLike>();
}

```

backend/src/Therapp.Domain/Entities/CommentLike.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class CommentLike : IHasCreatedAt
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid CommentId { get; set; }
    public Comment Comment { get; set; } = null!;

    public DateTime CreatedAt { get; set; }
}

```

backend/src/Therapp.Domain/Entities/Notification.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]

```

```

using Therapp.Domain.Common;
using Therapp.Domain.Enums;

namespace Therapp.Domain.Entities;

public class Notification : BaseEntity
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid? ActorUserId { get; set; }
    public User? Actor { get; set; }

    public NotificationType Type { get; set; }

    public Guid? PostId { get; set; }
    public Guid? CommentId { get; set; }

    public string? Message { get; set; }
    public bool IsRead { get; set; }
}

```

backend/src/Therapp.Domain/Entities/Post.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class Post : SoftDeletableEntity
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid CategoryId { get; set; }
    public Category Category { get; set; } = null!;

    public string Title { get; set; } = null!;
    public string Content { get; set; } = null!;
    public bool IsAnonymous { get; set; }

    public int LikeCount { get; set; }
    public int CommentCount { get; set; }
    public int SaveCount { get; set; }

    public bool IsHidden { get; set; }

    public ICollection<Comment> Comments { get; set; } = new List<Comment>();
    public ICollection<PostLike> Likes { get; set; } = new List<PostLike>();
    public ICollection<SavedPost> Saves { get; set; } = new List<SavedPost>();
}

```

backend/src/Therapp.Domain/Entities/PostLike.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class PostLike : IHasCreatedAt
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid PostId { get; set; }
    public Post Post { get; set; } = null!;

    public DateTime CreatedAt { get; set; }
}

```

backend/src/Therapp.Domain/Entities/RefreshToken.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class RefreshToken : BaseEntity
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public string TokenHash { get; set; } = null!;
}

```

```

        public DateTime ExpiresAt { get; set; }
        public DateTime? RevokedAt { get; set; }

        public bool IsActive => RevokedAt == null && DateTime.UtcNow < ExpiresAt;
    }

```

backend/src/Therapp.Domain/Entities/Report.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;
using Therapp.Domain.Enums;

namespace Therapp.Domain.Entities;

public class Report : BaseEntity
{
    public Guid ReporterUserId { get; set; }
    public User Reporter { get; set; } = null!;

    public ReportTargetType TargetType { get; set; }
    public Guid TargetId { get; set; }

    public ReportReason Reason { get; set; }
    public string? Description { get; set; }

    public ReportStatus Status { get; set; } = ReportStatus.Pending;

    public Guid? ResolvedByUserId { get; set; }
    public User? ResolvedBy { get; set; }
    public DateTime? ResolvedAt { get; set; }
}

```

backend/src/Therapp.Domain/Entities/SavedPost.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;

namespace Therapp.Domain.Entities;

public class SavedPost : IHasCreatedAt
{
    public Guid UserId { get; set; }
    public User User { get; set; } = null!;

    public Guid PostId { get; set; }
    public Post Post { get; set; } = null!;

    public DateTime CreatedAt { get; set; }
}

```

backend/src/Therapp.Domain/Entities/User.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Domain.Common;
using Therapp.Domain.Enums;

namespace Therapp.Domain.Entities;

public class User : SoftDeletableEntity
{
    public string Username { get; set; } = null!;
    public string Email { get; set; } = null!;
    public string PasswordHash { get; set; } = null!;
    public string DisplayName { get; set; } = null!;
    public string? AvatarUrl { get; set; }
    public string? Bio { get; set; }
    public UserRole Role { get; set; } = UserRole.User;
    public bool IsBanned { get; set; }
    public DateTime? AcceptedDisclaimerAt { get; set; }

    public ICollection<Post> Posts { get; set; } = new List<Post>();
    public ICollection<Comment> Comments { get; set; } = new List<Comment>();
    public ICollection<RefreshToken> RefreshTokens { get; set; } = new List<RefreshToken>();
}

```

backend/src/Therapp.Domain/Enums/NotificationType.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Enums;

public enum NotificationType : short
{

```

```

    PostLike = 0,
    Comment = 1,
    Reply = 2,
    CommentLike = 3,
    System = 4
}

```

backend/src/Therapp.Domain/Enums/ReportReason.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Enums;

public enum ReportReason : short
{
    Spam = 0,
    Harassment = 1,
    SelfHarm = 2,
    Misinformation = 3,
    HateSpeech = 4,
    SexualContent = 5,
    Other = 99
}

```

backend/src/Therapp.Domain/Enums/ReportStatus.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Enums;

public enum ReportStatus : short
{
    Pending = 0,
    Reviewed = 1,
    Actioned = 2,
    Dismissed = 3
}

```

backend/src/Therapp.Domain/Enums/ReportTargetType.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Enums;

public enum ReportTargetType : short
{
    Post = 0,
    Comment = 1,
    User = 2
}

```

backend/src/Therapp.Domain/Enums/UserRole.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
namespace Therapp.Domain.Enums;

public enum UserRole : short
{
    User = 0,
    Moderator = 1,
    Admin = 2
}

```

backend/src/Therapp.Domain/Therapp.Domain.csproj

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>Therapp.Domain</RootNamespace>
    <AssemblyName>Therapp.Domain</AssemblyName>
  </PropertyGroup>
</Project>

```

backend/src/Therapp.Infrastructure/Auth/BCryptPasswordHasher.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Therapp.Application.Abstractions;

namespace Therapp.Infrastructure.Auth;

public class BCryptPasswordHasher : IPasswordHasher
{
}

```

```

private const int WorkFactor = 11;

public string Hash(string password) =>
    BCrypt.Net.BCrypt.HashPassword(password, WorkFactor);

public bool Verify(string password, string hash)
{
    try { return BCrypt.Net.BCrypt.Verify(password, hash); }
    catch { return false; }
}
}

```

backend/src/Therapp.Infrastructure/DependencyInjection.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Therapp.Application.Abstractions;
using Therapp.Infrastructure.Auth;
using Therapp.Infrastructure.Persistence;

namespace Therapp.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguration cfg)
    {
        var connString = cfg.GetConnectionString("Postgres")
            ?? throw new InvalidOperationException("ConnectionStrings:Postgres missing");

        services.AddDbContext<AppDbContext>(opt =>
            opt.UseNpgsql(connString, npg =>
                npg.MigrationsAssembly(typeof(AppDbContext).Assembly.FullName));

        services.AddScoped<IAppDbContext>(sp => sp.GetRequiredService<AppDbContext>());
        services.AddScoped<IPasswordHasher, BCryptPasswordHasher>();

        return services;
    }
}

```

backend/src/Therapp.Infrastructure/Migrations/20260620204753_InitialCreate.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace Therapp.Infrastructure.Migrations
{
    /// <inheritdoc />
    public partial class InitialCreate : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.AlterDatabase()
                .Annotation("Npgsql:PostgresExtension:pgcrypto", ",", "");

            migrationBuilder.CreateTable(
                name: "categories",
                columns: table => new
                {
                    id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
                        "gen_random_uuid()"),
                    slug = table.Column<string>(type: "character varying(50)", maxLength: 50, nullable:
                        false),
                    name = table.Column<string>(type: "character varying(100)", maxLength: 100, nullable:
                        false),
                    description = table.Column<string>(type: "character varying(500)", maxLength: 500,
                        nullable: false),
                    icon = table.Column<string>(type: "character varying(50)", maxLength: 50, nullable:
                        false),
                    color = table.Column<string>(type: "character varying(7)", maxLength: 7, nullable:
                        false),
                    order_index = table.Column<int>(type: "integer", nullable: false, defaultValue: 0),
                    is_active = table.Column<bool>(type: "boolean", nullable: false, defaultValue: true),
                    created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
                    updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK categories", x => x.id);
                }
            );
        }
    }
}

```

```

    });

    migrationBuilder.CreateTable(
        name: "users",
        columns: table => new
        {
            id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
            username = table.Column<string>(type: "character varying(32)", maxLength: 32, nullable:
false),
            email = table.Column<string>(type: "character varying(255)", maxLength: 255, nullable:
false),
            password_hash = table.Column<string>(type: "character varying(255)", maxLength: 255,
nullable: false),
            display_name = table.Column<string>(type: "character varying(64)", maxLength: 64,
nullable: false),
            avatar_url = table.Column<string>(type: "character varying(500)", maxLength: 500,
nullable: true),
            bio = table.Column<string>(type: "character varying(280)", maxLength: 280, nullable:
true),
            role = table.Column<short>(type: "smallint", nullable: false, defaultValue: (short)0),
            is_banned = table.Column<bool>(type: "boolean", nullable: false, defaultValue: false),
            accepted_disclaimer_at = table.Column<DateTime>(type: "timestamp with time zone",
nullable: true),
            created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
            updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
            deleted_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_users", x => x.id);
        });

    migrationBuilder.CreateTable(
        name: "notifications",
        columns: table => new
        {
            id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
            user_id = table.Column<Guid>(type: "uuid", nullable: false),
            actor_user_id = table.Column<Guid>(type: "uuid", nullable: true),
            type = table.Column<short>(type: "smallint", nullable: false),
            post_id = table.Column<Guid>(type: "uuid", nullable: true),
            comment_id = table.Column<Guid>(type: "uuid", nullable: true),
            message = table.Column<string>(type: "character varying(255)", maxLength: 255, nullable:
true),
            is_read = table.Column<bool>(type: "boolean", nullable: false, defaultValue: false),
            created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
            updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_notifications", x => x.id);
            table.ForeignKey(
                name: "FK_notifications_users_actor_user_id",
                column: x => x.actor_user_id,
                principalTable: "users",
                principalColumn: "id",
                onDelete: ReferentialAction.SetNull);
            table.ForeignKey(
                name: "FK_notifications_users_user_id",
                column: x => x.user_id,
                principalTable: "users",
                principalColumn: "id",
                onDelete: ReferentialAction.Cascade);
        });

    migrationBuilder.CreateTable(
        name: "posts",
        columns: table => new
        {
            id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
            user_id = table.Column<Guid>(type: "uuid", nullable: false),
            category_id = table.Column<Guid>(type: "uuid", nullable: false),
            title = table.Column<string>(type: "character varying(150)", maxLength: 150, nullable:
false),
            content = table.Column<string>(type: "text", nullable: false),
            is_anonymous = table.Column<bool>(type: "boolean", nullable: false, defaultValue:
false),
            like_count = table.Column<int>(type: "integer", nullable: false, defaultValue: 0),
            comment_count = table.Column<int>(type: "integer", nullable: false, defaultValue: 0),
            save_count = table.Column<int>(type: "integer", nullable: false, defaultValue: 0),
            is_hidden = table.Column<bool>(type: "boolean", nullable: false, defaultValue: false),
            created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
            updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
            deleted_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
        },
    },

```

```

constraints: table =>
{
    table.PrimaryKey("PK_posts", x => x.id);
    table.ForeignKey(
        name: "FK_posts_categories_category_id",
        column: x => x.category_id,
        principalTable: "categories",
        principalColumn: "id",
        onDelete: ReferentialAction.Restrict);
    table.ForeignKey(
        name: "FK_posts_users_user_id",
        column: x => x.user_id,
        principalTable: "users",
        principalColumn: "id",
        onDelete: ReferentialAction.Restrict);
});

migrationBuilder.CreateTable(
    name: "refresh_tokens",
    columns: table => new
    {
        id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
        user_id = table.Column<Guid>(type: "uuid", nullable: false),
        token_hash = table.Column<string>(type: "character varying(255)", maxLength: 255,
nullable: false),
        expires_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        revoked_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: true),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_refresh_tokens", x => x.id);
        table.ForeignKey(
            name: "FK_refresh_tokens_users_user_id",
            column: x => x.user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateTable(
    name: "reports",
    columns: table => new
    {
        id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
        reporter_user_id = table.Column<Guid>(type: "uuid", nullable: false),
        target_type = table.Column<short>(type: "smallint", nullable: false),
        target_id = table.Column<Guid>(type: "uuid", nullable: false),
        reason = table.Column<short>(type: "smallint", nullable: false),
        description = table.Column<string>(type: "character varying(500)", maxLength: 500,
nullable: true),
        status = table.Column<short>(type: "smallint", nullable: false, defaultValue: (short)0),
        resolved_by_user_id = table.Column<Guid>(type: "uuid", nullable: true),
        resolved_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: true),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_reports", x => x.id);
        table.ForeignKey(
            name: "FK_reports_users_reporter_user_id",
            column: x => x.reporter_user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_reports_users_resolved_by_user_id",
            column: x => x.resolved_by_user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateTable(
    name: "comments",
    columns: table => new
    {
        id = table.Column<Guid>(type: "uuid", nullable: false, defaultValueSql:
"gen_random_uuid()"),
        post_id = table.Column<Guid>(type: "uuid", nullable: false),
        user_id = table.Column<Guid>(type: "uuid", nullable: false),
        parent_comment_id = table.Column<Guid>(type: "uuid", nullable: true),
        content = table.Column<string>(type: "character varying(1000)", maxLength: 1000,
nullable: false),

```

```

        like_count = table.Column<int>(type: "integer", nullable: false, defaultValue: 0),
        is_hidden = table.Column<bool>(type: "boolean", nullable: false, defaultValue: false),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        updated_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        deleted_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_comments", x => x.id);
        table.ForeignKey(
            name: "FK_comments_comments_parent_comment_id",
            column: x => x.parent_comment_id,
            principalTable: "comments",
            principalColumn: "id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_comments_posts_post_id",
            column: x => x.post_id,
            principalTable: "posts",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_comments_users_user_id",
            column: x => x.user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateTable(
    name: "post_likes",
    columns: table => new
    {
        user_id = table.Column<Guid>(type: "uuid", nullable: false),
        post_id = table.Column<Guid>(type: "uuid", nullable: false),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_post_likes", x => new { x.user_id, x.post_id });
        table.ForeignKey(
            name: "FK_post_likes_posts_post_id",
            column: x => x.post_id,
            principalTable: "posts",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_post_likes_users_user_id",
            column: x => x.user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateTable(
    name: "saved_posts",
    columns: table => new
    {
        user_id = table.Column<Guid>(type: "uuid", nullable: false),
        post_id = table.Column<Guid>(type: "uuid", nullable: false),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_saved_posts", x => new { x.user_id, x.post_id });
        table.ForeignKey(
            name: "FK_saved_posts_posts_post_id",
            column: x => x.post_id,
            principalTable: "posts",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_saved_posts_users_user_id",
            column: x => x.user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateTable(
    name: "comment_likes",
    columns: table => new
    {
        user_id = table.Column<Guid>(type: "uuid", nullable: false),
        comment_id = table.Column<Guid>(type: "uuid", nullable: false),
        created_at = table.Column<DateTime>(type: "timestamp with time zone", nullable: false)
    },
    constraints: table =>

```

```

    {
        table.PrimaryKey("PK_comment_likes", x => new { x.user_id, x.comment_id });
        table.ForeignKey(
            name: "FK_comment_likes_comments_comment_id",
            column: x => x.comment_id,
            principalTable: "comments",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_comment_likes_users_user_id",
            column: x => x.user_id,
            principalTable: "users",
            principalColumn: "id",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateIndex(
    name: "ux_categories_slug",
    table: "categories",
    column: "slug",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_comment_likes_comment_id",
    table: "comment_likes",
    column: "comment_id");

migrationBuilder.CreateIndex(
    name: "IX_comments_user_id",
    table: "comments",
    column: "user_id");

migrationBuilder.CreateIndex(
    name: "ix_comments_parent",
    table: "comments",
    column: "parent_comment_id");

migrationBuilder.CreateIndex(
    name: "ix_comments_post_created",
    table: "comments",
    columns: new[] { "post_id", "created_at" });

migrationBuilder.CreateIndex(
    name: "IX_notifications_actor_user_id",
    table: "notifications",
    column: "actor_user_id");

migrationBuilder.CreateIndex(
    name: "ix_notifications_user_created",
    table: "notifications",
    columns: new[] { "user_id", "created_at" });

migrationBuilder.CreateIndex(
    name: "ix_notifications_user_unread",
    table: "notifications",
    columns: new[] { "user_id", "is_read" });

migrationBuilder.CreateIndex(
    name: "IX_post_likes_post_id",
    table: "post_likes",
    column: "post_id");

migrationBuilder.CreateIndex(
    name: "ix_posts_category_created",
    table: "posts",
    columns: new[] { "category_id", "created_at" });

migrationBuilder.CreateIndex(
    name: "ix_posts_created",
    table: "posts",
    column: "created_at");

migrationBuilder.CreateIndex(
    name: "ix_posts_user_created",
    table: "posts",
    columns: new[] { "user_id", "created_at" });

migrationBuilder.CreateIndex(
    name: "ix_refresh_tokens_user",
    table: "refresh_tokens",
    column: "user_id");

migrationBuilder.CreateIndex(
    name: "ux_refresh_tokens_hash",
    table: "refresh_tokens",
    column: "token_hash",
    unique: true);

```

```

        migrationBuilder.CreateIndex(
            name: "IX_reports_reporter_user_id",
            table: "reports",
            column: "reporter_user_id");

        migrationBuilder.CreateIndex(
            name: "IX_reports_resolved_by_user_id",
            table: "reports",
            column: "resolved_by_user_id");

        migrationBuilder.CreateIndex(
            name: "ix_reports_status_created",
            table: "reports",
            columns: new[] { "status", "created_at" });

        migrationBuilder.CreateIndex(
            name: "ix_reports_target",
            table: "reports",
            columns: new[] { "target_type", "target_id" });

        migrationBuilder.CreateIndex(
            name: "IX_saved_posts_post_id",
            table: "saved_posts",
            column: "post_id");

        migrationBuilder.CreateIndex(
            name: "ix_saved_posts_user_created",
            table: "saved_posts",
            columns: new[] { "user_id", "created_at" });

        migrationBuilder.CreateIndex(
            name: "ux_users_email",
            table: "users",
            column: "email",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "ux_users_username",
            table: "users",
            column: "username",
            unique: true);
    }

    /// <inheritdoc />
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(
            name: "comment_likes");

        migrationBuilder.DropTable(
            name: "notifications");

        migrationBuilder.DropTable(
            name: "post_likes");

        migrationBuilder.DropTable(
            name: "refresh_tokens");

        migrationBuilder.DropTable(
            name: "reports");

        migrationBuilder.DropTable(
            name: "saved_posts");

        migrationBuilder.DropTable(
            name: "comments");

        migrationBuilder.DropTable(
            name: "posts");

        migrationBuilder.DropTable(
            name: "categories");

        migrationBuilder.DropTable(
            name: "users");
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/AppDbContext.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Application.Abstractions;
using Therapp.Domain.Common;
using Therapp.Domain.Entities;

```

```

namespace Therapp.Infrastructure.Persistence;

public class AppDbContext : DbContext, IAppDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Category> Categories => Set<Category>();
    public DbSet<Post> Posts => Set<Post>();
    public DbSet<Comment> Comments => Set<Comment>();
    public DbSet<PostLike> PostLikes => Set<PostLike>();
    public DbSet<CommentLike> CommentLikes => Set<CommentLike>();
    public DbSet<SavedPost> SavedPosts => Set<SavedPost>();
    public DbSet<Report> Reports => Set<Report>();
    public DbSet<Notification> Notifications => Set<Notification>();
    public DbSet<RefreshToken> RefreshTokens => Set<RefreshToken>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.HasPostgresExtension("pgcrypto");
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }

    public override Task<int> SaveChangesAsync(CancellationToken ct = default)
    {
        var now = DateTime.UtcNow;

        foreach (var entry in ChangeTracker.Entries<IHasCreatedAt>())
        {
            if (entry.State == EntityState.Added && entry.Entity.CreatedAt == default)
                entry.Entity.CreatedAt = now;
        }

        foreach (var entry in ChangeTracker.Entries<BaseEntity>())
        {
            if (entry.State == EntityState.Added || entry.State == EntityState.Modified)
                entry.Entity.UpdatedAt = now;
        }

        return base.SaveChangesAsync(ct);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/CategoryConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class CategoryConfiguration : IEntityTypeConfiguration<Category>
{
    public void Configure(EntityTypeBuilder<Category> b)
    {
        {
            b.ToTable("categories");
            b.HasKey(x => x.Id);
            b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

            b.Property(x => x.Slug).HasColumnName("slug").HasMaxLength(50).IsRequired();
            b.Property(x => x.Name).HasColumnName("name").HasMaxLength(100).IsRequired();
            b.Property(x => x.Description).HasColumnName("description").HasMaxLength(500).IsRequired();
            b.Property(x => x.Icon).HasColumnName("icon").HasMaxLength(50).IsRequired();
            b.Property(x => x.Color).HasColumnName("color").HasMaxLength(7).IsRequired();
            b.Property(x => x.OrderIndex).HasColumnName("order_index").HasDefaultValue(0);
            b.Property(x => x.IsActive).HasColumnName("is_active").HasDefaultValue(true);
            b.Property(x => x.CreatedAt).HasColumnName("created_at");
            b.Property(x => x.UpdatedAt).HasColumnName("updated_at");

            b.HasIndex(x => x.Slug).IsUnique().HasDatabaseName("ux_categories_slug");
        }
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/CommentConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

```

```

public class CommentConfiguration : IEntityTypeConfiguration<Comment>
{
    public void Configure(EntityTypeBuilder<Comment> b)
    {
        b.ToTable("comments");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.PostId).HasColumnName("post_id");
        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.ParentCommentId).HasColumnName("parent_comment_id");
        b.Property(x => x.Content).HasColumnName("content").HasMaxLength(1000).IsRequired();
        b.Property(x => x.LikeCount).HasColumnName("like_count").HasDefaultValue(0);
        b.Property(x => x.IsHidden).HasColumnName("is_hidden").HasDefaultValue(false);
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");
        b.Property(x => x.DeletedAt).HasColumnName("deleted_at");

        b.HasOne(x => x.Post)
            .WithMany(p => p.Comments)
            .HasForeignKey(x => x.PostId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(x => x.User)
            .WithMany(u => u.Comments)
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(x => x.ParentComment)
            .WithMany(c => c.Replies)
            .HasForeignKey(x => x.ParentCommentId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(x => new { x.PostId, x.CreatedAt }).HasDatabaseName("ix_comments_post_created");
        b.HasIndex(x => x.ParentCommentId).HasDatabaseName("ix_comments_parent");

        b.HasQueryFilter(x => x.DeletedAt == null);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/CommentLikeConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class CommentLikeConfiguration : IEntityTypeConfiguration<CommentLike>
{
    public void Configure(EntityTypeBuilder<CommentLike> b)
    {
        b.ToTable("comment_likes");
        b.HasKey(x => new { x.UserId, x.CommentId });

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.CommentId).HasColumnName("comment_id");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");

        b.HasOne(x => x.User)
            .WithMany()
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(x => x.Comment)
            .WithMany(c => c.Likes)
            .HasForeignKey(x => x.CommentId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/NotificationConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class NotificationConfiguration : IEntityTypeConfiguration<Notification>
{
    public void Configure(EntityTypeBuilder<Notification> b)
    {
    }
}

```

```

        b.ToTable("notifications");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.ActorUserId).HasColumnName("actor_user_id");
        b.Property(x => x.Type).HasColumnName("type").HasConversion<short>();
        b.Property(x => x.PostId).HasColumnName("post_id");
        b.Property(x => x.CommentId).HasColumnName("comment_id");
        b.Property(x => x.Message).HasColumnName("message").HasMaxLength(255);
        b.Property(x => x.IsRead).HasColumnName("is_read").HasDefaultValue(false);
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");

        b.HasOne(x => x.User)
            .WithMany()
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(x => x.Actor)
            .WithMany()
            .HasForeignKey(x => x.ActorUserId)
            .OnDelete(DeleteBehavior.SetNull);

        b.HasIndex(x => new { x.UserId, x.CreatedAt }).HasDatabaseName("ix_notifications_user_created");
        b.HasIndex(x => new { x.UserId, x.IsRead }).HasDatabaseName("ix_notifications_user_unread");
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/PostConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class PostConfiguration : IEntityTypeConfiguration<Post>
{
    public void Configure(EntityTypeBuilder<Post> b)
    {
        b.ToTable("posts");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.CategoryId).HasColumnName("category_id");
        b.Property(x => x.Title).HasColumnName("title").HasMaxLength(150).IsRequired();
        b.Property(x => x.Content).HasColumnName("content").IsRequired();
        b.Property(x => x.IsAnonymous).HasColumnName("is_anonymous").HasDefaultValue(false);
        b.Property(x => x.LikeCount).HasColumnName("like_count").HasDefaultValue(0);
        b.Property(x => x.CommentCount).HasColumnName("comment_count").HasDefaultValue(0);
        b.Property(x => x.SaveCount).HasColumnName("save_count").HasDefaultValue(0);
        b.Property(x => x.IsHidden).HasColumnName("is_hidden").HasDefaultValue(false);
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");
        b.Property(x => x.DeletedAt).HasColumnName("deleted_at");

        b.HasOne(x => x.User)
            .WithMany(u => u.Posts)
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(x => x.Category)
            .WithMany(c => c.Posts)
            .HasForeignKey(x => x.CategoryId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(x => new { x.CategoryId, x.CreatedAt }).HasDatabaseName("ix_posts_category_created");
        b.HasIndex(x => new { x.UserId, x.CreatedAt }).HasDatabaseName("ix_posts_user_created");
        b.HasIndex(x => x.CreatedAt).HasDatabaseName("ix_posts_created");

        b.HasQueryFilter(x => x.DeletedAt == null);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/PostLikeConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

```

```

public class PostLikeConfiguration : IEntityTypeConfiguration<PostLike>
{
    public void Configure(EntityTypeBuilder<PostLike> b)
    {
        b.ToTable("post_likes");
        b.HasKey(x => new { x.UserId, x.PostId });

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.PostId).HasColumnName("post_id");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");

        b.HasOne(x => x.User)
            .WithMany()
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(x => x.Post)
            .WithMany(p => p.Likes)
            .HasForeignKey(x => x.PostId)
            .OnDelete(DeleteBehavior.Cascade);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/RefreshTokenConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class RefreshTokenConfiguration : IEntityTypeConfiguration<RefreshToken>
{
    public void Configure(EntityTypeBuilder<RefreshToken> b)
    {
        b.ToTable("refresh_tokens");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.TokenHash).HasColumnName("token_hash").HasMaxLength(255).IsRequired();
        b.Property(x => x.ExpiresAt).HasColumnName("expires_at");
        b.Property(x => x.RevokedAt).HasColumnName("revoked_at");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");

        b.Ignore(x => x.IsActive);

        b.HasOne(x => x.User)
            .WithMany(u => u.RefreshTokens)
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasIndex(x => x.TokenHash).IsUnique().HasDatabaseName("ux_refresh_tokens_hash");
        b.HasIndex(x => x.UserId).HasDatabaseName("ix_refresh_tokens_user");
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/ReportConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class ReportConfiguration : IEntityTypeConfiguration<Report>
{
    public void Configure(EntityTypeBuilder<Report> b)
    {
        b.ToTable("reports");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.ReporterUserId).HasColumnName("reporter_user_id");
        b.Property(x => x.TargetType).HasColumnName("target_type").HasConversion<short>();
        b.Property(x => x.TargetId).HasColumnName("target_id");
        b.Property(x => x.Reason).HasColumnName("reason").HasConversion<short>();
        b.Property(x => x.Description).HasColumnName("description").HasMaxLength(500);
        b.Property(x => x.Status).HasColumnName("status").HasConversion<short>().HasDefaultValue(Therapp.Domain.Enums.ReportStatus.Pending);
    }
}

```

```

        b.Property(x => x.ResolvedById).HasColumnName("resolved_by_user_id");
        b.Property(x => x.ResolvedAt).HasColumnName("resolved_at");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");

        b.HasOne(x => x.Reporter)
            .WithMany()
            .HasForeignKey(x => x.ReporterUserId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(x => x.ResolvedBy)
            .WithMany()
            .HasForeignKey(x => x.ResolvedById)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(x => new { x.Status, x.CreatedAt }).HasDatabaseName("ix_reports_status_created");
        b.HasIndex(x => new { x.TargetType, x.TargetId }).HasDatabaseName("ix_reports_target");
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/SavedPostConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class SavedPostConfiguration : IEntityTypeConfiguration<SavedPost>
{
    public void Configure(EntityTypeBuilder<SavedPost> b)
    {
        b.ToTable("saved_posts");
        b.HasKey(x => new { x.UserId, x.PostId });

        b.Property(x => x.UserId).HasColumnName("user_id");
        b.Property(x => x.PostId).HasColumnName("post_id");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");

        b.HasOne(x => x.User)
            .WithMany()
            .HasForeignKey(x => x.UserId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasOne(x => x.Post)
            .WithMany(p => p.Saves)
            .HasForeignKey(x => x.PostId)
            .OnDelete(DeleteBehavior.Cascade);

        b.HasIndex(x => new { x.UserId, x.CreatedAt }).HasDatabaseName("ix_saved_posts_user_created");
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Configurations/UserConfiguration.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Configurations;

public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> b)
    {
        b.ToTable("users");
        b.HasKey(x => x.Id);
        b.Property(x => x.Id).HasColumnName("id").HasDefaultValueSql("gen_random_uuid()");

        b.Property(x => x.Username).HasColumnName("username").HasMaxLength(32).IsRequired();
        b.Property(x => x.Email).HasColumnName("email").HasMaxLength(255).IsRequired();
        b.Property(x => x.PasswordHash).HasColumnName("password_hash").HasMaxLength(255).IsRequired();
        b.Property(x => x.DisplayName).HasColumnName("display_name").HasMaxLength(64).IsRequired();
        b.Property(x => x.AvatarUrl).HasColumnName("avatar_url").HasMaxLength(500);
        b.Property(x => x.Bio).HasColumnName("bio").HasMaxLength(280);
        b.Property(x =>
            x.Role).HasColumnName("role").HasConversion<short>().HasDefaultValue(Therapp.Domain.Enums.UserRole.User);
        b.Property(x => x.IsBanned).HasColumnName("is_banned").HasDefaultValue(false);
        b.Property(x => x.AcceptedDisclaimerAt).HasColumnName("accepted_disclaimer_at");
        b.Property(x => x.CreatedAt).HasColumnName("created_at");
        b.Property(x => x.UpdatedAt).HasColumnName("updated_at");
        b.Property(x => x.DeletedAt).HasColumnName("deleted_at");
    }
}

```

```

        b.HasIndex(x => x.Username).IsUnique().HasDatabaseName("ux_users_username");
        b.HasIndex(x => x.Email).IsUnique().HasDatabaseName("ux_users_email");

        b.HasQueryFilter(x => x.DeletedAt == null);
    }
}

```

backend/src/Therapp.Infrastructure/Persistence/Seed/CategorySeeder.cs

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
using Microsoft.EntityFrameworkCore;
using Therapp.Domain.Entities;

namespace Therapp.Infrastructure.Persistence.Seed;

public static class CategorySeeder
{
    private record Seed(string Slug, string Name, string Description, string Icon, string Color, int Order);

    private static readonly Seed[] Defaults =
    {
        new("anksiyete", "Anksiyete", "Endişe ve kaygıyla ilgili deneyimler", "wind",
"#42DCA3", 1),
        new("depresyon", "Depresyon", "Çökkün ruh hali ve umut paylaşımları", "cloud-rain",
"#5DADE2", 2),
        new("panik-atak", "Panik Atak", "Panik atak deneyimleri ve baş etme yöntemleri", "zap",
"#F5A524", 3),
        new("yalnizlik", "Yalnızlık", "Yalnız hissetme ve bağ kurma deneyimleri", "user",
"#A78BFA", 4),
        new("stres", "Stres", "Günlük stresle baş etme paylaşımları", "activity",
"#FF6B81", 5),
        new("iliskiler", "İlişkiler", "Aile, arkadaş ve ilişki paylaşımları", "heart",
"#EC4899", 6),
        new("oz-bakim", "Öz Bakım", "Kendine iyi bakma rutinleri ve önerileri", "leaf",
"#34D399", 7),
        new("uyku", "Uyku", "Uyku sorunları ve çözüm paylaşımları", "moon",
"#818CF8", 8),
    };

    public static async Task SeedAsync(AppDbContext db, CancellationToken ct = default)
    {
        var existing = await db.Categories.Select(c => c.Slug).ToListAsync(ct);
        var missing = Defaults.Where(d => !existing.Contains(d.Slug)).ToList();
        if (missing.Count == 0) return;

        foreach (var s in missing)
        {
            db.Categories.Add(new Category
            {
                Slug = s.Slug,
                Name = s.Name,
                Description = s.Description,
                Icon = s.Icon,
                Color = s.Color,
                OrderIndex = s.Order,
                IsActive = true
            });
        }
        await db.SaveChangesAsync(ct);
    }
}

```

backend/src/Therapp.Infrastructure/Therapp.Infrastructure.csproj

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <RootNamespace>Therapp.Infrastructure</RootNamespace>
    <AssemblyName>Therapp.Infrastructure</AssemblyName>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\Therapp.Application\Therapp.Application.csproj" />
  </ItemGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.10" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.10">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="8.0.10" />
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="8.0.10" />
    <PackageReference Include="BCrypt.Net-Next" Version="4.0.3" />
  </ItemGroup>
</Project>

```

Mobil Kaynak Kodları (React Native / TypeScript)

mobile/app.json

```
{
  "expo": {
    "name": "Therapp",
    "slug": "therapp",
    "scheme": "therapp",
    "version": "1.0.0",
    "orientation": "portrait",
    "userInterfaceStyle": "light",
    "newArchEnabled": true,
    "splash": {
      "resizeMode": "contain",
      "backgroundColor": "#F1FFF8"
    },
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "com.therapp.mobile"
    },
    "android": {
      "package": "com.therapp.mobile",
      "adaptiveIcon": {
        "backgroundColor": "#F1FFF8"
      }
    },
    "web": {
      "bundler": "metro"
    },
    "plugins": [
      "expo-router",
      "expo-secure-store",
      "expo-font",
      "expo-asset"
    ],
    "experiments": {
      "typedRoutes": true
    },
    "extra": {
      "apiUrl": "http://192.168.68.214:5080/api"
    }
  }
}
```

mobile/app/(auth)/_layout.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Stack } from 'expo-router';
import { colors } from '@theme';

export default function AuthLayout() {
  return (
    <Stack
      screenOptions={{
        headerShown: false,
        contentStyle: { backgroundColor: colors.background },
        animation: 'slide_from_right',
      }}
    />
  );
}
```

mobile/app/(auth)/login.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { Link, useRouter } from 'expo-router';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { Sparkles } from 'lucide-react-native';
import { ScreenContainer } from '@components/layout/ScreenContainer';
import { Button } from '@components/ui/Button';
import { Input } from '@components/ui/Input';
import { colors, radius, spacing, typography } from '@theme';
import { authApi } from '@api/auth.api';
import { useAuthStore } from '@store/authStore';
import { toApiError } from '@api/client';

const schema = z.object({
```

```

    email: z.string().email('Geçerli bir e-posta girin.'),
    password: z.string().min(8, 'Şifre en az 8 karakter olmalı.'),
  });

  type FormData = z.infer<typeof schema>;

  export default function LoginScreen() {
    const router = useRouter();
    const setSession = useAuthStore((s) => s.setSession);
    const [submitting, setSubmitting] = useState(false);
    const [topError, setTopError] = useState<string | null>(null);

    const {
      control,
      handleSubmit,
      formState: { errors },
    } = useForm<FormData>({
      resolver: zodResolver(schema),
      defaultValues: { email: '', password: '' },
      mode: 'onTouched',
    });

    const onSubmit = async (values: FormData) => {
      setSubmitting(true);
      setTopError(null);
      try {
        const data = await authApi.login(values);
        await setSession(data);
        router.replace('/(tabs)/home');
      } catch (e) {
        const err = toApiError(e);
        setTopError(err.message);
      } finally {
        setSubmitting(false);
      }
    };

    return (
      <ScreenContainer scroll>
        <View style={styles.header}>
          <View style={styles.logoWrap}>
            <Sparkles size={28} color={colors.primaryDark} />
          </View>
          <Text style={styles.title}>Tekrar hoş geldin</Text>
          <Text style={styles.subtitle}>Devam etmek için giriş yap.</Text>
        </View>

        <View style={styles.form}>
          <Controller
            control={control}
            name="email"
            render={({ field: { value, onChange, onBlur } }) => (
              <Input
                label="E-posta"
                placeholder="ornek@mail.com"
                autoCapitalize="none"
                keyboardType="email-address"
                autoComplete="email"
                contentType="emailAddress"
                value={value}
                onChangeText={onChange}
                onBlur={onBlur}
                error={errors.email?.message}
              />
            )}
          />

          <Controller
            control={control}
            name="password"
            render={({ field: { value, onChange, onBlur } }) => (
              <Input
                label="Şifre"
                placeholder="....."
                secureTextEntry
                autoComplete="current-password"
                contentType="password"
                value={value}
                onChangeText={onChange}
                onBlur={onBlur}
                error={errors.password?.message}
              />
            )}
          />

          {topError && (
            <View style={styles.topError}>
              <Text style={styles.topErrorText}>{topError}</Text>
            </View>
          )}
        </View>
      </ScreenContainer>
    );
  }

```

```

    </View>
  )}

  <Button
    label="Giriş yap"
    onPress={handleSubmit(onSubmit)}
    loading={submitting}
    fullWidth
    size="lg"
  />

  <View style={styles.footer}>
    <Text style={styles.footerText}>Hesabın yok mu?</Text>
    <Link href="/(auth)/register" asChild>
      <Pressable hitSlop={8}>
        <Text style={styles.footerLink}>Kayıt ol</Text>
      </Pressable>
    </Link>
  </View>
</View>
</ScreenContainer>
);
}

const styles = StyleSheet.create({
  header: {
    paddingTop: spacing.xxl,
    paddingBottom: spacing.xl,
    alignItems: 'flex-start',
  },
  logoWrap: {
    width: 56,
    height: 56,
    borderRadius: radius.lg,
    backgroundColor: colors.primarySoft,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: spacing.lg,
  },
  title: {
    ...typography.title,
    color: colors.text,
    marginBottom: spacing.xs,
  },
  subtitle: {
    ...typography.body,
    color: colors.textMuted,
  },
  form: {
    gap: spacing.lg,
  },
  topError: {
    backgroundColor: colors.dangerSoft,
    borderRadius: radius.md,
    paddingVertical: spacing.md,
    paddingHorizontal: spacing.lg,
  },
  topErrorText: {
    ...typography.caption,
    color: colors.danger,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'center',
    alignItems: 'center',
    gap: spacing.xs,
    paddingVertical: spacing.lg,
  },
  footerText: {
    ...typography.body,
    color: colors.textMuted,
  },
  footerLink: {
    ...typography.bodyMedium,
    color: colors.primaryDark,
  },
});

```

mobile/app/(auth)/register.tsx

```

// Therapp | Hazırlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { Link, useRouter } from 'expo-router';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

```

```

import { z } from 'zod';
import { Leaf } from 'lucide-react-native';
import { ScreenContainer } from '@components/layout/ScreenContainer';
import { Button } from '@components/ui/Button';
import { Input } from '@components/ui/Input';
import { colors, radius, spacing, typography } from '@theme';
import { authApi } from '@api/auth.api';
import { useAuthStore } from '@store/authStore';
import { toApiError } from '@api/client';
import { DISCLAIMER_SHORT } from '@constants/disclaimer';

const schema = z.object({
  displayName: z
    .string()
    .min(2, 'Görünen ad en az 2 karakter olmalı.')
    .max(64, 'Görünen ad çok uzun.'),
  username: z
    .string()
    .min(3, 'Kullanıcı adı en az 3 karakter olmalı.')
    .max(32, 'Kullanıcı adı çok uzun.')
    .regex(/^[a-zA-Z0-9_]+$/, 'Yalnızca harf, rakam ve _ kullanılabilir.'),
  email: z.string().email('Geçerli bir e-posta girin.'),
  password: z.string().min(8, 'Şifre en az 8 karakter olmalı.'),
});

type FormData = z.infer<typeof schema>;

export default function RegisterScreen() {
  const router = useRouter();
  const setSession = useAuthStore((s) => s.setSession);
  const [submitting, setSubmitting] = useState(false);
  const [topError, setTopError] = useState<string | null>(null);

  const {
    control,
    handleSubmit,
    formState: { errors },
  } = useForm<FormData>({
    resolver: zodResolver(schema),
    defaultValues: { displayName: '', username: '', email: '', password: '' },
    mode: 'onTouched',
  });

  const onSubmit = async (values: FormData) => {
    setSubmitting(true);
    setTopError(null);
    try {
      const data = await authApi.register(values);
      await setSession(data);
      router.replace('/(tabs)/home');
    } catch (e) {
      const err = toApiError(e);
      setTopError(err.message);
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <ScreenContainer scroll>
      <View style={styles.header}>
        <View style={styles.logoWrap}>
          <Leaf size={28} color={colors.primaryDark} />
        </View>
        <Text style={styles.title}>Therapp'e katıl</Text>
        <Text style={styles.subtitle}>
          Birkaç adımda hesabınızı oluşturun, deneyim paylaşmaya başla.
        </Text>
      </View>

      <View style={styles.form}>
        <Controller
          control={control}
          name="displayName"
          render={({ field: { value, onChange, onBlur } }) => (
            <Input
              label="Görünen ad"
              placeholder="Örn. Ayşe"
              value={value}
              onChangeText={onChange}
              onBlur={onBlur}
              error={errors.displayName?.message}
            />
          )}
        />

        <Controller
          control={control}

```

```

name="username"
render={({ field: { value, onChange, onBlur } }) => (
  <Input
    label="Kullanıcı adı"
    placeholder="ornek_kullanici"
    autoCapitalize="none"
    autoCorrect={false}
    value={value}
    onChangeText={onChange}
    onBlur={onBlur}
    error={errors.username?.message}
    helper="Sadece harf, rakam ve _ kullanabilirsin."
  />
)}
/>

<Controller
  control={control}
  name="email"
  render={({ field: { value, onChange, onBlur } }) => (
    <Input
      label="E-posta"
      placeholder="ornek@mail.com"
      autoCapitalize="none"
      keyboardType="email-address"
      autoComplete="email"
      textContentType="emailAddress"
      value={value}
      onChangeText={onChange}
      onBlur={onBlur}
      error={errors.email?.message}
    />
  )}
/>

<Controller
  control={control}
  name="password"
  render={({ field: { value, onChange, onBlur } }) => (
    <Input
      label="Şifre"
      placeholder="En az 8 karakter"
      secureTextEntry
      autoComplete="new-password"
      textContentType="newPassword"
      value={value}
      onChangeText={onChange}
      onBlur={onBlur}
      error={errors.password?.message}
    />
  )}
/>

{topError && (
  <View style={styles.topError}>
    <Text style={styles.topErrorText}>{topError}</Text>
  </View>
)}

<Text style={styles.disclaimer}>{DISCLAIMER_SHORT}</Text>

<Button
  label="Hesap oluştur"
  onPress={handleSubmit(onSubmit)}
  loading={submitting}
  fullWidth
  size="lg"
/>

<View style={styles.footer}>
  <Text style={styles.footerText}>Zaten hesabın var mı?</Text>
  <Link href="/(auth)/login" asChild>
    <Pressable hitSlop={8}>
      <Text style={styles.footerLink}>Giriş yap</Text>
    </Pressable>
  </Link>
</View>
</View>
</ScreenContainer>
);
}

const styles = StyleSheet.create({
  header: {
    paddingTop: spacing.xxl,
    paddingBottom: spacing.xl,
    alignItems: 'flex-start',
  },
});

```

```

    logoWrap: {
      width: 56,
      height: 56,
      borderRadius: radius.lg,
      backgroundColor: colors.primarySoft,
      alignItems: 'center',
      justifyContent: 'center',
      marginBottom: spacing.lg,
    },
    title: {
      ...typography.title,
      color: colors.text,
      marginBottom: spacing.xs,
    },
    subtitle: {
      ...typography.body,
      color: colors.textMuted,
    },
    form: {
      gap: spacing.lg,
    },
    disclaimer: {
      ...typography.caption,
      color: colors.textMuted,
      textAlign: 'center',
      paddingHorizontal: spacing.md,
    },
    topError: {
      backgroundColor: colors.dangerSoft,
      borderRadius: radius.md,
      paddingVertical: spacing.md,
      paddingHorizontal: spacing.lg,
    },
    topErrorText: {
      ...typography.caption,
      color: colors.danger,
    },
    footer: {
      flexDirection: 'row',
      justifyContent: 'center',
      alignItems: 'center',
      gap: spacing.xs,
      paddingVertical: spacing.lg,
    },
    footerText: {
      ...typography.body,
      color: colors.textMuted,
    },
    footerLink: {
      ...typography.bodyMedium,
      color: colors.primaryDark,
    },
  },
});

```

mobile/app/(tabs)/_layout.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Tabs, useRouter } from 'expo-router';
import { Bell, Home, Layers, PlusCircle, User } from 'lucide-react-native';
import { colors } from '@theme';
import { useUnreadCount } from '@hooks/useNotifications';

export default function TabsLayout() {
  const router = useRouter();
  const { data: unread = 0 } = useUnreadCount();

  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: colors.primaryDark,
        tabBarInactiveTintColor: colors.textMuted,
        tabBarStyle: {
          backgroundColor: colors.background,
          borderTopColor: colors.border,
          height: 60,
          paddingTop: 6,
          paddingBottom: 8,
        },
        tabBarLabelStyle: { fontSize: 11, fontWeight: '600' },
      }}
    >
      <Tabs.Screen
        name="home"
        options={{ title: 'Akış', tabBarIcon: ({ color, size }) => <Home size={size} color={color} /> }}
      />
    </Tabs>
  );
}

```

```

<Tabs.Screen
  name="categories"
  options={{ title: 'Kategoriler', tabBarIcon: ({ color, size }) => <Layers size={size} color={color}
/> }}
/>
<Tabs.Screen
  name="create"
  options={{
    title: 'Paylas',
    tabBarIcon: ({ color, size }) => (
      <PlusCircle size={size + 2} color={color} strokeWidth={2.2} />
    ),
  }}
  listeners={{
    tabPress: (e) => {
      e.preventDefault();
      router.push('/post/new');
    },
  }}
/>
<Tabs.Screen
  name="notifications"
  options={{
    title: 'Bildirim',
    tabBarIcon: ({ color, size }) => <Bell size={size} color={color} />,
    tabBarBadge: unread > 0 ? (unread > 99 ? '99+' : unread) : undefined,
    tabBarBadgeStyle: {
      backgroundColor: colors.danger,
      color: colors.textOnPrimary,
      fontSize: 10,
      minWidth: 18,
      height: 18,
      lineHeight: 14,
    },
  }}
/>
<Tabs.Screen
  name="profile"
  options={{ title: 'Profil', tabBarIcon: ({ color, size }) => <User size={size} color={color} /> }}
/>
</Tabs>
);
}

```

mobile/app/(tabs)/categories.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { FlatList, RefreshControl, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useRouter } from 'expo-router';
import { CategoryCard } from '@components/category/CategoryCard';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingState } from '@components/ui/LoadingState';
import { colors, spacing, typography } from '@theme';
import { useCategories } from '@hooks/useCategories';
import type { Category } from '@types/category.types';

export default function CategoriesScreen() {
  const router = useRouter();
  const { data, isLoading, isRefetching, refetch } = useCategories();

  return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
      <StatusBar style="dark" />
      <View style={styles.header}>
        <Text style={styles.title}>Kategoriler</Text>
        <Text style={styles.subtitle}>İlgilendiğin konuya göre keşfet</Text>
      </View>

      <FlatList
        data={data ?? []}
        keyExtractor={({ c: Category }) => c.id}
        numColumns={2}
        columnWrapperStyle={{ gap: spacing.md }}
        ItemSeparatorComponent={() => <View style={{ height: spacing.md }} />}
        contentContainerStyle={styles.list}
        renderItem={({ item }) => (
          <CategoryCard category={item} onPress={() => router.push(`/category/${item.slug}`)} />
        )}
        ListEmptyComponent={
          isLoading ? (
            <LoadingState />
          ) : (
            <EmptyState title="Kategori bulunamadı" />
          )
        }
      />
    </SafeAreaView>
  );
}

```

```

    }
    refreshControl={
      <RefreshControl refreshing={isRefetching} onRefresh={refetch} tintColor={colors.primary} />
    }
    showsVerticalScrollIndicator={false}
  />
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  header: {
    paddingHorizontal: spacing.lg,
    paddingTop: spacing.md,
    paddingBottom: spacing.lg,
  },
  title: {
    ...typography.display,
    color: colors.text,
    marginBottom: 4,
  },
  subtitle: {
    ...typography.body,
    color: colors.textMuted,
  },
  list: {
    paddingHorizontal: spacing.lg,
    paddingBottom: spacing.xxl,
    flexGrow: 1,
  },
});
});

```

mobile/app/(tabs)/create.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { ScreenContainer } from '@components/layout/ScreenContainer';
import { EmptyState } from '@components/ui/EmptyState';

export default function CreateScreen() {
  return (
    <ScreenContainer>
      <EmptyState title="Paylaş" description="Bir sonraki adımda gelecek." />
    </ScreenContainer>
  );
}

```

mobile/app/(tabs)/home.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { FlatList, Pressable, RefreshControl, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { Sparkles } from 'lucide-react-native';
import { PostCard } from '@components/post/PostCard';
import { DisclaimerBanner } from '@components/common/DisclaimerBanner';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingState } from '@components/ui/LoadingState';
import { colors, radius, spacing, typography } from '@theme';
import { useFeed } from '@hooks/usePosts';
import type { FeedSort, PostListItem } from '@types/post.types';

export default function HomeScreen() {
  const [sort, setSort] = useState<FeedSort>('new');
  const { data, isLoading, isRefetching, refetch, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useFeed(sort);

  const items = data?.pages.flatMap((p) => p.items) ?? [];

  return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
      <StatusBar style="dark" />
      <View style={styles.header}>
        <View style={styles.titleRow}>
          <Text style={styles.title}>Therapp</Text>
          <Sparkles size={22} color={colors.primaryDark} />
        </View>
        <SortToggle value={sort} onChange={setSort} />
      </View>

      <FlatList
        data={items}
        keyExtractor={(p: PostListItem) => p.id}
        renderItem={({ item }) => <PostCard post={item} />}
      />
    </SafeAreaView>
  );
}

```

```

ItemSeparatorComponent={() => <View style={{ height: spacing.md }} />}
contentContainerStyle={styles.list}
ListHeaderComponent={
  <View style={styles.disclaimer}>
    <DisclaimerBanner />
  </View>
}
ListEmptyComponent={
  isLoading ? (
    <LoadingState />
  ) : (
    <EmptyState
      title="Henüz gönderi yok"
      description="İlk paylaşımı sen yap, alttaki + ikonuna dokun."
    />
  )
}
refreshControl={
  <RefreshControl
    refreshing={isRefetching && !isFetchingNextPage}
    onRefresh={refetch}
    tintColor={colors.primary}
  />
}
onEndReachedThreshold={0.5}
onEndReached={() => {
  if (hasNextPage && !isFetchingNextPage) fetchNextPage();
}}
ListFooterComponent={
  isFetchingNextPage ? (
    <View style={{ paddingVertical: spacing.lg }}>
      <LoadingState size="small" />
    </View>
  ) : null
}
showsVerticalScrollIndicator={false}
/>
</SafeAreaView>
);
}

function SortToggle({ value, onChange }: { value: FeedSort; onChange: (v: FeedSort) => void }) {
  return (
    <View style={styles.toggleWrap}>
      {[ 'new', 'popular' ] as const}.map((opt) => {
        const active = value === opt;
        return (
          <Pressable
            key={opt}
            onPress={() => onChange(opt)}
            style={[styles.toggleBtn, active && styles.toggleBtnActive]}
          >
            <Text style={styles.toggleLabel, active && styles.toggleLabelActive}>
              {opt === 'new' ? 'En Yeni' : 'Popüler'}
            </Text>
          </Pressable>
        );
      })
    </View>
  );
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  header: {
    paddingHorizontal: spacing.lg,
    paddingTop: spacing.md,
    paddingBottom: spacing.md,
    gap: spacing.md,
  },
  titleRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
  },
  title: {
    ...typography.display,
    color: colors.text,
  },
  toggleWrap: {
    flexDirection: 'row',
    backgroundColor: colors.surface,
    borderRadius: radius.pill,
    padding: 4,
    alignSelf: 'flex-start',
  },
  toggleBtn: {
    paddingVertical: 6,

```

```

paddingHorizontal: spacing.lg,
borderRadius: radius.pill,
},
toggleBtnActive: {
  backgroundColor: colors.primary,
},
toggleLabel: {
  ...typography.label,
  color: colors.textMuted,
},
toggleLabelActive: {
  color: colors.textOnPrimary,
},
list: {
  paddingHorizontal: spacing.lg,
  paddingBottom: spacing.xxl,
  flexGrow: 1,
},
disclaimer: {
  marginBottom: spacing.md,
},
});
});

```

mobile/app/(tabs)/notifications.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { FlatList, Pressable, RefreshControl, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useRouter } from 'expo-router';
import { Bell, CheckCheck } from 'lucide-react-native';
import { NotificationItem } from '@components/notification/NotificationItem';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingState } from '@components/ui/LoadingState';
import { colors, spacing, typography } from '@theme';
import {
  useMarkAllRead,
  useMarkRead,
  useNotifications,
  useUnreadCount,
} from '@hooks/useNotifications';
import type { AppNotification } from '@types/notification.types';

export default function NotificationsScreen() {
  const router = useRouter();
  const { data, isLoading, isRefetching, refetch, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useNotifications();
  const { data: unread = 0 } = useUnreadCount();
  const markRead = useMarkRead();
  const markAllRead = useMarkAllRead();

  const items = data?.pages.flatMap((p) => p.items) ?? [];

  const handleTap = (n: AppNotification) => {
    if (!n.isRead) markRead.mutate(n.id);
    if (n.postId) router.push(`/post/${n.postId}`);
  };

  return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
      <StatusBar style="dark" />

      <View style={styles.header}>
        <Text style={styles.title}>Bildirimler</Text>
        {unread > 0 && (
          <Pressable
            onPress={() => markAllRead.mutate()}
            hitSlop={8}
            style={{ pressed } => [styles.markAllBtn, pressed && { opacity: 0.7 }]}
          >
            <CheckCheck size={16} color={colors.primaryDark} />
            <Text style={styles.markAllText}>Tümünü okundu işaretle</Text>
          </Pressable>
        )}
      </View>

      <FlatList
        data={items}
        keyExtractor={(n) => n.id}
        renderItem={({ item }) => (
          <NotificationItem notification={item} onPress={() => handleTap(item)} />
        )}
        ItemSeparatorComponent={() => <View style={styles.separator} />}
        ListEmptyComponent={
          isLoading ? (
            <LoadingState />

```

```

    ) : (
      <EmptyState
        icon={<Bell size={48} color={colors.textMuted} strokeWidth={1.4} />}
        title="Henüz bildirimin yok"
        description="Birisini gönderine etkileşim verdiğinde burada görürsün."
      />
    )
  }
  refreshControl={
    <RefreshControl refreshing={isRefetching && !isFetchingNextPage} onRefresh={refetch}
    tint={colors.primary} />
  }
  onEndReachedThreshold={0.5}
  onEndReached={() => {
    if (hasNextPage && !isFetchingNextPage) fetchNextPage();
  }}
  ListFooterComponent={
    isFetchingNextPage ? (
      <View style={{ paddingVertical: spacing.lg }}>
        <LoadingState size="small" />
      </View>
    ) : null
  }
  contentContainerStyle={{ flexGrow: 1, paddingBottom: spacing.xl }}
  showsVerticalScrollIndicator={false}
/>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  header: {
    paddingHorizontal: spacing.lg,
    paddingTop: spacing.md,
    paddingBottom: spacing.md,
    gap: spacing.sm,
  },
  title: {
    ...typography.display,
    color: colors.text,
  },
  markAllBtn: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 4,
    alignSelf: 'flex-start',
    paddingVertical: 4,
  },
  markAllText: {
    ...typography.caption,
    color: colors.primaryDark,
    fontWeight: '600',
  },
  separator: {
    height: 1,
    backgroundColor: colors.divider,
    marginLeft: spacing.lg + 40 + spacing.md,
  },
});

```

mobile/app/(tabs)/profile.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { FlatList, Pressable, RefreshControl, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useRouter } from 'expo-router';
import { Settings as SettingsIcon } from 'lucide-react-native';
import { Avatar } from '@components/ui/Avatar';
import { PostCard } from '@components/post/PostCard';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingState } from '@components/ui/LoadingState';
import { colors, radius, spacing, typography } from '@theme';
import { useMe } from '@hooks/useUser';
import { useMyPosts, useMySaved } from '@hooks/usePosts';
import type { PostListItem } from '@types/post.types';

type Tab = 'posts' | 'saved';

export default function ProfileScreen() {
  const router = useRouter();
  const [tab, setTab] = useState<Tab>('posts');
  const { data: me, isLoading: meLoading } = useMe();

```

```

const postsQuery = useMyPosts();
const savedQuery = useMySaved();
const active = tab === 'posts' ? postsQuery : savedQuery;

const items = active.data?.pages.flatMap((p) => p.items) ?? [];

return (
  <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
    <StatusBar style="dark" />

    <FlatList
      data={items}
      keyExtractor={({ p: PostListItem }) => p.id}
      renderItem={({ item }) => <PostCard post={item} />}
      ItemSeparatorComponent={() => <View style={{ height: spacing.md }} />}
      contentContainerStyle={styles.list}
      ListHeaderComponent={
        <View style={styles.headerBlock}>
          <View style={styles.topRow}>
            <Text style={styles.title}>Profil</Text>
            <Pressable
              onPress={() => router.push('/settings')}
              hitSlop={8}
              style={styles.iconBtn}
            >
              <SettingsIcon size={22} color={colors.text} />
            </Pressable>
          </View>

          {meLoading || !me ? (
            <LoadingState size="small" />
          ) : (
            <View style={styles.profileCard}>
              <Avatar uri={me.avatarUrl} name={me.displayName} size={72} />
              <Text style={styles.displayName}>{me.displayName}</Text>
              <Text style={styles.username}>@{me.username}</Text>
              {me.bio ? <Text style={styles.bio}>{me.bio}</Text> : null}

              <View style={styles.statsRow}>
                <Stat value={me.postCount} label="Gönderi" />
                <View style={styles.statDivider} />
                <Stat value={me.savedCount} label="Kaydedilen" />
              </View>
            </View>
          )}

          <View style={styles.tabsRow}>
            <TabBtn label="Paylaşımlarım" active={tab === 'posts'} onPress={() => setTab('posts')} />
            <TabBtn label="Kaydettiklerim" active={tab === 'saved'} onPress={() => setTab('saved')} />
          </View>
        </View>
      }
      ListEmptyComponent={
        active.isLoading ? (
          <LoadingState />
        ) : (
          <EmptyState
            title={tab === 'posts' ? 'Henüz paylaşımın yok' : 'Henüz kaydettiğin gönderi yok'}
            description={
              tab === 'posts'
                ? '+ ikonuna dokunup ilk paylaşımını yap.'
                : 'Beğendiğin gönderileri kaydederek burada saklayabilirsin.'
            }
          />
        )
      }
      refreshControl={
        <RefreshControl
          refreshing={active.isRefetching && !active.isFetchingNextPage}
          onRefresh={active.refetch}
          tint={colors.primary}
        />
      }
      onEndReachedThreshold={0.5}
      onEndReached={() => {
        if (active.hasNextPage && !active.isFetchingNextPage) active.fetchNextPage();
      }}
      ListFooterComponent={
        active.isFetchingNextPage ? (
          <View style={{ paddingVertical: spacing.lg }}>
            <LoadingState size="small" />
          </View>
        ) : null
      }
      showsVerticalScrollIndicator={false}
    />
  </SafeAreaView>
);

```

```

}

function Stat({ value, label }: { value: number; label: string }) {
  return (
    <View style={styles.stat}>
      <Text style={styles.statValue}><value></Text>
      <Text style={styles.statLabel}><label></Text>
    </View>
  );
}

function TabBtn({ label, active, onPress }: { label: string; active: boolean; onPress: () => void }) {
  return (
    <Pressable onPress={onPress} style={[styles.tabBtn, active && styles.tabBtnActive]}>
      <Text style={[styles.tabBtnLabel, active && styles.tabBtnLabelActive]}><label></Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  list: {
    paddingHorizontal: spacing.lg,
    paddingBottom: spacing.xxl,
    flexGrow: 1,
  },
  headerBlock: {
    paddingTop: spacing.md,
    paddingBottom: spacing.lg,
    gap: spacing.lg,
  },
  topRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
  },
  title: {
    ...typography.display,
    color: colors.text,
  },
  iconBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    backgroundColor: colors.surface,
    alignItems: 'center',
    justifyContent: 'center',
  },
  profileCard: {
    backgroundColor: colors.surface,
    borderRadius: radius.lg,
    padding: spacing.xl,
    alignItems: 'center',
    gap: spacing.xs,
  },
  displayName: {
    ...typography.subtitle,
    color: colors.text,
    marginTop: spacing.md,
  },
  username: {
    ...typography.body,
    color: colors.textMuted,
  },
  bio: {
    ...typography.body,
    color: colors.textSecondary,
    textAlign: 'center',
    marginTop: spacing.sm,
  },
  statsRow: {
    flexDirection: 'row',
    alignItems: 'center',
    marginTop: spacing.md,
    backgroundColor: colors.background,
    borderRadius: radius.md,
    paddingVertical: spacing.md,
    paddingHorizontal: spacing.xl,
    gap: spacing.xl,
  },
  stat: {
    alignItems: 'center',
  },
  statValue: {
    ...typography.subtitle,
    color: colors.text,
  },
  statLabel: {

```

```

    ...typography.caption,
    color: colors.textMuted,
  },
  statDivider: {
    width: 1,
    height: 24,
    backgroundColor: colors.divider,
  },
  tabsRow: {
    flexDirection: 'row',
    backgroundColor: colors.surface,
    borderRadius: radius.pill,
    padding: 4,
  },
  tabBtn: {
    flex: 1,
    paddingVertical: spacing.sm,
    borderRadius: radius.pill,
    alignItems: 'center',
  },
  tabBtnActive: {
    backgroundColor: colors.primary,
  },
  tabBtnLabel: {
    ...typography.label,
    color: colors.textMuted,
  },
  tabBtnLabelActive: {
    color: colors.textOnPrimary,
  },
},
));

```

mobile/app/_layout.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useEffect, useState } from 'react';
import { Stack, useRouter, useSegments } from 'expo-router';
import { GestureHandlerRootView } from 'react-native-gesture-handler';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import * as SplashScreen from 'expo-splash-screen';
import { useAuthStore } from '@store/authStore';
import { useUiStore } from '@store/uiStore';
import { colors } from '@theme';

SplashScreen.preventAutoHideAsync().catch(() => {
  /* ignore */
});

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      staleTime: 30_000,
      refetchOnWindowFocus: false,
    },
  },
});

export default function RootLayout() {
  const hydrateAuth = useAuthStore((s) => s.hydrate);
  const hydrateUi = useUiStore((s) => s.hydrate);
  const authHydrated = useAuthStore((s) => s.hydrated);
  const uiHydrated = useUiStore((s) => s.hydrated);

  const [bootstrapped, setBootstrapped] = useState(false);

  useEffect(() => {
    (async () => {
      await Promise.all([hydrateAuth(), hydrateUi()]);
      setBootstrapped(true);
      await SplashScreen.hideAsync().catch(() => {});
    })();
  }, [hydrateAuth, hydrateUi]);

  if (!bootstrapped || !authHydrated || !uiHydrated) {
    return null;
  }

  return (
    <GestureHandlerRootView style={{ flex: 1, backgroundColor: colors.background }}>
      <SafeAreaProvider>
        <QueryClientProvider client={queryClient}>
          <AuthGate />
        </QueryClientProvider>
      </SafeAreaProvider>
    </GestureHandlerRootView>
  );
}

```

```

    </GestureHandlerRootView>
  );
}

function AuthGate() {
  const isAuthenticated = useAuthStore((s) => s.isAuthenticated);
  const hasOnboarded = useUiStore((s) => s.hasOnboarded);
  const segments = useSegments();
  const router = useRouter();

  useEffect(() => {
    const inAuthGroup = segments[0] === 'auth';
    const onOnboarding = segments[0] === 'onboarding';
    const atRoot = (segments as string[]).length === 0; // index (kök) ekranı

    if (!hasOnboarded && !onOnboarding) {
      router.replace('/onboarding');
      return;
    }

    if (hasOnboarded && !isAuthenticated && !inAuthGroup && !onOnboarding) {
      router.replace('/(auth)/login');
      return;
    }

    if (isAuthenticated && (inAuthGroup || onOnboarding || atRoot)) {
      router.replace('/(tabs)/home');
    }
  }, [isAuthenticated, hasOnboarded, segments, router]);

  return (
    <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: colors.background } }}>
      <Stack.Screen name="index" />
      <Stack.Screen name="onboarding" />
      <Stack.Screen name="(auth)" />
      <Stack.Screen name="(tabs)" />
      <Stack.Screen name="post/[id]" />
      <Stack.Screen name="post/new" options={{ presentation: 'modal' }} />
      <Stack.Screen name="category/[slug]" />
      <Stack.Screen name="report" options={{ presentation: 'modal' }} />
      <Stack.Screen name="settings" />
    </Stack>
  );
}
}

```

mobile/app/category/[slug].tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { FlatList, Pressable, RefreshControl, StyleSheet, Text, View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { ArrowLeft } from 'lucide-react-native';
import { PostCard } from '@components/post/PostCard';
import { CategoryIcon } from '@components/ui/CategoryIcon';
import { EmptyState } from '@components/ui/EmptyState';
import { LoadingState } from '@components/ui/LoadingState';
import { colors, radius, spacing, typography } from '@theme';
import { useCategory } from '@hooks/useCategories';
import { useCategoryFeed } from '@hooks/usePosts';
import type { FeedSort, PostListItem } from '@types/post.types';

export default function CategoryDetailScreen() {
  const { slug } = useLocalSearchParams<{ slug: string }>();
  const router = useRouter();
  const [sort, setSort] = useState<FeedSort>('new');

  const { data: category } = useCategory(slug);
  const { data, isLoading, isRefetching, refetch, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useCategoryFeed(slug, sort);

  const items = data?.pages.flatMap((p) => p.items) ?? [];

  return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
      <StatusBar style="dark" />
      <View style={styles.header}>
        <Pressable onPress={() => router.back()} hitSlop={8} style={styles.backBtn}>
          <ArrowLeft size={22} color={colors.text} />
        </Pressable>
      </View>

      <FlatList
        data={items}
        keyExtractor={({ p: PostListItem }) => p.id}
      />
    </SafeAreaView>
  );
}

```

```

renderItem={({ item }) => <PostCard post={item} />}
ItemSeparatorComponent={() => <View style={{ height: spacing.md }} />}
contentContainerStyle={styles.list}
ListHeaderComponent={
  <View style={styles.hero}>
    {category && (
      <View style={[[styles.iconWrap, { backgroundColor: hexToSoft(category.color) }]]}>
        <CategoryIcon name={category.icon} color={category.color} size={32} />
      </View>
    )}
    <Text style={styles.title}>{category?.name ?? 'Kategori'}</Text>
    {category?.description && (
      <Text style={styles.description}>{category.description}</Text>
    )}
    <View style={styles.toggleWrap}>
      {[ 'new', 'popular' ] as const}.map((opt) => {
        const active = sort === opt;
        return (
          <Pressable
            key={opt}
            onPress={() => setSort(opt)}
            style={[[styles.toggleBtn, active && styles.toggleBtnActive]]}
          >
            <Text style={[[styles.toggleLabel, active && styles.toggleLabelActive]]}>
              {opt === 'new' ? 'En Yeni' : 'Popüler'}
            </Text>
          </Pressable>
        );
      })
    </View>
  </View>
}
ListEmptyComponent={
  isLoading ? (
    <LoadingState />
  ) : (
    <EmptyState
      title="Bu kategoride henüz gönderi yok"
      description="İlk paylaşımı sen yap."
    />
  )
}
refreshControl={
  <RefreshControl refreshing={isRefetching && !isFetchingNextPage} onRefresh={refetch}
  tintColor={colors.primary} />
}
onEndReachedThreshold={0.5}
onEndReached={() => {
  if (hasNextPage && !isFetchingNextPage) fetchNextPage();
}}
ListFooterComponent={
  isFetchingNextPage ? (
    <View style={{ paddingVertical: spacing.lg }}>
      <LoadingState size="small" />
    </View>
  ) : null
}
showsVerticalScrollIndicator={false}
/>
</SafeAreaView>
);
}

function hexToSoft(hex: string): string {
  const clean = hex.replace('#', '');
  if (clean.length !== 6) return colors.primarySoft;
  const r = parseInt(clean.slice(0, 2), 16);
  const g = parseInt(clean.slice(2, 4), 16);
  const b = parseInt(clean.slice(4, 6), 16);
  const mix = (c: number) => Math.round(c + (255 - c) * 0.8);
  return `rgb(${mix(r)}, ${mix(g)}, ${mix(b)})`;
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  header: {
    paddingHorizontal: spacing.lg,
    paddingTop: spacing.sm,
    paddingBottom: spacing.xs,
  },
  backBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: colors.surface,
  },
});

```

```

list: {
  paddingHorizontal: spacing.lg,
  paddingBottom: spacing.xxl,
  flexGrow: 1,
},
hero: {
  paddingVertical: spacing.lg,
  alignItems: 'flex-start',
},
iconWrap: {
  width: 64,
  height: 64,
  borderRadius: radius.lg,
  alignItems: 'center',
  justifyContent: 'center',
  marginBottom: spacing.md,
},
title: {
  ...typography.display,
  color: colors.text,
  marginBottom: spacing.xs,
},
description: {
  ...typography.body,
  color: colors.textMuted,
  marginBottom: spacing.lg,
},
toggleWrap: {
  flexDirection: 'row',
  backgroundColor: colors.surface,
  borderRadius: radius.pill,
  padding: 4,
},
toggleBtn: {
  paddingVertical: 6,
  paddingHorizontal: spacing.lg,
  borderRadius: radius.pill,
},
toggleBtnActive: {
  backgroundColor: colors.primary,
},
toggleLabel: {
  ...typography.label,
  color: colors.textMuted,
},
toggleLabelActive: {
  color: colors.textOnPrimary,
},
});

```

mobile/app/index.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { StyleSheet, View } from 'react-native';
import { colors } from '@theme';
import { LoadingState } from '@components/ui/LoadingState';

// Bootstrap is handled in _layout.tsx (AuthGate). This screen renders briefly
// while the router resolves the initial redirect.
export default function Index() {
  return (
    <View style={styles.container}>
      <LoadingState />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: colors.background,
  },
});

```

mobile/app/onboarding.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useRouter } from 'expo-router';
import { Heart, MessageCircle, ShieldCheck } from 'lucide-react-native';
import { ScreenContainer } from '@components/layout/ScreenContainer';
import { Button } from '@components/ui/Button';
import { colors, radius, spacing, typography } from '@theme';
import { DISCLAIMER_BODY, DISCLAIMER_TITLE } from '@constants/disclaimer';

```

```

import { useUiStore } from '@store/uiStore';

type Slide = {
  icon: React.ReactNode;
  title: string;
  body: string;
};

const slides: Slide[] = [
  {
    icon: <Heart size={56} color={colors.primary} strokeWidth={1.5} />,
    title: 'Hislerini paylaşmak iyi gelir',
    body: 'Therapp, aynı şeyleri yaşayan insanlarla deneyimlerini paylaşabileceğin güvenli ve sakın bir alandır.',
  },
  {
    icon: <MessageCircle size={56} color={colors.primary} strokeWidth={1.5} />,
    title: 'Yalnız değilsin',
    body: 'Anksiyete, depresyon, yalnızlık, stres – kategorilere göre konuşmalara katıl, başkalarının hikâyelerinden güç al.',
  },
  {
    icon: <ShieldCheck size={56} color={colors.primary} strokeWidth={1.5} />,
    title: 'Sınırları bilelim',
    body: DISCLAIMER_BODY,
  },
];

export default function Onboarding() {
  const [step, setStep] = useState(0);
  const router = useRouter();
  const setOnboarded = useUiStore((s) => s.setOnboarded);

  const isLast = step === slides.length - 1;
  const slide = slides[step];

  const next = async () => {
    if (!isLast) {
      setStep(step + 1);
      return;
    }
    await setOnboarded(true);
    router.replace('/(auth)/login');
  };

  return (
    <ScreenContainer>
      <View style={styles.content}>
        <View style={styles.iconWrap}>{slide.icon}</View>
        {isLast && <Text style={styles.eyebrow}>{DISCLAIMER_TITLE}</Text>}
        <Text style={styles.title}>{slide.title}</Text>
        <Text style={styles.body}>{slide.body}</Text>
      </View>

      <View style={styles.dots}>
        {slides.map((_, i) => (
          <View key={i} style={[[styles.dot, i === step && styles.dotActive]]} />
        ))}
      </View>

      <View style={styles.actions}>
        <Button
          label={isLast ? 'Anladım, devam et' : 'Devam'}
          onPress={next}
          fullWidth
          size="lg"
        />
        {isLast && (
          <Pressable onPress={() => setStep(slides.length - 1)} style={styles.skip}>
            <Text style={styles.skipText}>Atla</Text>
          </Pressable>
        )}
      </View>
    </ScreenContainer>
  );
}

const styles = StyleSheet.create({
  content: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    paddingHorizontal: spacing.lg,
  },
  iconWrap: {
    width: 120,

```

```

    height: 120,
    borderRadius: radius.pill,
    backgroundColor: colors.primarySoft,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: spacing.xxl,
  },
  eyebrow: {
    ...typography.label,
    color: colors.primaryDark,
    marginBottom: spacing.sm,
    textTransform: 'uppercase',
    letterSpacing: 1,
  },
  title: {
    ...typography.title,
    color: colors.text,
    textAlign: 'center',
    marginBottom: spacing.md,
  },
  body: {
    ...typography.body,
    color: colors.textSecondary,
    textAlign: 'center',
    paddingHorizontal: spacing.md,
  },
  dots: {
    flexDirection: 'row',
    justifyContent: 'center',
    gap: spacing.sm,
    marginBottom: spacing.xl,
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: 4,
    backgroundColor: colors.border,
  },
  dotActive: {
    backgroundColor: colors.primary,
    width: 24,
  },
  actions: {
    paddingBottom: spacing.xl,
    gap: spacing.md,
  },
  skip: {
    alignSelf: 'center',
    padding: spacing.sm,
  },
  skipText: {
    ...typography.body,
    color: colors.textMuted,
  },
});

```

mobile/app/post/[id].tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import {
  FlatList,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  StyleSheet,
  Text,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { ArrowLeft, Flag, MoreHorizontal, Trash2 } from 'lucide-react-native';
import { Avatar } from '@/components/ui/Avatar';
import { CategoryChip } from '@/components/category/CategoryChip';
import { PostActions } from '@/components/post/PostActions';
import { CommentItem } from '@/components/comment/CommentItem';
import { CommentComposer } from '@/components/comment/CommentComposer';
import { EmptyState } from '@/components/ui/EmptyState';
import { LoadingState } from '@/components/ui/LoadingState';
import { colors, spacing, typography } from '@/theme';
import { relativeTime } from '@/utils/formatDate';
import { usePost, useLikePost, useSavePost, useDeletePost } from '@/hooks/usePosts';
import {
  useComments,
  useCreateComment,
}

```

```

    useCreateReply,
    useLikeComment,
  } from '@/hooks/useComments';
import type { Comment } from '@/types/post.types';

export default function PostDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const router = useRouter();

  const { data: post, isLoading } = usePost(id);
  const {
    data: commentsData,
    isLoading: commentsLoading,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
  } = useComments(id);

  const comments = commentsData?.pages.flatMap((p) => p.items) ?? [];

  const likeMutation = useLikePost();
  const saveMutation = useSavePost();
  const deletePostMutation = useDeletePost();
  const createCommentMutation = useCreateComment(id as string);
  const createReplyMutation = useCreateReply(id as string);
  const likeCommentMutation = useLikeComment(id as string);

  const [replyTo, setReplyTo] = useState<{ id: string; name: string } | null>(null);

  if (isLoading || !post) {
    return (
      <SafeAreaView style={styles.safe} edges={['top']}>
        <LoadingState />
      </SafeAreaView>
    );
  }

  const onSubmit = async (content: string) => {
    if (replyTo) {
      await createReplyMutation.mutateAsync({ parentCommentId: replyTo.id, content });
      setReplyTo(null);
    } else {
      await createCommentMutation.mutateAsync(content);
    }
  };

  const onDeletePost = async () => {
    await deletePostMutation.mutateAsync(post.id);
    router.back();
  };

  return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
      <StatusBar style="dark" />
      <View style={styles.header}>
        <Pressable onPress={() => router.back()} hitSlop={8} style={styles.iconBtn}>
          <ArrowLeft size={22} color={colors.text} />
        </Pressable>
        <Text style={styles.headerTitle}>Gönderi</Text>
        <Pressable
          onPress={() =>
            post.isMine
              ? onDeletePost()
              : router.push(`/report?type=0&id=${post.id}`)
          }
          hitSlop={8}
          style={styles.iconBtn}
        >
          {post.isMine ? (
            <Trash2 size={20} color={colors.danger} />
          ) : (
            <MoreHorizontal size={22} color={colors.text} />
          )}
        </Pressable>
      </View>

      <KeyboardAvoidingView
        style={styles.flex}
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
        keyboardVerticalOffset={Platform.OS === 'ios' ? 0 : 0}
      >
        <FlatList
          data={comments}
          keyExtractor={(c: Comment) => c.id}
          contentContainerStyle={styles.list}
          ListHeaderComponent={
            <View style={styles.postBlock}>
              <View style={styles.metaRow}>

```

```

        <CategoryChip name={post.category.name} color={post.category.color} />
        <Text style={styles.time}>{relativeTime(post.createdAt)}</Text>
    </View>

    <View style={styles.authorRow}>
        {post.isAnonymous || !post.author ? (
            <>
                <View style={styles.anonymousAvatar}>
                    <Text style={styles.anonymousInitial}>?</Text>
                </View>
                <Text style={styles.authorName}>Anonim</Text>
            </>
        ) : (
            <>
                <Avatar uri={post.author.avatarUrl} name={post.author.displayName} size={36} />
                <View style={{ flex: 1 }}>
                    <Text style={styles.authorName}>{post.author.displayName}</Text>
                    <Text style={styles.authorHandle}>@{post.author.username}</Text>
                </View>
            </>
        )}
    </View>

    <Text style={styles.title}>{post.title}</Text>
    <Text style={styles.content}>{post.content}</Text>

    <PostActions
        likeCount={post.likeCount}
        isLiked={post.isLikedByMe}
        onLike={() => likeMutation.mutate({ id: post.id, liked: post.isLikedByMe })}
        commentCount={post.commentCount}
        saveCount={post.saveCount}
        isSaved={post.isSavedByMe}
        onSave={() => saveMutation.mutate({ id: post.id, saved: post.isSavedByMe })}
    />

    <View style={styles.divider} />
    <Text style={styles.commentsHeader}>
        {post.commentCount > 0 ? `${post.commentCount} yorum` : 'Henüz yorum yok'}
    </Text>
</View>
}
renderItem={({ item }) => (
    <CommentItem
        comment={item}
        onLike={() => likeCommentMutation.mutate({ id: item.id, liked: item.isLikedByMe })}
        onReply={() => setReplyTo({ id: item.id, name: item.author.displayName })}
    />
)}
ItemSeparatorComponent={() => <View style={styles.commentDivider} />}
ListEmptyComponent={
    commentsLoading ? (
        <View style={{ paddingVertical: spacing.xl }}>
            <LoadingState size="small" />
        </View>
    ) : (
        <EmptyState title="İlk yorumu sen yap" />
    )
}
onEndReachedThreshold={0.5}
onEndReached={() => {
    if (hasNextPage && !isFetchingNextPage) fetchNextPage();
}}
ListFooterComponent={
    isFetchingNextPage ? (
        <View style={{ paddingVertical: spacing.lg }}>
            <LoadingState size="small" />
        </View>
    ) : null
}
showsVerticalScrollIndicator={false}
/>

<CommentComposer
    replyingTo={replyTo?.name ?? null}
    onCancelReply={() => setReplyTo(null)}
    submitting={createCommentMutation.isPending || createReplyMutation.isPending}
    onSubmit={onSubmit}
/>
</KeyboardAvoidingView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
    safe: { flex: 1, backgroundColor: colors.background },
    flex: { flex: 1 },
    header: {

```

```

    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: spacing.lg,
    paddingVertical: spacing.sm,
    borderBottomWidth: 1,
    borderBottomColor: colors.divider,
  },
  iconBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerTitle: {
    ...typography.subtitle,
    color: colors.text,
  },
  list: {
    paddingHorizontal: spacing.lg,
    paddingBottom: spacing.xxl,
  },
  postBlock: {
    paddingVertical: spacing.lg,
  },
  metaRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    marginBottom: spacing.md,
  },
  time: {
    ...typography.caption,
    color: colors.textMuted,
  },
  authorRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: spacing.sm,
    marginBottom: spacing.lg,
  },
  anonymousAvatar: {
    width: 36,
    height: 36,
    borderRadius: 18,
    backgroundColor: colors.divider,
    alignItems: 'center',
    justifyContent: 'center',
  },
  anonymousInitial: {
    ...typography.subtitle,
    color: colors.textMuted,
  },
  authorName: {
    ...typography.label,
    color: colors.text,
  },
  authorHandle: {
    ...typography.caption,
    color: colors.textMuted,
  },
  title: {
    ...typography.title,
    color: colors.text,
    marginBottom: spacing.md,
  },
  content: {
    ...typography.body,
    color: colors.textSecondary,
    fontSize: 17,
    lineHeight: 26,
    marginBottom: spacing.md,
  },
  divider: {
    height: 1,
    backgroundColor: colors.divider,
    marginTop: spacing.lg,
    marginBottom: spacing.lg,
  },
  commentsHeader: {
    ...typography.subtitle,
    color: colors.text,
  },
  commentDivider: {
    height: 1,
    backgroundColor: colors.divider,
    marginLeft: spacing.xxl,
  },

```

```
},  
});
```

mobile/app/post/new.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]  
import { useState } from 'react';  
import {  
  KeyboardAvoidingView,  
  Platform,  
  Pressable,  
  ScrollView,  
  StyleSheet,  
  Switch,  
  Text,  
  TextInput,  
  View,  
} from 'react-native';  
import { SafeAreaView } from 'react-native-safe-area-context';  
import { StatusBar } from 'expo-status-bar';  
import { useRouter } from 'expo-router';  
import { X } from 'lucide-react-native';  
import { Button } from '@components/ui/Button';  
import { Input } from '@components/ui/Input';  
import { LoadingState } from '@components/ui/LoadingState';  
import { colors, radius, spacing, typography } from '@theme';  
import { useCategories } from '@hooks/useCategories';  
import { useCreatePost } from '@hooks/usePosts';  
import { toApiError } from '@api/client';  
  
const TITLE_MAX = 150;  
const CONTENT_MAX = 5000;  
const CONTENT_MIN = 10;  
  
export default function NewPostScreen() {  
  const router = useRouter();  
  const { data: categories, isLoading: catsLoading } = useCategories();  
  const createMutation = useCreatePost();  
  
  const [categoryId, setCategoryId] = useState<string | null>(null);  
  const [title, setTitle] = useState('');  
  const [content, setContent] = useState('');  
  const [isAnonymous, setIsAnonymous] = useState(false);  
  const [topError, setTopError] = useState<string | null>(null);  
  
  const titleTrim = title.trim();  
  const contentTrim = content.trim();  
  const canSubmit =  
    !!categoryId &&  
    titleTrim.length >= 3 &&  
    contentTrim.length >= CONTENT_MIN &&  
    !createMutation.isPending;  
  
  const onSubmit = async () => {  
    if (!canSubmit || !categoryId) return;  
    setTopError(null);  
    try {  
      const post = await createMutation.mutateAsync({  
        categoryId,  
        title: titleTrim,  
        content: contentTrim,  
        isAnonymous,  
      });  
      router.replace(`/post/${post.id}`);  
    } catch (e) {  
      setTopError(toApiError(e).message);  
    }  
  };  
  
  return (  
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>  
      <StatusBar style="dark" />  
  
      <View style={styles.header}>  
        <Pressable onPress={() => router.back()} hitSlop={8} style={styles.iconBtn}>  
          <X size={22} color={colors.text} />  
        </Pressable>  
        <Text style={styles.headerTitle}>Paylaş</Text>  
        <View style={{ width: 40 }} />  
      </View>  
  
      <KeyboardAvoidingView  
        style={styles.flex}>  
        <ScrollView
```

```

contentContainerStyle={styles.content}
keyboardShouldPersistTaps="handled"
showsVerticalScrollIndicator={false}
>
<Text style={styles.sectionLabel}>Kategori</Text>
{catsLoading ? (
  <LoadingState size="small" />
) : (
  <View style={styles.chips}>
    {categories?.map((c) => {
      const active = c.id === categoryId;
      return (
        <Pressable
          key={c.id}
          onPress={() => setCategoryId(c.id)}
          style={[styles.chip, active && styles.chipActive]}
        >
          <Text style={[styles.chipLabel, active && styles.chipLabelActive]}>
            {c.name}
          </Text>
        </Pressable>
      );
    })}
  </View>
)}

<View style={{ height: spacing.lg }} />

<Input
  label="Başlık"
  placeholder="Kısa bir başlık yaz"
  value={title}
  onChangeText={(t) => setTitle(t.length > TITLE_MAX ? t.slice(0, TITLE_MAX) : t)}
  helper={` ${titleTrim.length}/${TITLE_MAX}`}
/>

<View style={{ height: spacing.lg }} />

<Text style={styles.sectionLabel}>İçerik</Text>
<View style={styles.textareaWrap}>
  <TextInput
    value={content}
    onChangeText={(t) => setContent(t.length > CONTENT_MAX ? t.slice(0, CONTENT_MAX) : t)}
    placeholder="Bugün nasıl hissediyorsun? Yaşadıklarınızı paylaşmak iyi gelir..."
    placeholderTextColor={colors.textMuted}
    multiline
    style={styles.textarea}
    textAlignVertical="top"
  />
</View>
<Text style={styles.helper}>
  {contentTrim.length}/{CONTENT_MAX} {contentTrim.length < CONTENT_MIN ? `(en az ${CONTENT_MIN})`
: ''}
</Text>

<View style={{ height: spacing.lg }} />

<View style={styles.anonRow}>
  <View style={{ flex: 1 }}>
    <Text style={styles.anonTitle}>Anonim paylaş</Text>
    <Text style={styles.anonDesc}>İsmin görünmez, sadece "Anonim" yazar.</Text>
  </View>
  <Switch
    value={isAnonymous}
    onValueChange={setIsAnonymous}
    trackColor={{ false: colors.divider, true: colors.primary }}
    thumbColor={colors.background}
  />
</View>

{topError && (
  <View style={styles.errorBox}>
    <Text style={styles.errorText}>{topError}</Text>
  </View>
)}
</ScrollView>

<View style={styles.footer}>
  <Button
    label="Paylaş"
    onPress={onSubmit}
    loading={createMutation.isPending}
    disabled={!canSubmit}
    fullWidth
    size="lg"
  />
</View>
</KeyboardAvoidingView>

```

```

    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  flex: { flex: 1 },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: spacing.lg,
    paddingVertical: spacing.sm,
    borderBottomWidth: 1,
    borderBottomColor: colors.divider,
  },
  iconBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerTitle: {
    ...typography.subtitle,
    color: colors.text,
  },
  content: {
    padding: spacing.lg,
    paddingBottom: spacing.xxxl,
  },
  sectionLabel: {
    ...typography.label,
    color: colors.text,
    marginBottom: spacing.sm,
  },
  chips: {
    flexDirection: 'row',
    flexWrap: 'wrap',
    gap: spacing.sm,
  },
  chip: {
    paddingVertical: spacing.sm,
    paddingHorizontal: spacing.md,
    borderRadius: radius.pill,
    borderWidth: 1.5,
    borderColor: colors.border,
    backgroundColor: colors.background,
  },
  chipActive: {
    backgroundColor: colors.primarySoft,
    borderColor: colors.primary,
  },
  chipLabel: {
    ...typography.caption,
    color: colors.textSecondary,
    fontWeight: '600',
  },
  chipLabelActive: {
    color: colors.primaryDark,
  },
  textareaWrap: {
    borderWidth: 1.5,
    borderColor: colors.border,
    borderRadius: radius.md,
    backgroundColor: colors.surface,
    padding: spacing.lg,
    minHeight: 180,
  },
  textarea: {
    ...typography.body,
    color: colors.text,
    minHeight: 160,
    padding: 0,
  },
  helper: {
    ...typography.caption,
    color: colors.textMuted,
    marginTop: spacing.xs,
    textAlign: 'right',
  },
  anonRow: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: colors.surface,
    borderRadius: radius.md,
    padding: spacing.lg,
    gap: spacing.md,
  },
});

```

```

    },
    anonTitle: {
      ...typography.label,
      color: colors.text,
      marginBottom: 2,
    },
    anonDesc: {
      ...typography.caption,
      color: colors.textMuted,
    },
    errorBox: {
      marginTop: spacing.lg,
      backgroundColor: colors.dangerSoft,
      borderRadius: radius.md,
      padding: spacing.md,
    },
    errorText: {
      ...typography.caption,
      color: colors.danger,
    },
    footer: {
      paddingHorizontal: spacing.lg,
      paddingVertical: spacing.md,
      borderTopWidth: 1,
      borderTopColor: colors.divider,
      backgroundColor: colors.background,
    },
  },
});

```

mobile/app/report.tsx

```

// Therapp | Hazırlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import {
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { Check, X } from 'lucide-react-native';
import { Button } from '@components/ui/Button';
import { colors, radius, spacing, typography } from '@theme';
import { useCreateReport } from '@hooks/useReport';
import { toApiError } from '@api/client';
import type { ReportReason, ReportTargetType } from '@api/reports.api';

type ReasonOption = { value: ReportReason; label: string; description?: string };

const REASONS: ReasonOption[] = [
  { value: 0, label: 'Spam', description: 'Tekrarlı veya reklam içeriği' },
  { value: 1, label: 'Taciz / Zorbalık', description: 'Hakaret, tehdit, kişisel saldırı' },
  { value: 2, label: 'Kendine zarar verme', description: 'Acil destek gerektiren içerik' },
  { value: 3, label: 'Yanlış bilgi', description: 'Yanıltıcı tıbbi/sağlık bilgisi' },
  { value: 4, label: 'Nefret söylemi', description: 'Ayrımcı ya da damgalayıcı dil' },
  { value: 5, label: 'Cinsel içerik', description: 'Uygunsuz cinsel içerik' },
  { value: 99, label: 'Diğer', description: 'Açıklama bırak' },
];

export default function ReportScreen() {
  const router = useRouter();
  const params = useLocalSearchParams<{ type?: string; id?: string }>();
  const targetType = (Number(params.type ?? '0') as ReportTargetType);
  const targetId = params.id ?? '';

  const [reason, setReason] = useState<ReportReason | null>(null);
  const [description, setDescription] = useState('');
  const [sent, setSent] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const create = useCreateReport();

  const canSubmit = reason !== null && targetId.length > 0 && !create.isPending;

  const onSubmit = async () => {
    if (!canSubmit || reason === null) return;
    setError(null);
    try {
      await create.mutateAsync({
        targetType,

```

```

        targetId,
        reason,
        description: description.trim() || undefined,
    });
    setSent(true);
    setTimeout(() => router.back(), 1200);
} catch (e) {
    setError(toApiError(e).message);
}
}
};

return (
    <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
        <StatusBar style="dark" />

        <View style={styles.header}>
            <Pressable onPress={() => router.back()} hitSlop={8} style={styles.iconBtn}>
                <X size={22} color={colors.text} />
            </Pressable>
            <Text style={styles.headerTitle}>İçeriği bildir</Text>
            <View style={{ width: 40 }} />
        </View>

        <KeyboardAvoidingView
            style={styles.flex}
            behavior={Platform.OS === 'ios' ? 'padding' : undefined}
        >
            <ScrollView contentContainerStyle={styles.content} keyboardShouldPersistTaps="handled">
                <Text style={styles.intro}>
                    Bu içerik neden topluluk kurallarımıza aykırı? Bildirimin moderatör ekibimize iletilir.
                </Text>

                <View style={styles.list}>
                    {REASONS.map((r) => {
                        const active = reason === r.value;
                        return (
                            <Pressable
                                key={r.value}
                                onPress={() => setReason(r.value)}
                                style={styles.option, active && styles.optionActive}>
                                <View style={{ flex: 1 }}>
                                    <Text style={styles.optionLabel, active && { color: colors.primaryDark }}>
                                        {r.label}
                                    </Text>
                                    <Text>
                                        {r.description && <Text style={styles.optionDesc}>{r.description}</Text>
                                    </Text>
                                    <View style={styles.radio, active && styles.radioActive}>
                                        {active && <Check size={14} color={colors.textOnPrimary} strokeWidth={3} />}
                                    </View>
                                </Pressable>
                            </View>
                        );
                    })}
                </View>

                {reason === 99 && (
                    <View style={styles.textareaWrap}>
                        <TextInput
                            value={description}
                            onChangeText={(t) => setDescription(t.length > 500 ? t.slice(0, 500) : t)}
                            placeholder="Lütfen açıkla..."
                            placeholderTextColor={colors.textMuted}
                            multiline
                            style={styles.textarea}
                            textAlignVertical="top"
                        />
                        <Text style={styles.helper}>{description.length}/500</Text>
                    </View>
                )}

                {error && (
                    <View style={styles.errorBox}>
                        <Text style={styles.errorText}>{error}</Text>
                    </View>
                )}
                {sent && (
                    <View style={styles.sentBox}>
                        <Text style={styles.sentText}>
                            Bildirimin alındı. Moderasyon ekibi inceleyecek.
                        </Text>
                    </View>
                )}
            </ScrollView>

            <View style={styles.footer}>
                <Button
                    label={sent ? 'Gönderildi' : 'Bildirimi gönder'}
                    onPress={onSubmit}
                />
            </View>
        </SafeAreaView>
    );
}

```

```

        loading={create.isPending}
        disabled={!canSubmit || sent}
        fullWidth
        size="lg"
      />
    </View>
  </KeyboardAvoidingView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  flex: { flex: 1 },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: spacing.lg,
    paddingVertical: spacing.sm,
    borderBottomWidth: 1,
    borderBottomColor: colors.divider,
  },
  iconBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerTitle: {
    ...typography.subtitle,
    color: colors.text,
  },
  content: {
    padding: spacing.lg,
    gap: spacing.lg,
    paddingBottom: spacing.xxxl,
  },
  intro: {
    ...typography.body,
    color: colors.textSecondary,
  },
  list: {
    gap: spacing.sm,
  },
  option: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: colors.surface,
    borderRadius: radius.md,
    padding: spacing.lg,
    borderWidth: 1.5,
    borderColor: colors.border,
    gap: spacing.md,
  },
  optionActive: {
    borderColor: colors.primary,
    backgroundColor: colors.primarySoft,
  },
  optionLabel: {
    ...typography.label,
    color: colors.text,
    marginBottom: 2,
  },
  optionDesc: {
    ...typography.caption,
    color: colors.textMuted,
  },
  radio: {
    width: 24,
    height: 24,
    borderRadius: 12,
    borderWidth: 2,
    borderColor: colors.border,
    alignItems: 'center',
    justifyContent: 'center',
  },
  radioActive: {
    backgroundColor: colors.primary,
    borderColor: colors.primary,
  },
  textAreaWrap: {
    borderWidth: 1.5,
    borderColor: colors.border,
    borderRadius: radius.md,
    backgroundColor: colors.surface,
    padding: spacing.lg,
  },
});

```

```

    },
    textarea: {
      ...typography.body,
      color: colors.text,
      minHeight: 100,
      padding: 0,
    },
    helper: {
      ...typography.caption,
      color: colors.textMuted,
      textAlign: 'right',
      marginTop: spacing.xs,
    },
    errorBox: {
      backgroundColor: colors.dangerSoft,
      borderRadius: radius.md,
      padding: spacing.md,
    },
    errorText: {
      ...typography.caption,
      color: colors.danger,
    },
    sentBox: {
      backgroundColor: colors.primarySoft,
      borderRadius: radius.md,
      padding: spacing.md,
    },
    sentText: {
      ...typography.caption,
      color: colors.primaryDark,
    },
    footer: {
      paddingHorizontal: spacing.lg,
      paddingVertical: spacing.md,
      borderTopWidth: 1,
      borderTopColor: colors.divider,
    },
  },
});

```

mobile/app/settings.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useEffect, useState } from 'react';
import {
  Alert,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { useRouter } from 'expo-router';
import {
  ArrowLeft,
  ChevronRight,
  Info,
  Logout,
  ShieldAlert,
  Trash2,
  UserCog,
} from 'lucide-react-native';
import { Button } from '@components/ui/Button';
import { Input } from '@components/ui/Input';
import { EmergencyHelpSheet } from '@components/common/EmergencyHelpSheet';
import { colors, radius, spacing, typography } from '@theme';
import { useAuthStore } from '@store/authStore';
import { useMe, useUpdateMe, useDeleteMe } from '@hooks/useUser';
import { toApiError } from '@api/client';
import { DISCLAIMER_BODY, DISCLAIMER_TITLE } from '@constants/disclaimer';

export default function SettingsScreen() {
  const router = useRouter();
  const logout = useAuthStore((s) => s.logout);
  const { data: me } = useMe();
  const updateMe = useUpdateMe();
  const deleteMe = useDeleteMe();

  const [displayName, setDisplayName] = useState('');
  const [bio, setBio] = useState('');
  const [avatarUrl, setAvatarUrl] = useState('');
  const [editError, setEditError] = useState<string | null>(null);
  const [saved, setSaved] = useState(false);

  const [emergencyOpen, setEmergencyOpen] = useState(false);

```

```

const [disclaimerOpen, setDisclaimerOpen] = useState(false);

useEffect(() => {
  if (me) {
    setDisplayName(me.displayName);
    setBio(me.bio ?? '');
    setAvatarUrl(me.avatarUrl ?? '');
  }
}, [me]);

const onSave = async () => {
  if (displayName.trim().length < 2) {
    setEditError('Görünen ad en az 2 karakter olmalı.');
```

return;

```

  }
  setEditError(null);
  try {
    await updateMe.mutateAsync({
      displayName: displayName.trim(),
      bio: bio.trim() || null,
      avatarUrl: avatarUrl.trim() || null,
    });
    setSaved(true);
    setTimeout(() => setSaved(false), 1500);
  } catch (e) {
    setEditError(toApiError(e).message);
  }
};

const confirmDelete = () => {
  Alert.alert(
    'Hesabını sil',
    'Bu işlem geri alınamaz. Tüm paylaşımların ve yorumların gizlenir, oturumların kapatılır. Devam etmek istiyor musun?',
    [
      { text: 'Vazgeç', style: 'cancel' },
      {
        text: 'Sil',
        style: 'destructive',
        onPress: async () => {
          try {
            await deleteMe.mutateAsync();
            await logout();
            router.replace('/(auth)/login');
          } catch (e) {
            Alert.alert('Hata', toApiError(e).message);
          }
        },
      },
    ],
  );
};

const confirmLogout = () => {
  Alert.alert('Çıkış yap', 'Hesabından çıkış yapmak istediğinden emin misin?', [
    { text: 'Vazgeç', style: 'cancel' },
    {
      text: 'Çıkış',
      style: 'destructive',
      onPress: async () => {
        await logout();
        router.replace('/(auth)/login');
      },
    },
  ],
);
};

return (
  <SafeAreaView style={styles.safe} edges={['top', 'left', 'right']}>
    <StatusBar style="dark" />

    <View style={styles.header}>
      <Pressable onPress={() => router.back()} hitSlop={8} style={styles.iconBtn}>
        <ArrowLeft size={22} color={colors.text} />
      </Pressable>
      <Text style={styles.headerTitle}>Ayarlar</Text>
      <View style={{ width: 40 }} />
    </View>

    <ScrollView
      contentContainerStyle={styles.content}
      keyboardShouldPersistTaps="handled"
      showsVerticalScrollIndicator={false}
    >
      <Section icon={<UserCog size={18} color={colors.primaryDark} />} title="Profil">
        <Input
          label="Görünen ad"
          value={displayName}

```

```

        onChangeText={setDisplayName}
        placeholder="Adın"
      />
      <Input
        label="Hakkımda"
        value={bio}
        onChangeText={setBio}
        placeholder="Kısa bir tanıtım (opsiyonel)"
        multiline
        numberOfLines={3}
        helper={` ${bio.length}/280` }
      />
      <Input
        label="Avatar URL"
        value={avatarUrl}
        onChangeText={setAvatarUrl}
        placeholder="https:// (opsiyonel)"
        autoCapitalize="none"
      />
      {editError && <Text style={styles.errorText}>{editError}</Text>}
      <Button
        label={saved ? 'Kaydedildi' : 'Değişiklikleri kaydet'}
        onPress={onSave}
        loading={updateMe.isPending}
        fullWidth
      />
    </Section>

    <Section icon={<Info size={18} color={colors.primaryDark} />} title="Bilgi & Destek">
      <Row label="Disclaimer'ı tekrar göster" onPress={() => setDisclaimerOpen(true)} />
      <Row
        label="Acil destek kaynakları"
        onPress={() => setEmergencyOpen(true)}
        danger
        icon={<ShieldAlert size={18} color={colors.warning} />}
      />
    </Section>

    <Section icon={<Logout size={18} color={colors.primaryDark} />} title="Hesap">
      <Row label="Çıkış yap" onPress={confirmLogout} />
      <Row
        label="Hesabımı sil"
        onPress={confirmDelete}
        danger
        icon={<Trash2 size={18} color={colors.danger} />}
      />
    </Section>

    <Text style={styles.versionText}>Therapp v1.0.0</Text>
  </ScrollView>

  <EmergencyHelpSheet visible={emergencyOpen} onClose={() => setEmergencyOpen(false)} />
  <DisclaimerModal visible={disclaimerOpen} onClose={() => setDisclaimerOpen(false)} />
</SafeAreaView>
);
}

function Section({
  icon,
  title,
  children,
}): {
  icon: React.ReactNode;
  title: string;
  children: React.ReactNode;
} {
  return (
    <View style={styles.section}>
      <View style={styles.sectionHeader}>
        {icon}
        <Text style={styles.sectionTitle}>{title}</Text>
      </View>
      <View style={styles.sectionBody}>{children}</View>
    </View>
  );
}

function Row({
  label,
  onPress,
  danger,
  icon,
}): {
  label: string;
  onPress?: () => void;
  danger?: boolean;
  icon?: React.ReactNode;
} {

```

```

return (
  <Pressable
    onPress={onPress}
    style={({ pressed }) => [styles.row, pressed && { opacity: 0.7 }]}
  >
    <View style={styles.rowLeft}>
      <icon>
        <Text style={[styles.rowLabel, danger && { color: colors.danger }]}>{label}</Text>
      </View>
      <ChevronRight size={18} color={colors.textMuted} />
    </Pressable>
  </View>
);
}

function DisclaimerModal({ visible, onClose }: { visible: boolean; onClose: () => void }) {
  if (!visible) return null;
  return (
    <View style={styles.disclaimerBackdrop}>
      <View style={styles.disclaimerCard}>
        <Text style={styles.disclaimerTitle}>{DISCLAIMER_TITLE}</Text>
        <Text style={styles.disclaimerBody}>{DISCLAIMER_BODY}</Text>
        <Button label="Anladım" onPress={onClose} fullWidth />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.background },
  header: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: spacing.lg,
    paddingVertical: spacing.sm,
    borderBottomWidth: 1,
    borderBottomColor: colors.divider,
  },
  iconBtn: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  headerTitle: {
    ...typography.subtitle,
    color: colors.text,
  },
  content: {
    padding: spacing.lg,
    paddingBottom: spacing.xxxl,
    gap: spacing.lg,
  },
  section: {
    gap: spacing.sm,
  },
  sectionHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: spacing.sm,
    paddingHorizontal: spacing.xs,
  },
  sectionTitle: {
    ...typography.label,
    color: colors.textSecondary,
    textTransform: 'uppercase',
    letterSpacing: 0.5,
  },
  sectionBody: {
    backgroundColor: colors.surface,
    borderRadius: radius.lg,
    padding: spacing.lg,
    gap: spacing.md,
  },
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingVertical: spacing.sm,
  },
  rowLeft: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: spacing.md,
  },
  rowLabel: {
    ...typography.body,
  },
});

```

```

    color: colors.text,
  },
  errorText: {
    ...typography.caption,
    color: colors.danger,
  },
  versionText: {
    ...typography.caption,
    color: colors.textMuted,
    textAlign: 'center',
    marginTop: spacing.md,
  },
  disclaimerBackdrop: {
    position: 'absolute',
    top: 0,
    left: 0,
    right: 0,
    bottom: 0,
    backgroundColor: colors.overlay,
    alignItems: 'center',
    justifyContent: 'center',
    padding: spacing.lg,
  },
  disclaimerCard: {
    backgroundColor: colors.background,
    borderRadius: radius.lg,
    padding: spacing.xl,
    gap: spacing.md,
    width: '100%',
    maxWidth: 480,
  },
  disclaimerTitle: {
    ...typography.subtitle,
    color: colors.text,
  },
  disclaimerBody: {
    ...typography.body,
    color: colors.textSecondary,
  },
},
});

```

mobile/babel.config.js

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
module.exports = function (api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo'],
    plugins: [
      'react-native-worklets/plugin'
    ]
  };
};

```

mobile/metro.config.js

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
const { getDefaultConfig } = require('expo/metro-config');

const config = getDefaultConfig(__dirname);

module.exports = config;

```

mobile/package.json

```

{
  "name": "therapp-mobile",
  "version": "1.0.0",
  "main": "expo-router/entry",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@hookform/resolvers": "^3.9.1",
    "@tanstack/react-query": "^5.59.16",
    "@types/react": "~19.1.10",
    "axios": "^1.7.7",
    "babel-preset-expo": "~54.0.10",
    "date-fns": "^4.1.0",

```

```

    "expo": "^54.0.35",
    "expo-asset": "~12.0.13",
    "expo-constants": "~18.0.13",
    "expo-font": "~14.0.12",
    "expo-linking": "~8.0.12",
    "expo-router": "~6.0.24",
    "expo-secure-store": "~15.0.8",
    "expo-splash-screen": "~31.0.13",
    "expo-status-bar": "~3.0.9",
    "expo-system-ui": "~6.0.9",
    "lucide-react-native": "^0.460.0",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "react-hook-form": "^7.53.1",
    "react-native": "0.81.5",
    "react-native-gesture-handler": "~2.28.0",
    "react-native-reanimated": "~4.1.1",
    "react-native-safe-area-context": "~5.6.0",
    "react-native-screens": "~4.16.0",
    "react-native-svg": "15.12.1",
    "react-native-web": "~0.21.0",
    "react-native-worklets": "0.5.1",
    "zod": "^3.23.8",
    "zustand": "^5.0.1"
  },
  "devDependencies": {
    "@babel/core": "^7.25.2",
    "@types/react": "~19.1.10",
    "typescript": "^5.3.3"
  },
  "private": true
}

```

mobile/src/api/auth.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient, bareClient } from './client';
import { endpoints } from './endpoints';
import type { AuthUser } from '@types/user.types';

export type AuthResponse = {
  accessToken: string;
  refreshToken: string;
  accessTokenExpiresAt: string;
  user: AuthUser;
};

export type RegisterPayload = {
  username: string;
  email: string;
  password: string;
  displayName: string;
};

export type LoginPayload = {
  email: string;
  password: string;
};

export const authApi = {
  register: async (payload: RegisterPayload) => {
    const { data } = await bareClient.post<AuthResponse>(endpoints.auth.register, payload);
    return data;
  },
  login: async (payload: LoginPayload) => {
    const { data } = await bareClient.post<AuthResponse>(endpoints.auth.login, payload);
    return data;
  },
  refresh: async (refreshToken: string) => {
    const { data } = await bareClient.post<AuthResponse>(endpoints.auth.refresh, { refreshToken });
    return data;
  },
  logout: async (refreshToken: string) => {
    await apiClient.post(endpoints.auth.logout, { refreshToken });
  },
};

```

mobile/src/api/categories.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from './client';
import { endpoints } from './endpoints';
import type { Category } from '@types/category.types';

export const categoriesApi = {

```

```

list: async (): Promise<Category[]> => {
  const { data } = await apiClient.get<Category[]>(endpoints.categories.list);
  return data;
},
bySlug: async (slug: string): Promise<Category> => {
  const { data } = await apiClient.get<Category>(endpoints.categories.bySlug(slug));
  return data;
},
};

```

mobile/src/api/client.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import axios, { AxiosError, AxiosRequestConfig } from 'axios';
import { getApiUrl } from '@utils/getApiUrl';
import { logger } from '@utils/logger';
import type { ApiError } from '@types/api.types';

const baseUrl = getApiUrl();

export const apiClient = axios.create({
  baseUrl,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' },
});

// Bare client for refresh (avoids interceptor recursion)
export const bareClient = axios.create({
  baseUrl,
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' },
});

// ----- Auth wiring (registered by the auth store) -----
// We intentionally do NOT import the auth store here. That would create a
// require cycle (auth.api -> client -> authStore -> auth.api) which can leave
// modules uninitialized under the new architecture. Instead the store
// registers its handlers at startup via configureAuth().
type AuthHandlers = {
  getToken: () => string | null;
  refresh: () => Promise<string | null>;
  onAuthFailure: () => Promise<void>;
};

let authHandlers: AuthHandlers = {
  getToken: () => null,
  refresh: async () => null,
  onAuthFailure: async () => {},
};

export function configureAuth(handlers: AuthHandlers) {
  authHandlers = handlers;
}

// ----- Request: attach token -----
apiClient.interceptors.request.use((config) => {
  const token = authHandlers.getToken();
  if (token && config.headers) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// ----- Response: auto-refresh on 401 -----
let refreshInFlight: Promise<string | null> | null = null;

apiClient.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const original = error.config as AxiosRequestConfig & { _retried?: boolean };
    const status = error.response?.status;

    // Don't intercept the refresh / login / register endpoints themselves
    const url = original?.url ?? '';
    const isAuthCall =
      url.includes('/auth/refresh') ||
      url.includes('/auth/login') ||
      url.includes('/auth/register');

    if (status === 401 && !original._retried && !isAuthCall) {
      original._retried = true;

      if (!refreshInFlight) {
        refreshInFlight = authHandlers.refresh().finally(() => {
          refreshInFlight = null;
        });
      }
    }
  }
);

```

```

    }

    try {
      const newToken = await refreshInFlight;
      if (newToken && original.headers) {
        (original.headers as Record<string, string>).Authorization = `Bearer ${newToken}`;
        return apiClient(original);
      }
    } catch (e) {
      logger.warn('Refresh failed', e);
    }

    // refresh failed -> clear session, let caller handle
    await authHandlers.onAuthFailure();
  }

  return Promise.reject(error);
}
);

// ----- Helpers -----
export function toApiError(error: unknown): ApiError {
  if (axios.isAxiosError(error)) {
    const data = error.response?.data as ApiError | undefined;
    if (data?.message) return data;
    return {
      code: 'NETWORK_ERROR',
      message: error.message || 'Bağlantı hatası, lütfen tekrar deneyin.',
    };
  }
  return {
    code: 'UNKNOWN',
    message: 'Beklenmeyen bir hata oluştu.',
  };
}
}

```

mobile/src/api/comments.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from './client';
import { endpoints } from './endpoints';
import type { PagedResult } from '@types/api.types';
import type { Comment } from '@types/post.types';

type PageParams = { page?: number; pageSize?: number };

export const commentsApi = {
  byPost: async (postId: string, params?: PageParams) => {
    const { data } = await apiClient.get<PagedResult<Comment>>(endpoints.comments.byPost(postId), {
      params: { page: params?.page ?? 1, pageSize: params?.pageSize ?? 20 },
    });
    return data;
  },
  replies: async (id: string, params?: PageParams) => {
    const { data } = await apiClient.get<PagedResult<Comment>>(endpoints.comments.replies(id), {
      params: { page: params?.page ?? 1, pageSize: params?.pageSize ?? 20 },
    });
    return data;
  },
  create: async (postId: string, content: string) => {
    const { data } = await apiClient.post<Comment>(endpoints.comments.byPost(postId), { content });
    return data;
  },
  reply: async (commentId: string, content: string) => {
    const { data } = await apiClient.post<Comment>(endpoints.comments.replies(commentId), { content });
    return data;
  },
  delete: async (id: string) => {
    await apiClient.delete(endpoints.comments.byId(id));
  },
  like: async (id: string) => {
    await apiClient.post(endpoints.comments.like(id));
  },
  unlike: async (id: string) => {
    await apiClient.delete(endpoints.comments.like(id));
  },
};

```

mobile/src/api/endpoints.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export const endpoints = {
  auth: {
    register: '/auth/register',
    login: '/auth/login',
  },
};

```

```

    refresh: '/auth/refresh',
    logout: '/auth/logout',
  },
  users: {
    me: '/users/me',
    byUsername: (username: string) => `/users/${encodeURIComponent(username)}`,
  },
  categories: {
    list: '/categories',
    bySlug: (slug: string) => `/categories/${encodeURIComponent(slug)}`,
  },
  posts: {
    feed: '/posts',
    byCategory: (slug: string) => `/posts/category/${encodeURIComponent(slug)}`,
    mine: '/posts/me',
    mySaved: '/posts/me/saved',
    byId: (id: string) => `/posts/${id}`,
    like: (id: string) => `/posts/${id}/like`,
    save: (id: string) => `/posts/${id}/save`,
  },
  comments: {
    byPost: (postId: string) => `/posts/${postId}/comments`,
    replies: (id: string) => `/comments/${id}/replies`,
    byId: (id: string) => `/comments/${id}`,
    like: (id: string) => `/comments/${id}/like`,
  },
  notifications: {
    list: '/notifications',
    unreadCount: '/notifications/unread-count',
    read: (id: string) => `/notifications/${id}/read`,
    readAll: '/notifications/read-all',
  },
  reports: {
    create: '/reports',
  },
} as const;

```

mobile/src/api/notifications.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from './client';
import { endpoints } from './endpoints';
import type { PagedResult } from '@types/api.types';
import type { AppNotification } from '@types/notification.types';

type PageParams = { page?: number; pageSize?: number };

export const notificationsApi = {
  list: async (params?: PageParams) => {
    const { data } = await apiClient.get<PagedResult<AppNotification>>(endpoints.notifications.list, {
      params: { page: params?.page ?? 1, pageSize: params?.pageSize ?? 20 },
    });
    return data;
  },
  unreadCount: async () => {
    const { data } = await apiClient.get<{ count: number }>(endpoints.notifications.unreadCount);
    return data.count;
  },
  markRead: async (id: string) => {
    await apiClient.post(endpoints.notifications.read(id));
  },
  markAllRead: async () => {
    await apiClient.post(endpoints.notifications.readAll);
  },
};

```

mobile/src/api/posts.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from './client';
import { endpoints } from './endpoints';
import type { PagedResult } from '@types/api.types';
import type { Post, PostListItem, FeedSort } from '@types/post.types';

type FeedParams = {
  sort?: FeedSort;
  page?: number;
  pageSize?: number;
};

function feedQuery(params?: FeedParams) {
  const sortMap: Record<FeedSort, number> = { new: 0, popular: 1 };
  return {
    sort: sortMap[params?.sort ?? 'new'],
    page: params?.page ?? 1,
  };
}

```

```

    pageSize: params?.pageSize ?? 20,
  };
}

export type CreatePostPayload = {
  categoryId: string;
  title: string;
  content: string;
  isAnonymous: boolean;
};

export type UpdatePostPayload = {
  title: string;
  content: string;
};

export const postsApi = {
  feed: async (params?: FeedParams) => {
    const { data } = await apiClient.get<PagedResult<PostListItem>>(endpoints.posts.feed, {
      params: feedQuery(params),
    });
    return data;
  },
  byCategory: async (slug: string, params?: FeedParams) => {
    const { data } = await apiClient.get<PagedResult<PostListItem>>(endpoints.posts.byCategory(slug), {
      params: feedQuery(params),
    });
    return data;
  },
  mine: async (params?: FeedParams) => {
    const { data } = await apiClient.get<PagedResult<PostListItem>>(endpoints.posts.mine, {
      params: feedQuery(params),
    });
    return data;
  },
  mySaved: async (params?: FeedParams) => {
    const { data } = await apiClient.get<PagedResult<PostListItem>>(endpoints.posts.mySaved, {
      params: feedQuery(params),
    });
    return data;
  },
  byId: async (id: string) => {
    const { data } = await apiClient.get<Post>(endpoints.posts.byId(id));
    return data;
  },
  create: async (payload: CreatePostPayload) => {
    const { data } = await apiClient.post<Post>(endpoints.posts.feed, payload);
    return data;
  },
  update: async (id: string, payload: UpdatePostPayload) => {
    const { data } = await apiClient.patch<Post>(endpoints.posts.byId(id), payload);
    return data;
  },
  delete: async (id: string) => {
    await apiClient.delete(endpoints.posts.byId(id));
  },
  like: async (id: string) => {
    await apiClient.post(endpoints.posts.like(id));
  },
  unlike: async (id: string) => {
    await apiClient.delete(endpoints.posts.like(id));
  },
  save: async (id: string) => {
    await apiClient.post(endpoints.posts.save(id));
  },
  unsaved: async (id: string) => {
    await apiClient.delete(endpoints.posts.save(id));
  },
};

```

mobile/src/api/reports.api.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from '../client';
import { endpoints } from '../endpoints';

export type ReportTargetType = 0 | 1 | 2; // 0=post, 1=comment, 2=user
export type ReportReason = 0 | 1 | 2 | 3 | 4 | 5 | 99;

export type CreateReportPayload = {
  targetType: ReportTargetType;
  targetId: string;
  reason: ReportReason;
  description?: string;
};

```

```
export const reportsApi = {
  create: async (payload: CreateReportPayload) => {
    await apiClient.post(endpoints.reports.create, payload);
  },
};
```

mobile/src/api/users.api.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { apiClient } from './client';
import { endpoints } from './endpoints';
import type { UserMe, UserProfile } from '@types/user.types';

export type UpdateProfilePayload = {
  displayName: string;
  bio?: string | null;
  avatarUrl?: string | null;
};

export const usersApi = {
  me: async () => {
    const { data } = await apiClient.getUserMe(endpoints.users.me);
    return data;
  },
  updateMe: async (payload: UpdateProfilePayload) => {
    const { data } = await apiClient.patch<UserMe>(endpoints.users.me, payload);
    return data;
  },
  deleteMe: async () => {
    await apiClient.delete(endpoints.users.me);
  },
  byUsername: async (username: string) => {
    const { data } = await apiClient.getUserProfile(endpoints.users.byUsername(username));
    return data;
  },
};
```

mobile/src/components/category/CategoryCard.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { CategoryIcon } from '@components/ui/CategoryIcon';
import { colors, radius, spacing, typography } from '@theme';
import type { Category } from '@types/category.types';

type Props = {
  category: Category;
  onPress?: () => void;
};

export function CategoryCard({ category, onPress }: Props) {
  return (
    <Pressable
      onPress={onPress}
      style={({ pressed }) => [styles.card, pressed && styles.pressed]}
    >
      <View style={[styles.iconWrap, { backgroundColor: hexToSoft(category.color) }]}>
        <CategoryIcon name={category.icon} color={category.color} size={26} />
      </View>
      <Text style={styles.name} numberOfLines={1}>
        {category.name}
      </Text>
      <Text style={styles.count}>
        {category.postCount} {category.postCount === 1 ? 'gönderi' : 'gönderi'}
      </Text>
    </Pressable>
  );
}

function hexToSoft(hex: string): string {
  const clean = hex.replace('#', '');
  if (clean.length !== 6) return colors.primarySoft;
  const r = parseInt(clean.slice(0, 2), 16);
  const g = parseInt(clean.slice(2, 4), 16);
  const b = parseInt(clean.slice(4, 6), 16);
  const mix = (c: number) => Math.round(c + (255 - c) * 0.78);
  return `rgb(${mix(r)}, ${mix(g)}, ${mix(b)})`;
}

const styles = StyleSheet.create({
  card: {
    flex: 1,
    backgroundColor: colors.surface,
    borderRadius: radius.lg,
    borderWidth: 1,
  },
});
```

```

borderColor: colors.border,
padding: spacing.lg,
minHeight: 130,
justifyContent: 'space-between',
},
pressed: { opacity: 0.9, transform: [{ scale: 0.98 }] },
iconWrap: {
width: 48,
height: 48,
borderRadius: radius.md,
alignItems: 'center',
justifyContent: 'center',
marginBottom: spacing.md,
},
name: {
...typography.subtitle,
color: colors.text,
marginBottom: 2,
},
count: {
...typography.caption,
color: colors.textMuted,
},
});

```

mobile/src/components/category/CategoryChip.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { StyleSheet, Text, View } from 'react-native';
import { colors, radius, spacing, typography } from '@theme';

type Props = {
name: string;
color: string;
};

export function CategoryChip({ name, color }: Props) {
return (
<View style={[[styles.chip, { backgroundColor: hexToSoft(color) }]]}>
<View style={[[styles.dot, { backgroundColor: color }]]} />
<Text style={[[styles.label, { color: colors.text }]]}>{name}</Text>
</View>
);
}

// Convert a hex color to a very light pastel by mixing it with white.
function hexToSoft(hex: string): string {
const clean = hex.replace('#', '');
if (clean.length !== 6) return colors.primarySoft;
const r = parseInt(clean.slice(0, 2), 16);
const g = parseInt(clean.slice(2, 4), 16);
const b = parseInt(clean.slice(4, 6), 16);
const mix = (c: number) => Math.round(c + (255 - c) * 0.82);
return `rgb(${mix(r)}, ${mix(g)}, ${mix(b)})`;
}

const styles = StyleSheet.create({
chip: {
flexDirection: 'row',
alignItems: 'center',
paddingHorizontal: spacing.md,
paddingVertical: 4,
borderRadius: radius.pill,
alignSelf: 'flex-start',
gap: 6,
},
dot: {
width: 6,
height: 6,
borderRadius: 3,
},
label: {
...typography.caption,
fontWeight: '600',
},
});

```

mobile/src/components/comment/CommentComposer.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useState } from 'react';
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';
import { Send, X } from 'lucide-react-native';
import { colors, radius, spacing, typography } from '@theme';

```

```

type Props = {
  placeholder?: string;
  replyingTo?: string | null;
  onCancelReply?: () => void;
  submitting?: boolean;
  onSubmit: (content: string) => Promise<void> | void;
};

const MAX_LEN = 1000;

export function CommentComposer({
  placeholder = 'Yorumunu yaz...',
  replyingTo,
  onCancelReply,
  submitting = false,
  onSubmit,
}: Props) {
  const [value, setValue] = useState('');
  const trimmed = value.trim();
  const canSubmit = trimmed.length >= 2 && trimmed.length <= MAX_LEN && !submitting;

  const handle = async () => {
    if (!canSubmit) return;
    await onSubmit(trimmed);
    setValue('');
  };

  return (
    <View style={styles.wrap}>
      {replyingTo && (
        <View style={styles.replyBar}>
          <Text style={styles.replyText} numberOfLines={1}>
            {replyingTo} kullanıcısına yanıt veriyorsun
          </Text>
          <Pressable onPress={onCancelReply} hitSlop={8}>
            <X size={16} color={colors.textMuted} />
          </Pressable>
        </View>
      )}
      <View style={styles.inputRow}>
        <TextInput
          value={value}
          onChangeText={(t) => setValue(t.length > MAX_LEN ? t.slice(0, MAX_LEN) : t)}
          placeholder={placeholder}
          placeholderTextColor={colors.textMuted}
          multiline
          style={styles.input}
        />
        <Pressable
          onPress={handle}
          disabled={!canSubmit}
          style={({ pressed }) => [
            styles.sendBtn,
            !canSubmit && styles.sendBtnDisabled,
            pressed && canSubmit && { opacity: 0.85 },
          ]}
        >
          <Send size={18} color={canSubmit ? colors.textOnPrimary : colors.textMuted} />
        </Pressable>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: {
    borderTopWidth: 1,
    borderTopColor: colors.border,
    backgroundColor: colors.background,
    paddingHorizontal: spacing.lg,
    paddingTop: spacing.sm,
    paddingBottom: spacing.md,
  },
  replyBar: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    backgroundColor: colors.surface,
    borderRadius: radius.sm,
    paddingHorizontal: spacing.md,
    paddingVertical: 6,
    marginBottom: spacing.sm,
  },
  replyText: {
    ...typography.caption,
    color: colors.textSecondary,
    flex: 1,
  },
});

```

```

inputRow: {
  flexDirection: 'row',
  alignItems: 'flex-end',
  gap: spacing.sm,
},
input: {
  ...typography.body,
  flex: 1,
  color: colors.text,
  backgroundColor: colors.surface,
  borderRadius: radius.lg,
  paddingHorizontal: spacing.lg,
  paddingVertical: spacing.md,
  maxHeight: 120,
  minHeight: 44,
},
sendBtn: {
  width: 44,
  height: 44,
  borderRadius: 22,
  backgroundColor: colors.primary,
  alignItems: 'center',
  justifyContent: 'center',
},
sendBtnDisabled: {
  backgroundColor: colors.divider,
},
});

```

mobile/src/components/comment/CommentItem.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { Heart, MessageSquare } from 'lucide-react-native';
import { Avatar } from '@components/ui/Avatar';
import { colors, spacing, typography } from '@theme';
import { relativeTime } from '@utils/formatDate';
import type { Comment } from '@types/post.types';

type Props = {
  comment: Comment;
  onLike?: () => void;
  onReply?: () => void;
  isReply?: boolean;
};

export function CommentItem({ comment, onLike, onReply, isReply = false }: Props) {
  return (
    <View style={[styles.row, isReply && styles.reply]}>
      <Avatar uri={comment.author.avatarUrl} name={comment.author.displayName} size={32} />
      <View style={styles.body}>
        <View style={styles.headerRow}>
          <Text style={styles.name} numberOfLines={1}>
            {comment.author.displayName}
          </Text>
          <Text style={styles.time}>{relativeTime(comment.createdAt)}</Text>
        </View>
        <Text style={styles.content}>{comment.content}</Text>

        <View style={styles.actionsRow}>
          <Pressable
            onPress={onLike}
            hitSlop={8}
            style={({ pressed }) => [styles.actionBtn, pressed && { opacity: 0.6 }]}
          >
            <Heart
              size={14}
              color={comment.isLikedByMe ? colors.like : colors.textMuted}
              fill={comment.isLikedByMe ? colors.like : 'transparent'}
              strokeWidth={1.8}
            />
            <Text
              style={[
                styles.actionLabel,
                comment.isLikedByMe && { color: colors.like, fontWeight: '600' },
              ]}
            >
              {comment.likeCount > 0 ? comment.likeCount : 'Beğen'}
            </Text>
          </Pressable>

          {!isReply && (
            <Pressable
              onPress={onReply}
              hitSlop={8}
              style={({ pressed }) => [styles.actionBtn, pressed && { opacity: 0.6 }]}
            >

```

```

        >
        <MessageSquare size={14} color={colors.textMuted} strokeWidth={1.8} />
        <Text style={styles.actionLabel}>
          {comment.replyCount > 0 ? `${comment.replyCount} yanıt` : 'Yanıtla'}
        </Text>
      </Pressable>
    )}
  </View>
</View>
</View>
);
}

const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    gap: spacing.sm,
    paddingVertical: spacing.md,
  },
  reply: {
    marginLeft: spacing.xxl,
  },
  body: {
    flex: 1,
  },
  headerRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    marginBottom: 4,
  },
  name: {
    ...typography.label,
    color: colors.text,
    flex: 1,
  },
  time: {
    ...typography.caption,
    color: colors.textMuted,
  },
  content: {
    ...typography.body,
    color: colors.textSecondary,
    marginBottom: spacing.xs,
  },
  actionsRow: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: spacing.lg,
    marginTop: 4,
  },
  actionBtn: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 4,
  },
  actionLabel: {
    ...typography.caption,
    color: colors.textMuted,
  },
});

```

mobile/src/components/common/DisclaimerBanner.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { AlertCircle, ChevronRight } from 'lucide-react-native';
import { colors, radius, spacing, typography } from '@theme';
import { DISCLAIMER_SHORT } from '@constants/disclaimer';

type Props = {
  onPress?: () => void;
};

export function DisclaimerBanner({ onPress }: Props) {
  return (
    <Pressable
      onPress={onPress}
      style={({ pressed }) => [styles.container, pressed && styles.pressed]}
    >
      <View style={styles.icon}>
        <AlertCircle size={18} color={colors.warning} />
      </View>
      <Text style={styles.text} numberOfLines={2}>
        {DISCLAIMER_SHORT}
      </Text>
    </Pressable>
  );
}

```

```

        {onPress && <ChevronRight size={18} color={colors.textMuted} />}
      </Pressable>
    );
  }

  const styles = StyleSheet.create({
    container: {
      flexDirection: 'row',
      alignItems: 'center',
      backgroundColor: colors.warningSoft,
      borderRadius: radius.md,
      paddingVertical: spacing.md,
      paddingHorizontal: spacing.lg,
      gap: spacing.sm,
    },
    pressed: { opacity: 0.85 },
    icon: { marginRight: 2 },
    text: {
      ...typography.caption,
      color: colors.text,
      flex: 1,
    },
  });
});

```

mobile/src/components/common/EmergencyHelpSheet.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import {
  Linking,
  Modal,
  Pressable,
  StyleSheet,
  Text,
  View,
} from 'react-native';
import { Phone, ShieldAlert, X } from 'lucide-react-native';
import { colors, radius, spacing, typography } from '@theme';
import { EMERGENCY_RESOURCES } from '@constants/emergencyResources';

type Props = {
  visible: boolean;
  onClose: () => void;
};

export function EmergencyHelpSheet({ visible, onClose }: Props) {
  return (
    <Modal visible={visible} animationType="slide" transparent onRequestClose={onClose}>
      <View style={styles.backdrop}>
        <Pressable style={styles.dismiss} onPress={onClose} />
        <View style={styles.sheet}>
          <View style={styles.handle} />

          <View style={styles.header}>
            <View style={styles.iconWrap}>
              <ShieldAlert size={28} color={colors.warning} strokeWidth={1.8} />
            </View>
            <View style={{ flex: 1 }}>
              <Text style={styles.title}>Acil Yardım Kaynakları</Text>
              <Text style={styles.subtitle}>
                Kendine veya çevrendeki birine zarar verme düşüncen varsa lütfen profesyonel
                destek al.
              </Text>
            </View>
            <Pressable onPress={onClose} hitSlop={8} style={styles.closeBtn}>
              <X size={20} color={colors.textMuted} />
            </Pressable>
          </View>

          <View style={styles.list}>
            {EMERGENCY_RESOURCES.map((r) => (
              <Pressable
                key={r.name}
                onPress={() => {
                  if (r.phone) Linking.openURL(`tel:${r.phone}`).catch(() => {});
                  else if (r.url) Linking.openURL(r.url).catch(() => {});
                }}
                style={({ pressed }) => [styles.resource, pressed && styles.pressed]}
              >
                <View style={{ flex: 1 }}>
                  <Text style={styles.resourceName}>{r.name}</Text>
                  <Text style={styles.resourceDesc}>{r.description}</Text>
                </View>
                {r.phone && (
                  <View style={styles.phoneBadge}>
                    <Phone size={14} color={colors.primaryDark} strokeWidth={2} />
                    <Text style={styles.phoneText}>{r.phone}</Text>
                  </View>
                )}
              </Pressable>
            ))}
          </View>
        </View>
      </Modal>
  );
}

```

```

        </View>
      )}
    </Pressable>
  )}
</View>

    <Text style={styles.disclaimer}>
      Therapp tıbbi tavsiye sunmaz. Acil durumlarda lütfen yetkililerle iletişime geçin.
    </Text>
  </View>
</View>
</Modal>
);
}

const styles = StyleSheet.create({
  backdrop: {
    flex: 1,
    backgroundColor: colors.overlay,
    justifyContent: 'flex-end',
  },
  dismiss: { flex: 1 },
  sheet: {
    backgroundColor: colors.background,
    borderTopLeftRadius: radius.xl,
    borderTopRightRadius: radius.xl,
    paddingHorizontal: spacing.lg,
    paddingBottom: spacing.xxl,
    paddingTop: spacing.sm,
    gap: spacing.lg,
  },
  handle: {
    width: 40,
    height: 4,
    borderRadius: 2,
    backgroundColor: colors.divider,
    alignSelf: 'center',
    marginBottom: spacing.sm,
  },
  header: {
    flexDirection: 'row',
    gap: spacing.md,
    alignItems: 'flex-start',
  },
  iconWrap: {
    width: 48,
    height: 48,
    borderRadius: radius.md,
    backgroundColor: colors.warningSoft,
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    ...typography.subtitle,
    color: colors.text,
    marginBottom: 4,
  },
  subtitle: {
    ...typography.caption,
    color: colors.textMuted,
  },
  closeBtn: {
    width: 32,
    height: 32,
    alignItems: 'center',
    justifyContent: 'center',
  },
  list: {
    gap: spacing.sm,
  },
  resource: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: colors.surface,
    borderRadius: radius.md,
    padding: spacing.lg,
    gap: spacing.md,
  },
  pressed: { opacity: 0.85 },
  resourceName: {
    ...typography.label,
    color: colors.text,
    marginBottom: 2,
  },
  resourceDesc: {
    ...typography.caption,
    color: colors.textMuted,
  },
});

```

```

phoneBadge: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 4,
  backgroundColor: colors.primarySoft,
  borderRadius: radius.pill,
  paddingVertical: 6,
  paddingHorizontal: spacing.md,
},
phoneText: {
  ...typography.label,
  color: colors.primaryDark,
},
disclaimer: {
  ...typography.caption,
  color: colors.textMuted,
  textAlign: 'center',
},
});

```

mobile/src/components/layout/ScreenContainer.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { PropsWithChildren } from 'react';
import {
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StyleSheet,
  View,
  ViewStyle,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { StatusBar } from 'expo-status-bar';
import { colors, spacing } from '@theme';

type Props = PropsWithChildren<{
  scroll?: boolean;
  padded?: boolean;
  contentType?: ViewStyle;
  edges?: ('top' | 'bottom' | 'left' | 'right')[];
}>;

export function ScreenContainer({
  children,
  scroll = false,
  padded = true,
  contentType,
  edges = ['top', 'left', 'right'],
}: Props) {
  const inner = (
    <View style={styles.inner, padded && styles.padded, contentType}>{children}</View>
  );

  const content = scroll ? (
    <ScrollView
      contentContainerStyle={styles.scrollContent}
      keyboardShouldPersistTaps="handled"
      showsVerticalScrollIndicator={false}
    >
      {inner}
    </ScrollView>
  ) : (
    inner
  );

  return (
    <SafeAreaView style={styles.safe} edges={edges}>
      <StatusBar style="dark" />
      <KeyboardAvoidingView
        style={styles.flex}
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
      >
        {content}
      </KeyboardAvoidingView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safe: {
    flex: 1,
    backgroundColor: colors.background,
  },
  flex: {
    flex: 1,

```

```

    },
    inner: {
      flex: 1,
    },
    padded: {
      paddingHorizontal: spacing.lg,
    },
    scrollContent: {
      flexGrow: 1,
    },
  },
});

```

mobile/src/components/notification/NotificationItem.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { Heart, MessageCircle, MessageSquare, Sparkles } from 'lucide-react-native';
import { Avatar } from '@components/ui/Avatar';
import { colors, spacing, typography } from '@theme';
import { relativeTime } from '@utils/formatDate';
import type { AppNotification, NotificationType } from '@types/notification.types';

type Props = {
  notification: AppNotification;
  onPress?: () => void;
};

export function NotificationItem({ notification, onPress }: Props) {
  const meta = describe(notification);

  return (
    <Pressable
      onPress={onPress}
      style={({ pressed }) => [
        styles.row,
        !notification.isRead && styles.unread,
        pressed && styles.pressed,
      ]
    >
      {notification.actor ? (
        <Avatar
          uri={notification.actor.avatarUrl}
          name={notification.actor.displayName}
          size={40}
        />
      ) : (
        <View style={[styles.avatar, { backgroundColor: colors.primarySoft }]}>
          <Sparkles size={20} color={colors.primaryDark} />
        </View>
      )}

      <View style={styles.body}>
        <Text style={styles.text}>
          {notification.actor && (
            <Text style={styles.actor}>{notification.actor.displayName}</Text>
          )}
          <Text style={styles.action}>{meta.text}</Text>
        </Text>
        <Text style={styles.time}>{relativeTime(notification.createdAt)}</Text>
      </View>

      <View style={[styles.iconBadge, { backgroundColor: meta.color + '22' }]}>
        {meta.icon}
      </View>

      {!notification.isRead && <View style={styles.dot} />}
    </Pressable>
  );
}

function describe(n: AppNotification): { text: string; icon: React.ReactNode; color: string } {
  const type = n.type as NotificationType;
  switch (type) {
    case 0:
      return {
        text: 'gönderini beğendi.',
        icon: <Heart size={16} color={colors.like} strokeWidth={2} />,
        color: colors.like,
      };
    case 1:
      return {
        text: 'gönderine yorum yaptı.',
        icon: <MessageCircle size={16} color={colors.primaryDark} strokeWidth={2} />,
        color: colors.primary,
      };
    case 2:

```

```

    return {
      text: 'yorumuna yanıt verdi.',
      icon: <MessageSquare size={16} color={colors.primaryDark} strokeWidth={2} />,
      color: colors.primary,
    };
  case 3:
    return {
      text: 'yorumunu beğendi.',
      icon: <Heart size={16} color={colors.like} strokeWidth={2} />,
      color: colors.like,
    };
  case 4:
  default:
    return {
      text: n.message ?? 'Yeni bir bildirim.',
      icon: <Sparkles size={16} color={colors.primaryDark} strokeWidth={2} />,
      color: colors.primary,
    };
  }
}

const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingVertical: spacing.md,
    paddingHorizontal: spacing.lg,
    gap: spacing.md,
    backgroundColor: colors.background,
  },
  unread: {
    backgroundColor: colors.surface,
  },
  pressed: { opacity: 0.85 },
  avatar: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  body: {
    flex: 1,
    gap: 2,
  },
  text: {
    ...typography.body,
    color: colors.text,
  },
  actor: {
    fontWeight: '600',
  },
  action: {
    color: colors.textSecondary,
  },
  time: {
    ...typography.caption,
    color: colors.textMuted,
  },
  iconBadge: {
    width: 28,
    height: 28,
    borderRadius: 14,
    alignItems: 'center',
    justifyContent: 'center',
  },
  dot: {
    width: 8,
    height: 8,
    borderRadius: 4,
    backgroundColor: colors.primary,
    marginLeft: spacing.xs,
  },
});

```

mobile/src/components/post/PostActions.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { Bookmark, Heart, MessageCircle } from 'lucide-react-native';
import { colors, spacing, typography } from '@theme';

type Props = {
  likeCount: number;
  isLiked: boolean;
  onLike: () => void;
};

```

```

    commentCount: number;
    onComment?: () => void;

    saveCount: number;
    isSaved: boolean;
    onSave: () => void;
  };

export function PostActions({
  likeCount,
  isLiked,
  onLike,
  commentCount,
  onComment,
  saveCount,
  isSaved,
  onSave,
}: Props) {
  return (
    <View style={styles.row}>
      <Action
        icon={
          <Heart
            size={20}
            color={isLiked ? colors.like : colors.textMuted}
            fill={isLiked ? colors.like : 'transparent'}
            strokeWidth={1.8}
          />
        }
        count={likeCount}
        active={isLiked}
        activeColor={colors.like}
        onPress={onLike}
      />
      <Action
        icon={<MessageCircle size={20} color={colors.textMuted} strokeWidth={1.8} />}
        count={commentCount}
        onPress={onComment}
      />
      <View style={{ flex: 1 }} />
      <Action
        icon={
          <Bookmark
            size={20}
            color={isSaved ? colors.save : colors.textMuted}
            fill={isSaved ? colors.save : 'transparent'}
            strokeWidth={1.8}
          />
        }
        count={saveCount}
        active={isSaved}
        activeColor={colors.save}
        onPress={onSave}
      />
    </View>
  );
}

function Action({
  icon,
  count,
  onPress,
  active,
  activeColor,
}: {
  icon: React.ReactNode;
  count: number;
  onPress?: () => void;
  active?: boolean;
  activeColor?: string;
}) {
  return (
    <Pressable
      onPress={onPress}
      hitSlop={8}
      style={({ pressed }) => [styles.action, pressed && { opacity: 0.6 }]}
    >
      {icon}
      <Text
        style={[
          styles.count,
          active && activeColor ? { color: activeColor, fontWeight: '600' } : null,
        ]}
      >
        {count}
      </Text>
    </Pressable>
  );
}

```

```

    );
  }

  const styles = StyleSheet.create({
    row: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: spacing.lg,
      marginTop: spacing.md,
    },
    action: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: 6,
    },
    count: {
      ...typography.caption,
      color: colors.textMuted,
      fontWeight: '500',
      minWidth: 16,
    },
  });
});

```

mobile/src/components/post/PostCard.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useRouter } from 'expo-router';
import { Avatar } from '@components/ui/Avatar';
import { CategoryChip } from '@components/category/CategoryChip';
import { PostActions } from './PostActions';
import { colors, radius, spacing, typography } from '@theme';
import { relativeTime } from '@utils/formatDate';
import { useLikePost, useSavePost } from '@hooks/usePosts';
import type { PostListItem } from '@types/post.types';

type Props = {
  post: PostListItem;
};

export function PostCard({ post }: Props) {
  const router = useRouter();
  const likeMutation = useLikePost();
  const saveMutation = useSavePost();

  const onOpen = () => router.push(`/post/${post.id}`);
  const onLike = () => likeMutation.mutate({ id: post.id, liked: post.isLikedByMe });
  const onSave = () => saveMutation.mutate({ id: post.id, saved: post.isSavedByMe });

  return (
    <Pressable
      onPress={onOpen}
      style={({ pressed }) => [styles.card, pressed && styles.pressed]}
    >
      <View style={styles.headerRow}>
        <CategoryChip name={post.category.name} color={post.category.color} />
        <Text style={styles.time}>{relativeTime(post.createdAt)}</Text>
      </View>

      <View style={styles.authorRow}>
        {post.isAnonymous || !post.author ? (
          <>
            <View style={styles.anonymousAvatar}>
              <Text style={styles.anonymousInitial}>?</Text>
            </View>
            <Text style={styles.authorName}>Anonim</Text>
          </>
        ) : (
          <>
            <Avatar uri={post.author.avatarUrl} name={post.author.displayName} size={28} />
            <Text style={styles.authorName} numberOfLines={1}>
              {post.author.displayName}
            </Text>
          </>
        )}
      </View>

      <Text style={styles.title} numberOfLines={2}>
        {post.title}
      </Text>
      {post.excerpt ? (
        <Text style={styles.excerpt} numberOfLines={3}>
          {post.excerpt}
        </Text>
      ) : null}
    </Pressable>
  );
}

```

```

        <PostActions
          likeCount={post.likeCount}
          isLiked={post.isLikedByMe}
          onLike={onLike}
          commentCount={post.commentCount}
          onComment={onOpen}
          saveCount={post.saveCount}
          isSaved={post.isSavedByMe}
          onSave={onSave}
        />
      </Pressable>
    );
  }

  const styles = StyleSheet.create({
    card: {
      backgroundColor: colors.surface,
      borderRadius: radius.lg,
      borderWidth: 1,
      borderColor: colors.border,
      padding: spacing.lg,
    },
    pressed: { opacity: 0.95 },
    headerRow: {
      flexDirection: 'row',
      alignItems: 'center',
      justifyContent: 'space-between',
      marginBottom: spacing.md,
    },
    time: {
      ...typography.caption,
      color: colors.textMuted,
    },
    authorRow: {
      flexDirection: 'row',
      alignItems: 'center',
      gap: spacing.sm,
      marginBottom: spacing.md,
    },
    anonymousAvatar: {
      width: 28,
      height: 28,
      borderRadius: 14,
      backgroundColor: colors.divider,
      alignItems: 'center',
      justifyContent: 'center',
    },
    anonymousInitial: {
      ...typography.label,
      color: colors.textMuted,
    },
    authorName: {
      ...typography.label,
      color: colors.textSecondary,
      flex: 1,
    },
    title: {
      ...typography.subtitle,
      color: colors.text,
      marginBottom: spacing.xs,
    },
    excerpt: {
      ...typography.body,
      color: colors.textSecondary,
    },
  });
});

```

mobile/src/components/ui/Avatar.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { Image, StyleSheet, Text, View } from 'react-native';
import { colors, radius } from '@theme';

type Props = {
  uri?: string | null;
  name?: string | null;
  size?: number;
};

export function Avatar({ uri, name, size = 40 }: Props) {
  const initial = (name?.trim()?.[0] ?? '?').toUpperCase();
  const fontSize = Math.round(size * 0.42);

  if (uri) {
    return (
      <Image

```

```

        source={{ uri }}
        style={[styles.image, { width: size, height: size, borderRadius: size / 2 }]}
      />
    );
  }

  return (
    <View
      style={[
        styles.fallback,
        { width: size, height: size, borderRadius: size / 2 },
      ]}
    >
      <Text style={[styles.initial, { fontSize }]}>{initial}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  image: {
    backgroundColor: colors.primarySoft,
  },
  fallback: {
    backgroundColor: colors.primarySoft,
    alignItems: 'center',
    justifyContent: 'center',
    borderRadius: radius.pill,
  },
  initial: {
    color: colors.primaryDark,
    fontWeight: '700',
  },
});
});

```

mobile/src/components/ui/Button.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { ActivityIndicator, Pressable, StyleSheet, Text, ViewStyle } from 'react-native';
import { colors, radius, spacing, typography } from '@theme';

type Variant = 'primary' | 'secondary' | 'ghost' | 'danger';
type Size = 'sm' | 'md' | 'lg';

type Props = {
  label: string;
  onPress?: () => void;
  variant?: Variant;
  size?: Size;
  loading?: boolean;
  disabled?: boolean;
  fullWidth?: boolean;
  style?: ViewStyle;
};

export function Button({
  label,
  onPress,
  variant = 'primary',
  size = 'md',
  loading = false,
  disabled = false,
  fullWidth = false,
  style,
}: Props) {
  const isDisabled = disabled || loading;

  return (
    <Pressable
      onPress={isDisabled ? undefined : onPress}
      style={({ pressed }) => [
        styles.base,
        sizeStyles[size],
        variantStyles[variant].container,
        fullWidth && styles.fullWidth,
        isDisabled && styles.disabled,
        pressed && !isDisabled && styles.pressed,
        style,
      ]}
    >
      {loading ? (
        <ActivityIndicator
          color={variant === 'primary' || variant === 'danger' ? colors.textOnPrimary : colors.primary}
        />
      ) : (
        <Text style={[typography.button, variantStyles[variant].label]}>{label}</Text>
      )}
    </Pressable>
  );
}

```

```

    </Pressable>
  );
}

const styles = StyleSheet.create({
  base: {
    alignItems: 'center',
    justifyContent: 'center',
    borderRadius: radius.lg,
    flexDirection: 'row',
  },
  fullWidth: { alignSelf: 'stretch' },
  disabled: { opacity: 0.5 },
  pressed: { opacity: 0.85, transform: [{ scale: 0.98 }] },
});

const sizeStyles: Record<Size, ViewStyle> = {
  sm: { paddingVertical: spacing.sm, paddingHorizontal: spacing.lg, minHeight: 36 },
  md: { paddingVertical: spacing.md, paddingHorizontal: spacing.xl, minHeight: 48 },
  lg: { paddingVertical: spacing.lg, paddingHorizontal: spacing.xl, minHeight: 56 },
};

const variantStyles: Record<Variant, { container: ViewStyle; label: { color: string } }> = {
  primary: {
    container: { backgroundColor: colors.primary },
    label: { color: colors.textOnPrimary },
  },
  secondary: {
    container: { backgroundColor: colors.primarySoft },
    label: { color: colors.primaryDark },
  },
  ghost: {
    container: { backgroundColor: 'transparent' },
    label: { color: colors.text },
  },
  danger: {
    container: { backgroundColor: colors.danger },
    label: { color: colors.textOnPrimary },
  },
};

```

mobile/src/components/ui/Card.tsx

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { PropsWithChildren } from 'react';
import { Pressable, StyleSheet, View, ViewStyle } from 'react-native';
import { colors, radius, spacing } from '@theme';

type Props = PropsWithChildren<{
  onPress?: () => void;
  style?: ViewStyle;
  padding?: keyof typeof spacing;
}>;

export function Card({ children, onPress, style, padding = 'lg' }: Props) {
  const padded = { padding: spacing[padding] };

  if (onPress) {
    return (
      <Pressable
        onPress={onPress}
        style={({ pressed }) => [styles.card, padded, pressed && styles.pressed, style]}
      >
        {children}
      </Pressable>
    );
  }

  return <View style={[styles.card, padded, style]}>{children}</View>;
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: colors.surface,
    borderRadius: radius.lg,
    borderWidth: 1,
    borderColor: colors.border,
  },
  pressed: {
    opacity: 0.92,
    transform: [{ scale: 0.995 }],
  },
});

```

mobile/src/components/ui/CategoryIcon.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import {
  Activity,
  CloudRain,
  Heart,
  HelpCircle,
  Leaf,
  Moon,
  User,
  Wind,
  Zap,
  type LucideIcon,
} from 'lucide-react-native';
import { colors } from '@theme';

const iconMap: Record<string, LucideIcon> = {
  wind: Wind,
  'cloud-rain': CloudRain,
  zap: Zap,
  user: User,
  activity: Activity,
  heart: Heart,
  leaf: Leaf,
  moon: Moon,
};

type Props = {
  name: string;
  size?: number;
  color?: string;
};

export function CategoryIcon({ name, size = 24, color = colors.primaryDark }: Props) {
  const Icon = iconMap[name] ?? HelpCircle;
  return <Icon size={size} color={color} strokeWidth={1.8} />;
}
```

mobile/src/components/ui/EmptyState.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { StyleSheet, Text, View } from 'react-native';
import { colors, spacing, typography } from '@theme';

type Props = {
  title: string;
  description?: string;
  icon?: React.ReactNode;
};

export function EmptyState({ title, description, icon }: Props) {
  return (
    <View style={styles.container}>
      {icon && <View style={styles.icon}><icon></View>}
      <Text style={styles.title}><title></Text>
      {description && <Text style={styles.description}><description></Text>}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    paddingVertical: spacing.xxxl,
    paddingHorizontal: spacing.xl,
    alignItems: 'center',
  },
  icon: {
    marginBottom: spacing.lg,
  },
  title: {
    ...typography.subtitle,
    color: colors.text,
    textAlign: 'center',
    marginBottom: spacing.sm,
  },
  description: {
    ...typography.body,
    color: colors.textMuted,
    textAlign: 'center',
  },
});
```

mobile/src/components/ui/Input.tsx

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { forwardRef, useState } from 'react';
import {
  StyleSheet,
  Text,
  TextInput,
  TextInputProps,
  View,
  ViewStyle,
} from 'react-native';
import { colors, radius, spacing, typography } from '@theme';

type Props = TextInputProps & {
  label?: string;
  error?: string;
  helper?: string;
  containerStyle?: ViewStyle;
};

export const Input = forwardRef<TextInput, Props>(function Input(
  { label, error, helper, containerStyle, style, onFocus, onBlur, ...rest },
  ref
) {
  const [focused, setFocused] = useState(false);
  const showError = !!error;

  return (
    <View style={[[styles.wrapper, containerStyle]]}>
      {label && <Text style={styles.label}>{label}</Text>}
      <View
        style={[
          styles.field,
          focused && !showError && styles.fieldFocused,
          showError && styles.fieldError,
        ]}
      >
        <TextInput
          ref={ref}
          {...rest}
          placeholderTextColor={colors.textMuted}
          style={[[styles.input, style]]}
          onFocus={(e) => {
            setFocused(true);
            onFocus?.(e);
          }}
          onBlur={(e) => {
            setFocused(false);
            onBlur?.(e);
          }}
        />
      </View>
      {showError ? (
        <Text style={styles.errorText}>{error}</Text>
      ) : helper ? (
        <Text style={styles.helperText}>{helper}</Text>
      ) : null}
    </View>
  );
});

const styles = StyleSheet.create({
  wrapper: {
    width: '100%',
  },
  label: {
    ...typography.label,
    color: colors.text,
    marginBottom: spacing.xs,
  },
  field: {
    borderWidth: 1.5,
    borderColor: colors.border,
    borderRadius: radius.md,
    backgroundColor: colors.surface,
    paddingHorizontal: spacing.lg,
    minHeight: 52,
    justifyContent: 'center',
  },
  fieldFocused: {
    borderColor: colors.primary,
    backgroundColor: colors.background,
  },
  fieldError: {
    borderColor: colors.danger,
    backgroundColor: colors.dangerSoft,
  },
});
```

```

    },
    input: {
      ...typography.body,
      color: colors.text,
      paddingVertical: 0,
    },
    helperText: {
      ...typography.caption,
      color: colors.textMuted,
      marginTop: spacing.xs,
    },
    errorText: {
      ...typography.caption,
      color: colors.danger,
      marginTop: spacing.xs,
    },
  },
});

```

mobile/src/components/ui/LoadingState.tsx

```

// Therapp | Hazırlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { ActivityIndicator, StyleSheet, View } from 'react-native';
import { colors, spacing } from '@theme';

type Props = { size?: 'small' | 'large' };

export function LoadingState({ size = 'large' }: Props) {
  return (
    <View style={styles.container}>
      <ActivityIndicator size={size} color={colors.primary} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    paddingVertical: spacing.xxxl,
  },
});

```

mobile/src/constants/disclaimer.ts

```

// Therapp | Hazırlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export const DISCLAIMER_TITLE = 'Önemli Bilgilendirme';

export const DISCLAIMER_BODY =
  'Therapp tıbbi tavsiye, teşhis veya tedavi sunmaz. Uygulamadaki içerikler ' +
  'kullanıcıların kişisel deneyim paylaşımlarıdır ve profesyonel sağlık ' +
  'hizmetinin yerini tutmaz. Kendinize veya çevrenizdeki birine zarar verme ' +
  'düşünceniz varsa lütfen derhal profesyonel destek alın veya 112\'yi arayın.';

export const DISCLAIMER_SHORT =
  'Bu içerikler tıbbi tavsiye değildir. Acil durumda profesyonel destek alın.';

```

mobile/src/constants/emergencyResources.ts

```

// Therapp | Hazırlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type EmergencyResource = {
  name: string;
  description: string;
  phone?: string;
  url?: string;
};

export const EMERGENCY_RESOURCES: EmergencyResource[] = [
  {
    name: 'Acil Çağrı Merkezi',
    description: 'Her türlü acil durum için 7/24 ulaşılabilir',
    phone: '112',
  },
  {
    name: 'İntihar Önleme Hattı',
    description: 'Ruhsal kriz anlarında destek hattı',
    phone: '182',
  },
  {
    name: 'KADES – Kadın Destek Uygulaması',
    description: 'Şiddete maruz kalan kadınlar için acil bildirim',
    url: 'https://kades.gov.tr',
  },
];

```

```
];
```

mobile/src/hooks/queryKeys.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import type { FeedSort } from '@types/post.types';

export const queryKeys = {
  categories: {
    all: ['categories'] as const,
    bySlug: (slug: string) => ['categories', slug] as const,
  },
  posts: {
    root: ['posts'] as const,
    feed: (sort: FeedSort) => ['posts', 'feed', sort] as const,
    byCategory: (slug: string, sort: FeedSort) => ['posts', 'category', slug, sort] as const,
    byId: (id: string) => ['posts', 'detail', id] as const,
    mine: (sort: FeedSort) => ['posts', 'me', sort] as const,
    saved: (sort: FeedSort) => ['posts', 'me', 'saved', sort] as const,
  },
  comments: {
    byPost: (postId: string) => ['comments', 'post', postId] as const,
    replies: (id: string) => ['comments', id, 'replies'] as const,
  },
  notifications: {
    list: ['notifications'] as const,
    unreadCount: ['notifications', 'unread-count'] as const,
  },
  users: {
    me: ['users', 'me'] as const,
    byUsername: (username: string) => ['users', username] as const,
  },
} as const;
```

mobile/src/hooks/useCategories.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useQuery } from '@tanstack/react-query';
import { categoriesApi } from '@api/categories.api';
import { queryKeys } from '../queryKeys';

export function useCategories() {
  return useQuery({
    queryKey: queryKeys.categories.all,
    queryFn: categoriesApi.list,
    staleTime: 5 * 60 * 1000,
  });
}

export function useCategory(slug: string | undefined) {
  return useQuery({
    queryKey: slug ? queryKeys.categories.bySlug(slug) : ['categories', 'noop'],
    queryFn: () => categoriesApi.bySlug(slug as string),
    enabled: !!slug,
    staleTime: 5 * 60 * 1000,
  });
}
```

mobile/src/hooks/useComments.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import {
  useInfiniteQuery,
  useMutation,
  useQueryClient,
  type QueryClient,
} from '@tanstack/react-query';
import { commentsApi } from '@api/comments.api';
import type { PagedResult } from '@types/api.types';
import type { Comment } from '@types/post.types';
import { queryKeys } from '../queryKeys';
import { bumpCommentCount } from '../usePosts';

type InfiniteComments = { pages: PagedResult<Comment>[]; pageParams: number[] };

// ----- Reads -----

export function useComments(postId: string | undefined) {
  return useInfiniteQuery({
    queryKey: postId ? queryKeys.comments.byPost(postId) : ['comments', 'noop'],
    queryFn: ({ pageParam }) =>
      commentsApi.byPost(postId as string, { page: pageParam, pageSize: 20 }),
    enabled: !!postId,
```

```

    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function useReplies(commentId: string | undefined, enabled = false) {
  return useInfiniteQuery({
    queryKey: commentId ? queryKeys.comments.replies(commentId) : ['comments', 'replies', 'noop'],
    queryFn: ({ pageParam }) =>
      commentsApi.replies(commentId as string, { page: pageParam, pageSize: 20 }),
    enabled: !!commentId && enabled,
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

// ----- Optimistic helpers -----

function patchCommentInList(qc: QueryClient, postId: string, commentId: string, patch: (c: Comment) => Comment) {
  qc.setQueryData<InfiniteComments>(queryKeys.comments.byPost(postId), (data) => {
    if (!data?.pages) return data;
    return {
      ...data,
      pages: data.pages.map((p) => ({
        ...p,
        items: p.items.map((c) => (c.id === commentId ? patch(c) : c)),
      })),
    };
  });
}

// ----- Mutations -----

export function useCreateComment(postId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (content: string) => commentsApi.create(postId, content),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: queryKeys.comments.byPost(postId) });
      bumpCommentCount(qc, postId, 1);
    },
  });
}

export function useCreateReply(postId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ parentCommentId, content }: { parentCommentId: string; content: string }) =>
      commentsApi.reply(parentCommentId, content),
    onSuccess: (_data, { parentCommentId }) => {
      qc.invalidateQueries({ queryKey: queryKeys.comments.replies(parentCommentId) });
      patchCommentInList(qc, postId, parentCommentId, (c) => ({ ...c, replyCount: c.replyCount + 1 }));
      bumpCommentCount(qc, postId, 1);
    },
  });
}

export function useDeleteComment(postId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => commentsApi.delete(id),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: queryKeys.comments.byPost(postId) });
      bumpCommentCount(qc, postId, -1);
    },
  });
}

export function useLikeComment(postId: string) {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, liked }: { id: string; liked: boolean }) => {
      if (liked) await commentsApi.unlike(id);
      else await commentsApi.like(id);
    },
    onMutate: async ({ id, liked }) => {
      await qc.cancelQueries({ queryKey: queryKeys.comments.byPost(postId) });
      const delta = liked ? -1 : 1;
      const next = !liked;
      patchCommentInList(qc, postId, id, (c) => ({
        ...c,
        isLikedByMe: next,
        likeCount: Math.max(0, c.likeCount + delta),
      }));
    },
    onError: () => {
      qc.invalidateQueries({ queryKey: queryKeys.comments.byPost(postId) });
    },
  });
}

```

```

    },
  });
}

```

mobile/src/hooks/useNotifications.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import {
  useInfiniteQuery,
  useMutation,
  useQuery,
  useQueryClient,
} from '@tanstack/react-query';
import { notificationsApi } from '@api/notifications.api';
import type { PagedResult } from '@types/api.types';
import type { AppNotification } from '@types/notification.types';
import { queryKeys } from './queryKeys';

type InfiniteNotifs = { pages: PagedResult<AppNotification>[]; pageParams: number[] };

export function useNotifications() {
  return useInfiniteQuery({
    queryKey: queryKeys.notifications.list,
    queryFn: ({ pageParam }) => notificationsApi.list({ page: pageParam, pageSize: 20 }),
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function useUnreadCount(enabled = true) {
  return useQuery({
    queryKey: queryKeys.notifications.unreadCount,
    queryFn: notificationsApi.unreadCount,
    enabled,
    refetchInterval: 60_000,
    staleTime: 30_000,
  });
}

export function useMarkRead() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => notificationsApi.markRead(id),
    onMutate: async (id) => {
      qc.setQueryData<InfiniteNotifs>(queryKeys.notifications.list, (data) => {
        if (!data?.pages) return data;
        return {
          ...data,
          pages: data.pages.map((p) => ({
            ...p,
            items: p.items.map((n) => (n.id === id ? { ...n, isRead: true } : n)),
          })),
        };
      });
    },
    qc.setQueryData<number>(queryKeys.notifications.unreadCount, (c) =>
      c && c > 0 ? c - 1 : 0
    ),
  },
  {
    onError: () => {
      qc.invalidateQueries({ queryKey: queryKeys.notifications.list });
      qc.invalidateQueries({ queryKey: queryKeys.notifications.unreadCount });
    },
  },
);
}

export function useMarkAllRead() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: () => notificationsApi.markAllRead(),
    onMutate: async () => {
      qc.setQueryData<InfiniteNotifs>(queryKeys.notifications.list, (data) => {
        if (!data?.pages) return data;
        return {
          ...data,
          pages: data.pages.map((p) => ({
            ...p,
            items: p.items.map((n) => ({ ...n, isRead: true })),
          })),
        };
      });
      qc.setQueryData<number>(queryKeys.notifications.unreadCount, 0);
    },
    onError: () => {
      qc.invalidateQueries({ queryKey: queryKeys.notifications.list });
      qc.invalidateQueries({ queryKey: queryKeys.notifications.unreadCount });
    },
  },
);
}

```

```

});
}

```

mobile/src/hooks/usePosts.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import {
  useInfiniteQuery,
  useMutation,
  useQuery,
  useQueryClient,
  type QueryClient,
} from '@tanstack/react-query';
import { postsApi, type CreatePostPayload, type UpdatePostPayload } from '@api/posts.api';
import type { PagedResult } from '@types/api.types';
import type { FeedSort, Post, PostListItem } from '@types/post.types';
import { queryKeys } from './queryKeys';

type InfiniteFeed = { pages: PagedResult<PostListItem>[]; pageParams: number[] };

// ----- Reads -----

export function useFeed(sort: FeedSort) {
  return useInfiniteQuery({
    queryKey: queryKeys.posts.feed(sort),
    queryFn: ({ pageParam }) => postsApi.feed({ sort, page: pageParam, pageSize: 20 }),
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function useCategoryFeed(slug: string | undefined, sort: FeedSort) {
  return useInfiniteQuery({
    queryKey: slug ? queryKeys.posts.byCategory(slug, sort) : ['posts', 'category', 'noop'],
    queryFn: ({ pageParam }) =>
      postsApi.byCategory(slug as string, { sort, page: pageParam, pageSize: 20 }),
    enabled: !!slug,
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function useMyPosts(sort: FeedSort = 'new') {
  return useInfiniteQuery({
    queryKey: queryKeys.posts.mine(sort),
    queryFn: ({ pageParam }) => postsApi.mine({ sort, page: pageParam, pageSize: 20 }),
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function useMySaved(sort: FeedSort = 'new') {
  return useInfiniteQuery({
    queryKey: queryKeys.posts.saved(sort),
    queryFn: ({ pageParam }) => postsApi.mySaved({ sort, page: pageParam, pageSize: 20 }),
    initialPageParam: 1,
    getNextPageParam: (last) => (last.hasNext ? last.page + 1 : undefined),
  });
}

export function usePost(id: string | undefined) {
  return useQuery({
    queryKey: id ? queryKeys.posts.byId(id) : ['posts', 'detail', 'noop'],
    queryFn: () => postsApi.byId(id as string),
    enabled: !!id,
  });
}

// ----- Optimistic helpers -----

type ListPatch = (item: PostListItem) => PostListItem;
type DetailPatch = (item: Post) => Post;

function patchListsAndDetail(qc: QueryClient, postId: string, listPatch: ListPatch, detailPatch: DetailPatch) {
  // Detail
  qc.setQueryData<Post>(queryKeys.posts.byId(postId), (old) => (old ? detailPatch(old) : old));

  // All infinite lists under ['posts']
  qc.setQueriesData<InfiniteFeed>({ queryKey: queryKeys.posts.root }, (data) => {
    if (!data?.pages) return data;
    return {
      ...data,
      pages: data.pages.map((p) => ({
        ...p,
        items: p.items.map((item) => (item.id === postId ? listPatch(item) : item)),
      })),
    };
  });
}

```

```

    })),
  });
});
}

// ----- Mutations -----

export function useLikePost() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, liked }: { id: string; liked: boolean }) => {
      if (liked) await postsApi.unlike(id);
      else await postsApi.like(id);
    },
    onMutate: async ({ id, liked }) => {
      await qc.cancelQueries({ queryKey: queryKeys.posts.root });
      const delta = liked ? -1 : 1;
      const next = !liked;
      patchListsAndDetail(
        qc,
        id,
        (item) => ({ ...item, isLikedByMe: next, likeCount: Math.max(0, item.likeCount + delta) }),
        (item) => ({ ...item, isLikedByMe: next, likeCount: Math.max(0, item.likeCount + delta) })
      );
    },
    onError: () => {
      // Roll back by refetching – simpler than snapshotting many caches
      qc.invalidateQueries({ queryKey: queryKeys.posts.root });
    },
  });
}

export function useSavePost() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({ id, saved }: { id: string; saved: boolean }) => {
      if (saved) await postsApi.unsave(id);
      else await postsApi.save(id);
    },
    onMutate: async ({ id, saved }) => {
      await qc.cancelQueries({ queryKey: queryKeys.posts.root });
      const delta = saved ? -1 : 1;
      const next = !saved;
      patchListsAndDetail(
        qc,
        id,
        (item) => ({ ...item, isSavedByMe: next, saveCount: Math.max(0, item.saveCount + delta) }),
        (item) => ({ ...item, isSavedByMe: next, saveCount: Math.max(0, item.saveCount + delta) })
      );
    },
    onSettled: (_d, _e, { id }) => {
      // Saved list might need refresh when toggled
      qc.invalidateQueries({ queryKey: ['posts', 'me', 'saved'] });
    },
    onError: () => {
      qc.invalidateQueries({ queryKey: queryKeys.posts.root });
    },
  });
}

export function useCreatePost() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (payload: CreatePostPayload) => postsApi.create(payload),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: queryKeys.posts.root });
      qc.invalidateQueries({ queryKey: queryKeys.categories.all });
    },
  });
}

export function useUpdatePost() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ id, payload }: { id: string; payload: UpdatePostPayload }) =>
      postsApi.update(id, payload),
    onSuccess: (data) => {
      qc.setQueryData(queryKeys.posts.byId(data.id), data);
      qc.invalidateQueries({ queryKey: queryKeys.posts.root });
    },
  });
}

export function useDeletePost() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: string) => postsApi.delete(id),
    onSuccess: () => {

```

```

    qc.invalidateQueries({ queryKey: queryKeys.posts.root });
  },
});
}

// Called from comment hooks to bump count without refetch
export function bumpCommentCount(qc: QueryClient, postId: string, delta: number) {
  qc.setQueryData<Post>(queryKeys.posts.byId(postId), (old) =>
    old ? { ...old, commentCount: Math.max(0, old.commentCount + delta) } : old
  );
  qc.setQueryData<InfiniteFeed>({ queryKey: queryKeys.posts.root }, (data) => {
    if (!data?.pages) return data;
    return {
      ...data,
      pages: data.pages.map((p) => ({
        ...p,
        items: p.items.map((item) =>
          item.id === postId
            ? { ...item, commentCount: Math.max(0, item.commentCount + delta) }
            : item
        ),
      })),
    };
  });
}

```

mobile/src/hooks/useReport.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useMutation } from '@tanstack/react-query';
import { reportsApi, type CreateReportPayload } from '@api/reports.api';

export function useCreateReport() {
  return useMutation({
    mutationFn: (payload: CreateReportPayload) => reportsApi.create(payload),
  });
}

```

mobile/src/hooks/useUser.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query';
import { usersApi, type UpdateProfilePayload } from '@api/users.api';
import { queryKeys } from './queryKeys';

export function useMe() {
  return useQuery({
    queryKey: queryKeys.users.me,
    queryFn: usersApi.me,
    staleTime: 60_000,
  });
}

export function useUpdateMe() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (payload: UpdateProfilePayload) => usersApi.updateMe(payload),
    onSuccess: (data) => {
      qc.setQueryData(queryKeys.users.me, data);
    },
  });
}

export function useDeleteMe() {
  return useMutation({
    mutationFn: () => usersApi.deleteMe(),
  });
}

```

mobile/src/store/authStore.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { create } from 'zustand';
import { authApi, AuthResponse } from '@api/auth.api';
import { configureAuth } from '@api/client';
import { deleteSecure, getSecure, SecureKeys, setSecure } from '@utils/secureStorage';
import { logger } from '@utils/logger';
import type { AuthUser } from '@types/user.types';

type AuthState = {
  user: AuthUser | null;
  accessToken: string | null;
  refreshToken: string | null;
};

```

```

hydrated: boolean;
isAuthenticated: boolean;

hydrate: () => Promise<void>;
setSession: (data: AuthResponse) => Promise<void>;
refresh: () => Promise<string | null>;
logout: () => Promise<void>;
clearSession: () => Promise<void>;
};

export const useAuthStore = create<AuthState>((set, get) => ({
  user: null,
  accessToken: null,
  refreshToken: null,
  hydrated: false,
  isAuthenticated: false,

  hydrate: async () => {
    const [access, refresh, userJson] = await Promise.all([
      getSecure(SecureKeys.AccessToken),
      getSecure(SecureKeys.RefreshToken),
      getSecure(SecureKeys.User),
    ]);

    let user: AuthUser | null = null;
    if (userJson) {
      try {
        user = JSON.parse(userJson) as AuthUser;
      } catch {
        user = null;
      }
    }

    set({
      accessToken: access,
      refreshToken: refresh,
      user,
      isAuthenticated: !!access && !!user,
      hydrated: true,
    });
  },

  setSession: async (data) => {
    await Promise.all([
      setSecure(SecureKeys.AccessToken, data.accessToken),
      setSecure(SecureKeys.RefreshToken, data.refreshToken),
      setSecure(SecureKeys.User, JSON.stringify(data.user)),
    ]);
    set({
      accessToken: data.accessToken,
      refreshToken: data.refreshToken,
      user: data.user,
      isAuthenticated: true,
    });
  },

  refresh: async () => {
    const rt = get().refreshToken;
    if (!rt) return null;
    try {
      const data = await authApi.refresh(rt);
      await get().setSession(data);
      return data.accessToken;
    } catch (e) {
      logger.warn('Token refresh failed', e);
      await get().clearSession();
      return null;
    }
  },

  logout: async () => {
    const rt = get().refreshToken;
    if (rt) {
      try {
        await authApi.logout(rt);
      } catch (e) {
        logger.warn('Logout API call failed', e);
      }
    }
    await get().clearSession();
  },

  clearSession: async () => {
    await Promise.all([
      deleteSecure(SecureKeys.AccessToken),
      deleteSecure(SecureKeys.RefreshToken),
      deleteSecure(SecureKeys.User),
    ]);
  }
});

```

```

    set({
      accessToken: null,
      refreshToken: null,
      user: null,
      isAuthenticated: false,
    });
  },
  {}));

// Wire the axios client to the store without creating an import cycle.
configureAuth({
  getToken: () => useAuthStore.getState().accessToken,
  refresh: () => useAuthStore.getState().refresh(),
  onAuthFailure: () => useAuthStore.getState().clearSession(),
});

```

mobile/src/store/uiStore.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { create } from 'zustand';
import { deleteSecure, getSecure, SecureKeys, setSecure } from '@/utils/secureStorage';

type UiState = {
  hasOnboarded: boolean;
  hydrated: boolean;

  hydrate: () => Promise<void>;
  setOnboarded: (value: boolean) => Promise<void>;
};

export const useUiStore = create<UiState>((set) => ({
  hasOnboarded: false,
  hydrated: false,

  hydrate: async () => {
    const value = await getSecure(SecureKeys.HasOnboarded);
    set({ hasOnboarded: value === '1', hydrated: true });
  },

  setOnboarded: async (value) => {
    if (value) {
      await setSecure(SecureKeys.HasOnboarded, '1');
    } else {
      await deleteSecure(SecureKeys.HasOnboarded);
    }
    set({ hasOnboarded: value });
  },
}));

```

mobile/src/theme/colors.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export const colors = {
  // Primary brand
  primary: '#42DCA3',
  primaryDark: '#2ABF88',
  primarySoft: '#D4F7E7',

  // Surfaces
  background: 'FFFFFF',
  surface: 'F1FFF8',
  surfaceAlt: 'F7FBF9',
  border: 'E1F4EA',
  divider: 'EDF5F0',

  // Text
  text: '12382B',
  textSecondary: '3D5C4F',
  textMuted: '6B8178',
  textOnPrimary: 'FFFFFF',
  textInverse: 'FFFFFF',

  // Semantic
  danger: 'E5484D',
  dangerSoft: 'FDEAEA',
  warning: 'F5A524',
  warningSoft: 'FFF4DD',
  info: '5DADE2',
  success: '#42DCA3',

  // Interactions
  like: 'FF6B81',
  likeSoft: 'FFE6EA',
  save: '#42DCA3',

```

```
// Overlays
overlay: 'rgba(18, 56, 43, 0.4)',
shadow: 'rgba(18, 56, 43, 0.08)',
} as const;

export type ColorKey = keyof typeof colors;
```

mobile/src/theme/index.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export { colors } from './colors';
export { spacing } from './spacing';
export { radius } from './radius';
export { typography } from './typography';

export type { ColorKey } from './colors';
export type { SpacingKey } from './spacing';
export type { RadiusKey } from './radius';
```

mobile/src/theme/radius.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export const radius = {
  sm: 8,
  md: 12,
  lg: 16,
  xl: 24,
  pill: 999,
} as const;

export type RadiusKey = keyof typeof radius;
```

mobile/src/theme/spacing.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export const spacing = {
  xs: 4,
  sm: 8,
  md: 12,
  lg: 16,
  xl: 24,
  xxl: 32,
  xxxl: 48,
} as const;

export type SpacingKey = keyof typeof spacing;
```

mobile/src/theme/typography.ts

```
// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { TextStyle } from 'react-native';

export const typography = {
  display: {
    fontSize: 32,
    lineHeight: 40,
    fontWeight: '700',
    letterSpacing: -0.5,
  } as TextStyle,
  title: {
    fontSize: 24,
    lineHeight: 32,
    fontWeight: '700',
    letterSpacing: -0.3,
  } as TextStyle,
  subtitle: {
    fontSize: 18,
    lineHeight: 24,
    fontWeight: '600',
  } as TextStyle,
  body: {
    fontSize: 16,
    lineHeight: 24,
    fontWeight: '400',
  } as TextStyle,
  bodyMedium: {
    fontSize: 16,
    lineHeight: 24,
    fontWeight: '500',
  } as TextStyle,
  caption: {
    fontSize: 13,
```

```

        lineHeight: 18,
        fontWeight: '400',
      } as TextStyle,
      label: {
        fontSize: 14,
        lineHeight: 20,
        fontWeight: '600',
      } as TextStyle,
      button: {
        fontSize: 16,
        lineHeight: 22,
        fontWeight: '600',
      } as TextStyle,
    };

```

mobile/src/types/api.types.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type ApiError = {
  code: string;
  message: string;
  errors?: Array<{ field: string; message: string }>;
};

export type PagedResult<T> = {
  items: T[];
  page: number;
  pageSize: number;
  totalCount: number;
  hasNext: boolean;
};

```

mobile/src/types/category.types.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type Category = {
  id: string;
  slug: string;
  name: string;
  description: string;
  icon: string;
  color: string;
  orderIndex: number;
  postCount: number;
};

```

mobile/src/types/notification.types.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type NotificationType =
  | 0 // PostLike
  | 1 // Comment
  | 2 // Reply
  | 3 // CommentLike
  | 4; // System

export type NotificationActor = {
  id: string;
  username: string;
  displayName: string;
  avatarUrl?: string | null;
};

export type AppNotification = {
  id: string;
  type: NotificationType;
  actor: NotificationActor | null;
  postId: string | null;
  commentId: string | null;
  message: string | null;
  isRead: boolean;
  createdAt: string;
};

```

mobile/src/types/post.types.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type PostAuthor = {
  id: string;
  username: string;
  displayName: string;
  avatarUrl?: string | null;
};

```

```

};

export type PostCategory = {
  id: string;
  slug: string;
  name: string;
  color: string;
  icon: string;
};

export type PostListItem = {
  id: string;
  title: string;
  excerpt: string;
  isAnonymous: boolean;
  category: PostCategory;
  author: PostAuthor | null;
  likeCount: number;
  commentCount: number;
  saveCount: number;
  createdAt: string;
  isLikedByMe: boolean;
  isSavedByMe: boolean;
};

export type Post = {
  id: string;
  title: string;
  content: string;
  isAnonymous: boolean;
  category: PostCategory;
  author: PostAuthor | null;
  likeCount: number;
  commentCount: number;
  saveCount: number;
  createdAt: string;
  updatedAt: string;
  isLikedByMe: boolean;
  isSavedByMe: boolean;
  isMine: boolean;
};

export type CommentAuthor = {
  id: string;
  username: string;
  displayName: string;
  avatarUrl?: string | null;
};

export type Comment = {
  id: string;
  postId: string;
  parentCommentId: string | null;
  content: string;
  author: CommentAuthor;
  likeCount: number;
  replyCount: number;
  isLikedByMe: boolean;
  isMine: boolean;
  createdAt: string;
};

export type FeedSort = 'new' | 'popular';

```

mobile/src/types/user.types.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
export type AuthUser = {
  id: string;
  username: string;
  email: string;
  displayName: string;
  avatarUrl?: string | null;
  role: number;
};

export type UserMe = AuthUser & {
  bio?: string | null;
  createdAt: string;
  postCount: number;
  savedCount: number;
};

export type UserProfile = {
  id: string;
  username: string;

```

```

    displayName: string;
    avatarUrl?: string | null;
    bio?: string | null;
    createdAt: string;
    postCount: number;
  };

```

mobile/src/utils/formatDate.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import { formatDistanceToNowStrict } from 'date-fns';
import { tr } from 'date-fns/locale';

export function relativeTime(iso: string): string {
  try {
    return formatDistanceToNowStrict(new Date(iso), { addSuffix: true, locale: tr });
  } catch {
    return '';
  }
}

export function shortDate(iso: string): string {
  try {
    return new Date(iso).toLocaleDateString('tr-TR', {
      day: '2-digit',
      month: 'short',
      year: 'numeric',
    });
  } catch {
    return '';
  }
}

```

mobile/src/utils/getApiUrl.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import Constants from 'expo-constants';

export function getApiUrl(): string {
  const envUrl = process.env.EXPO_PUBLIC_API_URL;
  if (envUrl && envUrl.trim().length > 0) return envUrl;

  const extra = (Constants.expoConfig?.extra ?? {}) as { apiUrl?: string };
  return extra.apiUrl ?? 'http://localhost:5080/api';
}

```

mobile/src/utils/logger.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
const isDev = __DEV__;

export const logger = {
  debug: (...args: unknown[]) => isDev && console.log('[debug]', ...args),
  info: (...args: unknown[]) => isDev && console.log('[info]', ...args),
  warn: (...args: unknown[]) => console.warn('[warn]', ...args),
  error: (...args: unknown[]) => console.error('[error]', ...args),
};

```

mobile/src/utils/secureStorage.ts

```

// Therapp | Hazirlayanlar: Gaye Gorkem [No], Rabia Nur Gun [No]
import * as SecureStore from 'expo-secure-store';

export const SecureKeys = {
  AccessToken: 'therapp_access_token',
  RefreshToken: 'therapp_refresh_token',
  User: 'therapp_user',
  HasOnboarded: 'therapp_has_onboarded',
} as const;

export type SecureKey = (typeof SecureKeys)[keyof typeof SecureKeys];

export async function getSecure(key: SecureKey): Promise<string | null> {
  try {
    return await SecureStore.getItemAsync(key);
  } catch {
    return null;
  }
}

export async function setSecure(key: SecureKey, value: string): Promise<void> {
  await SecureStore.setItemAsync(key, value);
}

```

```
}  
  
export async function deleteSecure(key: SecureKey): Promise<void> {  
  try {  
    await SecureStore.deleteItemAsync(key);  
  } catch {  
    // ignore  
  }  
}
```

mobile/tsconfig.json

```
{  
  "extends": "expo/tsconfig.base",  
  "compilerOptions": {  
    "strict": true,  
    "baseUrl": ".",  
    "paths": {  
      "@/*": ["src/*"]  
    },  
    "jsx": "react-native",  
    "moduleResolution": "bundler",  
    "esModuleInterop": true,  
    "skipLibCheck": true,  
    "resolveJsonModule": true,  
    "allowSyntheticDefaultImports": true  
  },  
  "include": [  
    "**/*.ts",  
    "**/*.tsx",  
    ".expo/types/**/*.ts",  
    "expo-env.d.ts"  
  ]  
}
```

KAYNAKÇA

- [1] Microsoft. (2024). ASP.NET Core documentation. <https://learn.microsoft.com/aspnet/core>
- [2] Microsoft. (2024). Entity Framework Core documentation. <https://learn.microsoft.com/ef/core>
- [3] PostgreSQL Global Development Group. (2024). PostgreSQL 15 documentation. <https://www.postgresql.org/docs/15/>
- [4] Meta Platforms. (2024). React Native documentation. <https://reactnative.dev>
- [5] Expo. (2024). Expo documentation (SDK 54). <https://docs.expo.dev>
- [6] TanStack. (2024). TanStack Query documentation. <https://tanstack.com/query>
- [7] Auth0 / JWT.io. (2024). Introduction to JSON Web Tokens. <https://jwt.io/introduction>
- [8] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- [9] FluentValidation. (2024). FluentValidation documentation. <https://docs.fluentvalidation.net>
- [10] OWASP Foundation. (2021). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- [11] Provos, N. & Mazières, D. (1999). A Future-Adaptable Password Scheme (bcrypt). USENIX Annual Technical Conference.